

Acknowledgements

Before presenting this work, I would like to express my deep gratitude to everyone who, in one way or another, contributed to its completion.

First and foremost, I would like to express my deepest and most sincere gratitude to my academic supervisor, **Mr. Haythem Ghazouani**, for his exceptional guidance, his constant availability, his insightful advice, and his unwavering support throughout every stage of this project. His expertise, patience, and encouragement were truly invaluable, and I owe him a great deal of thanks for everything he has done to help me complete this work in the best possible conditions.

I would also like to thank my professional supervisor, **Mr. Abdelkhalek Bakkari**, for his guidance and support during this internship.

My sincere thanks also go to **Ms. Insaf**, for her valuable technical guidance throughout this project, her advice on AI-related design decisions, and her availability whenever model behaviour or evaluation results required refinement.

My thanks also go to the entire teaching staff for the quality of the education provided throughout my studies, as well as to the members of the jury for the interest they showed in this work and the time they dedicated to its evaluation.

Finally, I would like to thank everyone who, through their advice, encouragement, or support, made it possible to carry out this project under the best possible conditions.

Table of Contents

Dedication	ii
Acknowledgements	iii
GENERAL INTRODUCTION	1
1 General Project Context	3
1.1 Presentation of the Host Organization and the SUBUL Platform	4
1.2 The EdTech and HRTech Landscape : Context and Challenges	5
1.2.1 Educational Technology : A Growing Market for Skill Development	5
1.2.2 Human Resources Technology : Digitalization of Talent Acquisition	6
1.2.3 Smartovate at the Intersection of Both Markets	7
1.3 Business Model and Personalized Career Guidance	7
1.4 Design Constraints and Project Challenges	9
2 Project Analysis and Requirements Specification	12
Introduction	13
2.1 Problem Statement	13
2.1.1 The Invisible Barrier of Automated Screening	13
2.1.2 The Fragmentation of Job Market Information	14
2.1.3 The Absence of Semantic Profile-to-Offer Matching	14
2.2 Study of Existing Solutions	15
2.2.1 Analysis of Existing Tools	15
2.2.2 Comparative Positioning	17
2.3 Project Objectives	17
2.4 Project Scope	18
2.5 Actors Identification	18
2.6 Functional Requirements	19
2.6.1 CV Agent	19
2.6.2 Job Agent	20
2.6.3 ATS Agent	21
2.6.4 Conversational Assistant	22
2.7 Non-Functional Requirements	22

2.8	Use Case Modeling	24
2.8.1	General Use Case Diagram	24
2.8.2	Use Case Descriptions	24
2.9	Expected Deliverables	36
	Conclusion	37
3	Methodology and Development Approach	38
	Introduction	39
3.1	Why an Agentic AI Architecture	39
3.1.1	Limitations of a Single LLM Pipeline	39
3.1.2	Limitations of Conventional Software Development without AI	39
3.1.3	The Adopted Position : Agentic AI Orchestrated with LangGraph	40
3.1.4	Comparison of Agent Orchestration Frameworks	40
3.2	Software Development Methodology : Scrum	40
3.2.1	Comparative Analysis of Development Methodologies	40
3.2.2	Justification of the Scrum Choice	42
3.3	Data Science Methodology : CRISP-ML(Q)	42
3.3.1	Comparative Analysis of Data-Science Methodologies	42
3.3.2	Justification of the CRISP-ML(Q) Choice	43
3.4	Hybrid Methodology : Combining Scrum and CRISP-ML(Q)	44
3.4.1	Genesis of the Hybrid Approach	44
3.4.2	Rationale for Combining Both Methodologies	44
3.4.3	Product Backlog Organized by Epic	45
3.4.4	Sprint ↔ CRISP-ML(Q) Phase Mapping	46
3.4.5	Assessment of the Hybrid Approach	47
3.5	CRISP-ML(Q) Phases Applied to the Project	48
3.5.1	Business and Data Understanding	48
3.5.2	Data Preparation	48
3.5.3	Modeling	48
3.5.4	Evaluation	49
3.5.5	Deployment	49
3.5.6	Monitoring and Maintenance	49
3.6	Global System Architecture	49
3.7	Multi-Agent Architecture	50

3.7.1	The Supervisor Agent	50
3.7.2	The CV Agent	51
3.7.3	The Job Agent	51
3.7.4	The ATS Agent	51
3.7.5	Communication Inter-Agents	51
3.8	Data Flow	52
3.9	Technology Stack	52
3.9.1	Software and Frameworks	53
3.9.2	Large Language Model Provider : OpenAI	54
3.9.3	Cloud Platform : Microsoft Azure	57
3.9.4	Machine Learning and AI Orchestration Frameworks	59
3.9.5	Specific Machine Learning Models Used	60
3.9.6	Databases	61
3.9.7	Document Extraction and Generation	61
3.9.8	Web Scraping	62
	Conclusion	63
4	Detailed Design	64
	Introduction	65
4.1	CV Agent Pipeline Design	65
4.1.1	Node-by-Node Description	66
4.1.2	Conditional Retry Logic	67
4.2	Job Agent Pipeline Design	68
4.2.1	Node-by-Node Description	69
4.3	ATS Agent Pipeline Design	72
4.3.1	Node-by-Node Description	73
4.3.2	Job-Specific CV Enhancement Endpoint	74
4.4	The ATS Scoring Formula	75
4.5	Data Model	76
4.5.1	Azure Cosmos DB Entities	76
4.5.2	Azure PostgreSQL Entities	77
4.6	Prompt Engineering and Anti-Hallucination Safeguards	77
4.6.1	Structured Prompt Design	77
4.6.2	The Anti-Hallucination Guard	78

Conclusion	79
5 Fine-tuned CV-to-Job Matching Model	80
Introduction	81
5.1 Business and Data Understanding	81
5.2 Data Preparation	81
5.2.1 Ground-Truth Scoring Function	81
5.2.2 Pair Construction and Split	82
5.3 Modeling	83
5.3.1 Architecture	83
5.3.2 Training Configuration	84
5.4 Evaluation	84
5.4.1 Regression Performance on the Held-Out Test Set	84
5.4.2 Qualitative Benchmark Suite	86
5.4.3 Baseline Comparison and Ranking Evaluation	90
5.5 Deployment	96
5.6 Monitoring and Maintenance	97
Conclusion	97
6 Implementation	98
Introduction	99
6.1 Technical Environment	99
6.1.1 Development Tools	99
6.1.2 Execution Environment and Cloud AI/ML Services	99
6.1.3 Project Management Tooling	99
6.2 Product Backlog and User Stories	100
6.2.1 Product Backlog by Feature	100
6.2.2 User Stories	103
6.3 Chronological Development Iterations	108
6.4 Reliability Best Practices	111
6.4.1 Traceability	112
6.4.2 Retry Mechanisms	112
6.4.3 Guardrails	112
6.5 Learner Journey Walkthrough	112
Conclusion	119

7 Testing, Evaluation and Results	120
Introduction	121
7.1 Testing Strategy	121
7.1.1 Approach	121
7.1.2 Scope and Boundaries	122
7.2 Consolidated Evaluation Results	123
7.2.1 Fine-Tuned Matching Model — Synthesis	123
7.2.2 ATS Scoring — Before and After Improvement	123
7.3 Identified Limitations	124
Conclusion	126
General Conclusion and Perspectives	127
Bibliography & Webography	130

List of Figures

1.1	Smartovate Logo	4
1.2	Growth of the global EdTech market between 2019 and 2025 (Source: HolonIQ, 2020).	6
1.3	Smartovate’s positioning at the intersection of EdTech and HRTech	7
1.4	The SUBUL learner journey: from assessment to employment	9
2.1	Comparative feature matrix: existing solutions vs. AI Career Assistant (full, partial, or absent)	17
2.2	SMART-based definition for the AI Career Assistant system.	18
2.3	Project scope: in scope and out of scope boundaries	18
2.4	Actors of the AI Career Assistant system	19
2.5	General use case diagram of the AI Career Assistant system	24
2.6	AI Career Assistant system deliverables overview	37
3.1	Articulation between the macro-level Scrum timeline and the CRISP-ML(Q) phases applied to the matching-model sub-project. The Data Preparation phase shown under Sprint 3 refers specifically to the construction of the model’s labeled training dataset (positive/negative CV–job pairs), distinct from the general CV text extraction already covered under Sprint 1 in Table 3.4. Business & Data Understanding and Monitoring & Maintenance are addressed at the project-wide level or discussed as future work (Section 3.5.6), respectively.	44
3.2	CRISP-ML(Q) cycle applied to the AI Career Assistant project	48
3.3	Global architecture of the AI Career Assistant system	50
3.4	Communication flow between the Supervisor Agent and its specialized agents	52
3.5	Data flow across the AI Career Assistant system, from raw CV upload to the final career report	52
3.6	Python	53
3.7	FastAPI	53
3.8	Uvicorn	53
3.9	React	53
3.10	Next.js	54
3.11	TypeScript	54
3.12	GitHub	54

3.13	OpenAI	54
3.14	Microsoft Azure	57
3.15	LangGraph	59
3.16	sentence-transformers	60
3.17	PyTorch	60
3.18	scikit-learn	60
3.19	Azure Cosmos DB	61
3.20	Azure PostgreSQL	61
3.21	PyMuPDF	61
3.22	ReportLab	62
3.23	aiohttp	62
3.24	BeautifulSoup	62
3.25	SerpAPI	62
4.1	State diagram of the CV Agent’s eight-node pipeline	66
4.2	Sequence diagram of the resume upload, scoring, enrichment, and persistence flow . . .	68
4.3	State diagram of the Job Agent’s eight-node pipeline	69
4.4	Architecture of the compatibility scoring and LLM-based explanation generation pipeline	71
4.5	Sequence diagram of the multi-source scan pipeline	72
4.6	State diagram of the ATS Agent’s six-node pipeline	73
4.7	Sequence diagram of the ATS Agent’s job-specific scoring pipeline	74
4.8	Class diagram of the entities persisted in Azure Cosmos DB and Azure PostgreSQL . .	76
4.9	Internal decision logic of the anti-hallucination guard	78
5.1	Training curves: loss, validation MAE, and validation R^2 over 10 epochs	85
5.2	Regression performance on the held-out test set	85
5.3	Discrimination benchmark console output	86
5.4	Adversarial resistance benchmark console output	87
5.5	Coherence test console output	88
5.6	Linguistic robustness test console output	89
5.7	Calibration test console output	89
5.8	TF-IDF vs fine-tuned MiniLM: scores, ranking, cross-validation, and pipeline demonstration	91
5.9	Console output: baseline separation, TF-IDF vs MiniLM	91
5.10	Console output: Precision@K, Recall@K, and NDCG@K	93
5.11	Console output: 5-fold cross-validation, TF-IDF vs MiniLM	94

5.12	Console output: ranked job postings for a fixed CV	95
6.1	Scrum backlog organization in ClickUp: (a) Epics board view, (b) Epics list view, and (c) sub-task detail view.	100
6.2	Project roadmap and sprint-aligned Gantt chart (February July 2026)	111
6.3	CV upload and enrichment journey: (a) upload screen, (b) enrichment screen for missing sections, and (c) confirmation screen once the profile is saved.	113
6.4	The Matches view, listing job postings ranked by score, with an expanded explanation panel showing the score breakdown and skill coverage for one listing	114
6.5	Skill gap analysis	114
6.6	Learning roadmap: phased plan with per-skill duration, project ideas, and sequencing rationale	115
6.7	Market overview: key indicators and top skills demanded	115
6.8	Market overview: top companies and top locations	116
6.9	Consolidated career report (PDF)	117
6.10	Job-specific ATS scoring screen, showing the six-dimension compatibility breakdown	117
6.11	Job-specific CV enhancement screen	118
6.12	Conversational assistant interface	119
7.1	Home page of the SUBUL AI Career Assistant	122

List of Tables

1.1	Smartovate Company overview	5
1.2	AI agents within the SUBUL platform ecosystem	8
1.3	SUBUL subscription plans and pricing (TND Tunisia)	9
1.4	Constraints identified within the SUBUL environment	10
2.1	Analysis of existing solutions in the professional development ecosystem	16
2.2	Functional requirements CV Agent	20
2.3	Functional requirements Job Agent	21
2.4	Functional requirements ATS Agent	22
2.5	Functional requirements Conversational Assistant	22
2.6	Non-functional requirements	23
2.7	Summary of primary use cases	26
2.8	UC01 Upload CV and Extract Profile	27
2.9	UC02 Enhance CV and Generate Canonical Version	28
2.10	UC03 Search and Match Job Opportunities	29
2.11	UC04 Analyze Skill Gaps	30
2.12	UC05 Analyze Job Market Trends	31
2.13	UC06 Generate a Personalized Learning Roadmap	31
2.14	UC07 Generate a PDF Report	32
2.15	UC08 Evaluate CV ATS Baseline Score	33
2.16	UC09 Tailor CV for Specific Job and Export	34
2.17	UC10 Interact with Career Assistant Chatbot	35
2.18	UC11 Monitor System Health	36
3.1	Comparison of development methodologies according to their relevance to this project	42
3.2	Comparison of data-science methodologies	43
3.3	Product backlog organized by epic	45
3.4	Correspondence between development iterations and CRISP-ML(Q) phases	47
3.5	Comparison of large language model providers by criterion	57
3.6	Comparison of cloud platforms on the criteria relevant to this project	59
4.1	Weighted dimensions of the deterministic ATS scoring formula	75

5.1	Weighted factors of the deterministic ground-truth scoring function	82
5.2	Regression performance on the held-out test set	85
5.3	Ranking metrics: TF-IDF baseline vs fine-tuned MiniLM, learner’s perspective (n=44)	94
6.1	Product backlog: features by epic, with MoSCoW priority (M = Must Have, S = Should Have, C = Could Have)	103
6.2	Product backlog: user stories by sprint and associated epic, with MoSCoW priority (M = Must Have, S = Should Have, C = Could Have)	107
6.3	The five development sprints and their main deliverables	109
6.4	Scrum roles and ceremonies as applied on this project	110
7.1	Consolidated fine-tuned matching model evaluation results (full detail in Chapter 5) .	123
7.2	Consolidated limitations identified across the system	125

List of Abbreviations

Large Language Model Server-Sent Events Unified Modeling Language Application Programming Interface Representational State Transfer Portable Document Format Office Open XML Document Plain Text file Use Case Functional Requirement Non-Functional Requirement Mean Absolute Error Specific, Measurable, Achievable, Relevant, Time-bound Asynchronous Server Gateway Interface Cross-Industry Standard Process for Data Mining Cross-Industry Standard Process for Machine Learning with Quality assurance HyperText Transfer Protocol JavaScript Object Notation Knowledge Discovery in Databases Not only SQL Team Data Science Process Explainable Artificial Intelligence Extreme Programming Central Processing Unit Discounted Cumulative Gain Graphics Processing Unit Ideal Discounted Cumulative Gain Normalized Discounted Cumulative Gain Root Mean Square Error Term Frequency–Inverse Document Frequency

— = LLM

— = SSE

— = UML

— = API

— = REST

— = PDF

— = DOCX

— = TXT

— = UC

— = FR

— = NFR

— = MAE

— = SMART

— = ASGI

— = CRISP-DM

— = CRISP-ML(Q)

— = HTTP

— = JSON

— = KDD

—	=	NoSQL
—	=	TDSP
—	=	XAI
—	=	XP
—	=	CPU
—	=	DCG
—	=	GPU
—	=	IDCG
—	=	NDCG
—	=	RMSE
—	=	TF-IDF
—	AI	= Artificial Intelligence
—	ATS	= Applicant Tracking System
—	AWS	= Amazon Web Services
—	CEO	= Chief Executive Officer
—	COO	= Chief Operating Officer
—	CTO	= Chief Technology Officer
—	CV	= Curriculum Vitae
—	EdTech	= Educational Technology
—	HRTech	= Human Resources Technology
—	IT	= Information Technology
—	JD	= Job Description
—	LMS	= Learning Management System
—	ML	= Machine Learning
—	MoSCoW	= Must have, Should have, Could have, Won't have
—	NLP	= Natural Language Processing
—	QA	= Quality Assurance
—	TND	= Tunisian Dinar
—	UI	= User Interface
—	UX	= User Experience

GENERAL INTRODUCTION

The job market has undergone a fundamental transformation over the past decade. Job searching is no longer limited to browsing listings: today, before any human review takes place, applications are filtered by Applicant Tracking Systems (ATS) that automatically sort, score, and discard a significant portion of received resumes. For candidates, this means that an otherwise qualified profile can be rejected not due to a lack of skills, but because of incompatible wording, structure, or vocabulary that fails to meet the criteria set by these algorithms. This gap between a candidate's actual competence and their resume's readability by a machine has become one of the most significant yet invisible barriers in today's job search process.

At the same time, the proliferation of recruitment platforms (LinkedIn, Indeed, Glassdoor, company career pages, specialized aggregators, etc.) has made monitoring relevant opportunities both time-consuming and fragmented: candidates must navigate multiple sources, manually compare each listing against their own profile, and assess their chances on their own before applying. This scattered landscape, combined with the lack of tools capable of intelligently cross-referencing a profile with a job posting, places a substantial analytical burden on the candidate, often without any objective feedback on how well their application actually fits the role.

It is within this context that the present final year project takes shape. It consists of designing and developing an intelligent job search assistance platform that combines automatic resume extraction and structuring, semantic CV-to-job compatibility analysis (ATS scoring), multi-source job aggregation through web scraping, and personalized recommendation generation, all orchestrated by an architecture of autonomous agents powered by Large Language Models (LLMs) and the LangGraph framework. The objective is to provide candidates with a unified tool capable not only of objectively diagnosing the quality of their resume against a given job description, but also of explaining that diagnosis transparently and suggesting job opportunities genuinely aligned with their profile, thereby reducing the uncertainty that typically accompanies every application submitted.

From a technical standpoint, this project required the mobilization of competencies spanning applied artificial intelligence, natural language processing, and software engineering. Each component was designed, tested, and integrated following an iterative approach, with a constant focus on reliability, explainability of the produced results, and user experience.

This report documents the complete process followed throughout the development of this project and is organized into seven chapters. The first chapter, entitled "*General Project Context*,"

presents the framework within which this work is situated, the host organization, and the motivation behind the project. The second chapter, “*Project Analysis and Requirements Specification*,” formally frames the problem being addressed, analyzes existing solutions, and specifies the system’s actors, functional and non-functional requirements, and use cases. The third chapter, “*Methodology and Development Approach*,” justifies the adoption of an agentic AI architecture and presents the hybrid project methodology combining Scrum for iterative software delivery with CRISP-ML(Q) for the data-science and machine-learning components, before introducing the system’s global architecture. The fourth chapter, “*Detailed Design*,” details the internal design of the system’s core pipelines, its data model, and its prompt-engineering and safeguard mechanisms. The fifth chapter is entirely dedicated to the fine-tuned CV-to-job matching model, structured according to the CRISP-ML(Q) phases applied specifically to this artifact. The sixth chapter, “*Implementation*,” presents the technical environment and the chronological development iterations that produced the platform’s interfaces and features. Finally, the seventh chapter, “*Testing, Evaluation, and Results*,” presents the testing strategy, the evaluation of the matching model and the ATS scoring mechanism, and the limitations identified. The report concludes with a general conclusion that summarizes the work accomplished and outlines future perspectives for the project’s evolution.

It should be noted that this report combines two complementary readings of the same work: a chronological reading, organized around the successive development iterations, for the parts of the report concerned with project management and application delivery, and a methodological reading, structured according to the CRISP-ML(Q) process model, presented transversally in the chapter dedicated to the CV-to-job matching model.

GENERAL PROJECT CONTEXT

Introduction

This chapter introduces the organizational and market context in which the present project was carried out. It presents Smartovate, its positioning at the intersection of the EdTech and HRTech industries, its flagship platform SUBUL, and the business model that underpins its value proposition. It also examines the broader landscape of online learning and digital recruitment, highlighting the strategic context that motivated the development of the present project. Finally, it explains why artificial intelligence represents a concrete and timely opportunity to bridge this gap, and outlines the constraints that shaped the early decisions of the project.

1.1 Presentation of the Host Organization and the SUBUL Platform

Smartovate is a London-based technology company specializing in the development of **Artificial Intelligence (AI)**-powered solutions for learning, certification, and digital transformation [1]. The company focuses on delivering innovative technologies in the fields of **Artificial Intelligence (AI)**, **Cloud Computing**, **Cybersecurity**, and **Intelligent Learning Platforms**.

Founded with the vision of making advanced technology education more accessible, personalized, and effective, Smartovate develops solutions that help learners and organizations acquire in-demand skills while supporting their digital transformation.

The founding team brings together complementary expertise in business strategy, operations, and technology. **Abdelkhalek Bakkari, CEO and Founder**, defines the company's strategic vision and leads its overall development. **Taoufik Baccari, COO and Co-Founder**, oversees business operations and ensures the efficient delivery of the company's services. **Sausan Al Suti, CTO and Co-Founder**, is responsible for the technical architecture and leads the development of the AI-powered technologies that constitute the core of Smartovate's solutions.



Figure 1.1: Smartovate Logo

Smartovate's flagship platform, **SUBUL**, is a hybrid intelligent learning system that personalizes education through AI-driven adaptive learning experiences. It covers three major training domains: artificial intelligence, cloud computing, and cybersecurity. The platform provides learners with theoretical content, hands-on laboratories, and certification preparation aligned with globally recognized programs.

The company has established strategic partnerships with leading technology companies including **Microsoft** [2], **Amazon Web Services (AWS)** [3], **Google** [4], **NVIDIA** [5], and **IBM** [6], enabling it to deliver training content and certification paths that are recognized across the global technology industry.

Attribute	Details
Headquarters	71-75 Shelton Street, Covent Garden, London, WC2H 9JQ, United Kingdom
Leadership	Abdelkhalek Bakkari (CEO & Founder), Taoufik Baccari (COO & Co-Founder), Sausan Al Suti (CTO & Co-Founder)
Domain of Activity	AI-powered training and certification in Artificial Intelligence, Cloud Computing, and Cybersecurity
Flagship Product	SUBUL Hybrid Intelligent Learning Platform
Target Market	French-speaking tech professionals, MENA region, North Africa
Technology Partners	Microsoft, Amazon (AWS), Google, NVIDIA
Key Figures	10,000+ active learners, 500+ available courses, 200+ business partnerships, 95% satisfaction rate [7]

Tableau 1.1: Smartovate Company overview

The positioning of Smartovate is therefore both educational and professional. The company does not simply deliver courses: it aims to accompany each learner through a complete career development journey, from initial skill assessment and structured training to professional certification and, ultimately, access to employment opportunities. This dual ambition training and employability is what defines the strategic context in which the present project was initiated.

1.2 The EdTech and HRTech Landscape: Context and Challenges

To understand the context in which this project operates, it is essential to examine the two industries whose intersection defines the strategic positioning of Smartovate: educational technology (EdTech) and human resources technology (HRTech).

1.2.1 Educational Technology: A Growing Market for Skill Development

Educational technology, commonly referred to as EdTech, encompasses the set of digital tools, platforms, and systems designed to facilitate and enhance learning experiences. It includes online

learning platforms, learning management systems (LMS), adaptive learning environments, certification programs, virtual laboratories, and AI-powered tutoring tools.

As illustrated in Figure 1.2, the global EdTech market has experienced sustained growth in recent years, with HolonIQ estimating total global expenditure to reach \$404 billion by 2025, representing a 16.3% compound annual growth rate between 2019 and 2025 [8].

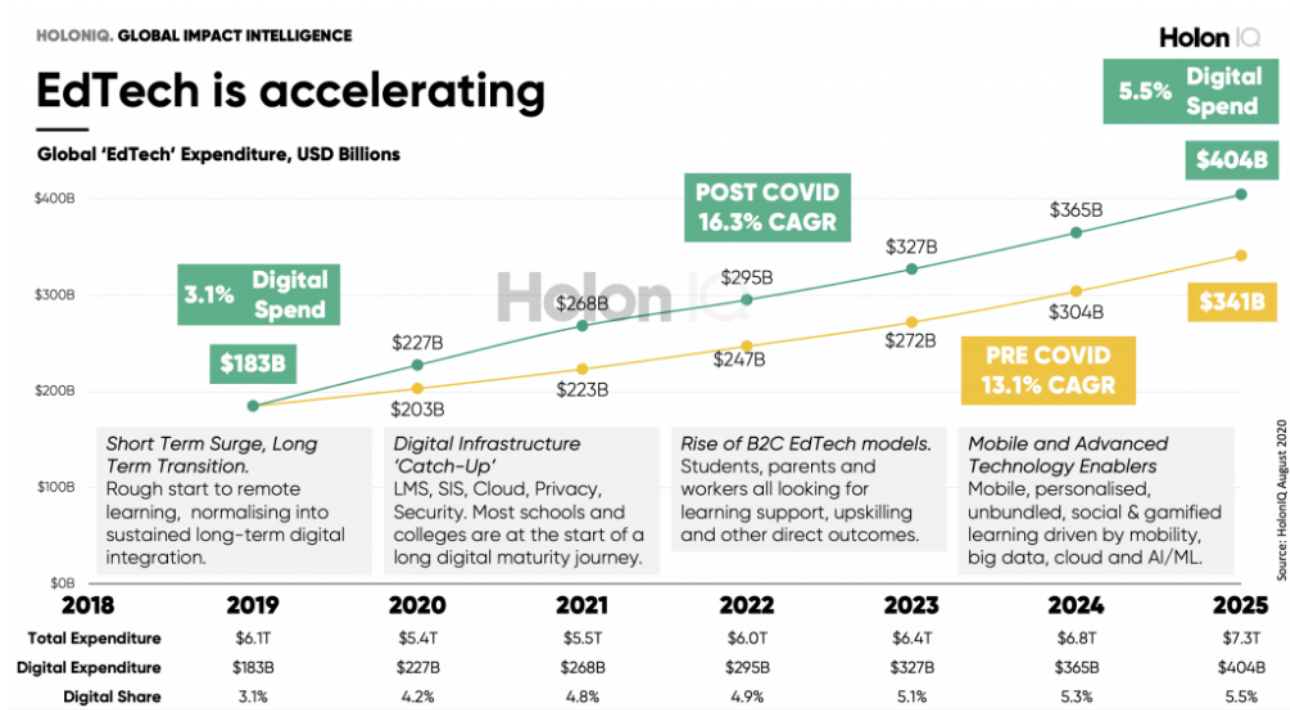


Figure 1.2: Growth of the global EdTech market between 2019 and 2025 (Source: HolonIQ, 2020).

Within this landscape, Smartovate positions SUBUL as a next-generation EdTech platform that goes beyond content delivery, incorporating adaptive learning powered by artificial intelligence to personalize each learner's training journey according to their individual profile, pace, and objectives.

1.2.2 Human Resources Technology: Digitalization of Talent Acquisition

Human resources technology, or HRTech, refers to the set of software solutions and digital platforms that support the management of human capital within organizations. In the domain of talent acquisition specifically, HRTech encompasses applicant tracking systems (ATS), job listing platforms, professional networks, candidate assessment tools, and recruitment workflow management systems.

These technologies have fundamentally transformed the way organizations identify, evaluate, and recruit talent. Applicant Tracking Systems, in particular, have become a central component of modern recruitment pipelines, enabling employers to process large volumes of applications efficiently by automating the parsing, scoring, and filtering of candidate profiles. Professional networks and job boards have similarly become the primary channels through which employment opportunities are

advertised and discovered at scale.

HRTech solutions are designed to serve the needs of employers and recruitment teams, optimizing the efficiency of hiring processes and supporting data-driven talent decisions at the organizational level.

1.2.3 Smartovate at the Intersection of Both Markets

While EdTech and HRTech have each evolved significantly as independent industries, they have historically operated in parallel rather than in integration. Training platforms focus on the acquisition and certification of skills, while recruitment platforms focus on the matching of candidates with available positions. The transition between these two worlds from completing a learning journey to successfully entering the job market has remained largely unaddressed by mainstream solutions in either sector.

It is precisely at this intersection that Smartovate defines its strategic positioning. The SUBUL platform is designed to bridge the gap between skill development and professional opportunity, combining the personalized learning capabilities of an EdTech platform with career-oriented features that connect learners directly to the job market. This dual positioning training and employability within a single ecosystem constitutes the core business vision that motivates the development of the present project.

Figure 1.3: Smartovate’s positioning at the intersection of EdTech and HRTech

1.3 Business Model and Personalized Career Guidance

Smartovate does not operate like a conventional e-learning marketplace where users browse a catalog, select a course, and study independently. The SUBUL platform is built around a fundamentally different premise: every learner arrives with a unique combination of existing skills, knowledge gaps, career objectives, and market positioning. Before recommending a learning path, a certification track, or a career direction, the system must first understand the learner’s profile in depth.

This qualification-first approach is reflected in the platform’s architecture, which relies on a set of specialized AI agents, each responsible for a specific dimension of the learner’s journey:

Agent	Role in the Platform
Quiz Agent	Administers an initial assessment at the start of the platform experience: 3 questions to identify the learner’s professional domain, followed by 7 questions to determine their proficiency level. This evaluation drives the personalization of all subsequent recommendations.

Agent	Role in the Platform
Tutor Agent	Generates personalized course content covering the platform's main training domains: Artificial Intelligence, Cloud Computing (AWS, Azure), NVIDIA technologies, and Cybersecurity. Explains concepts, answers learner questions, proposes quizzes and case studies at the end of each course, evaluates answers with constructive feedback, and dynamically adapts the difficulty level according to each learner's individual progress.
Accompagnement Agent	Guides users through practical, hands-on tasks via a conversational chatbot interface. For example: how to create a virtual machine in Azure, how to configure an S3 bucket in AWS, or how to set up a CI/CD pipeline.
LinkedIn Agent	Analyzes a user's LinkedIn profile and provides actionable recommendations: connection suggestions aligned with their career goals, post ideas to increase visibility, and profile optimization strategies for better recruiter discoverability.
Supervisor Agent	Orchestrates the career assistance pipeline, interpreting each learner request and routing it to the appropriate specialized agent below. Developed as part of the present final year project.
CV Agent	Extracts, evaluates, and improves the learner's resume into a canonical, ATS-optimized profile. Developed as part of the present final year project.
Job Agent	Aggregates job opportunities from multiple sources and ranks them by semantic compatibility with the learner's profile. Developed as part of the present final year project.
ATS Agent	Computes a job-specific compatibility score between the canonical CV and a selected position. Developed as part of the present final year project.

Tableau 1.2: AI agents within the SUBUL platform ecosystem

The business model reflects this multi-agent architecture through a tiered subscription structure designed to serve different user segments:

Plan	Price	Key Features
Basic (Free)	Free	7-day certification prep, expert trainers, interactive learning, 10 quizzes. <i>CV enhancement, LinkedIn booster, and job search are not included.</i>
Standard	99.99 TND/month <i>49.99 TND for students & clubs</i>	30-day certification prep, expert trainers, interactive learning, 50 quizzes, CV enhancement, LinkedIn profile booster, 5 targeted job offers
University	Custom quote	Unlimited certification prep, unlimited quizzes, CV enhancement, LinkedIn booster, unlimited job search, student management, bulk student enrollment
Enterprise	Custom quote	Unlimited certification prep, unlimited quizzes, CV enhancement, LinkedIn booster, unlimited job search, workforce management, job posting on the platform

Tableau 1.3: SUBUL subscription plans and pricing (TND Tunisia)

Within these packages, ATS compatibility scoring and job-specific CV tailoring are provided as part of the CV enhancement feature, while the conversational assistant is available across all subscription tiers to support the learner throughout their journey. The pricing structure reveals that career-oriented features CV enhancement, LinkedIn optimization, and job search are reserved for paid plans, reflecting their strategic importance within the platform’s value proposition.

Figure 1.4: The SUBUL learner journey: from assessment to employment

1.4 Design Constraints and Project Challenges

Beyond the market context and the strategic positioning outlined above, the project also had to account for a set of real-world constraints inherent to the SUBUL environment. These constraints are not specific to any particular solution, but they define the boundaries within which any viable approach must operate.

Constraint	Description
------------	-------------

Constraint	Description
Multilingual Environment	Learners on the SUBUL platform submit documents and interact in French, English, and Arabic. Any career-related functionality must handle multilingual inputs consistently and produce reliable results regardless of the language used.
Semantic Complexity	The professional vocabulary used in resumes and job listings is highly heterogeneous. The same skill can be expressed in dozens of ways across documents, requiring an understanding that goes beyond simple keyword matching.
Source Heterogeneity	Job opportunities are scattered across numerous platforms, each with its own structure, data format, and access policies. Aggregating them into a unified and comparable format presents significant technical challenges.
Scalability	The platform serves thousands of concurrent users. Any career assistance functionality must remain responsive under load, even when processing computationally intensive tasks.
Cost Sensitivity	Advanced AI capabilities, particularly those relying on large language models, incur usage-based costs. The design must balance quality of results with economic sustainability.
Transparency and Trust	Career decisions carry significant personal consequences. Users need to understand the reasoning behind any recommendation or evaluation they receive, not simply trust a numerical output.
Ecosystem Integration	Any new functionality must coexist with the existing SUBUL agents, share the same data infrastructure, and respect the established user experience patterns of the platform.

Tableau 1.4: Constraints identified within the SUBUL environment

These constraints do not prescribe a specific solution, but they define the requirements that any approach must satisfy. They will serve as evaluation criteria throughout the design and implementation phases described in the subsequent chapters.

Conclusion

This first chapter has set the scene for the project. Smartovate operates at the intersection of two rapidly evolving industries — educational technology and human resources technology — carried commercially by its flagship platform SUBUL, where every learner’s journey requires a deep understanding of their profile, their skill level, and their career objectives.

The central point is therefore not simply the existence of training content, certification programs, or AI agents within the platform. The real value lies in the ability to connect these elements intelligently: learner profiles, acquired competencies, market opportunities, and career guidance — all within a single, coherent experience. It is precisely in this space, between professional training and employment access, that the present project finds its purpose.

The project does not start from a blank page. It is embedded within an organization that already possesses a mature platform, established partnerships, and a clear commercial vision — but faces a concrete challenge: extending the SUBUL experience beyond learning, toward the job market, in a way that is intelligent, personalized, and transparent.

The following chapter transforms this context into a formal project framework.

PROJECT ANALYSIS AND REQUIREMENTS SPECIFICATION

Introduction

This chapter constitutes the analytical foundation of the project. Building upon the organizational and market context established in the previous chapter, it formally frames the problem that motivated the development of the AI Career Assistant system, analyzes the landscape of existing solutions, defines the project objectives and scope, and translates the identified needs into a rigorous specification of functional and non-functional requirements. It concludes with a use case model that captures the interactions between the system's actors and its core functionalities, providing the reference framework upon which the design and implementation phases described in the subsequent chapters are built.

2.1 Problem Statement

The transition from professional training to employment represents one of the most critical and most poorly supported stages in a learner's career development journey. While the previous chapter established that Smartovate operates at the intersection of EdTech and HRTech, it is precisely at this intersection that a structural gap persists: learning platforms produce skilled professionals but do not connect them to the job market, while recruitment tools manage applications but offer no guidance to candidates preparing for them. Between these two endpoints, the learner is left to navigate alone, without objective support, without unified tools, and without actionable feedback. This gap manifests across three interconnected dimensions.

2.1.1 The Invisible Barrier of Automated Screening

The first dimension concerns the processing of candidate resumes. In the current recruitment landscape, the vast majority of applications submitted to companies are never reviewed by a human recruiter. Instead, they are first processed by Applicant Tracking Systems (ATS)¹ automated software that parses, scores, and filters resumes based on keyword matching, structural criteria, and formatting rules. The use of Applicant Tracking Systems is now nearly universal among large employers, with adoption reaching approximately 99% among Fortune 500 companies [9]. While the widely circulated claim that 75% of resumes are automatically rejected by ATS lacks a verifiable original source, industry surveys consistently show that resumes failing to meet keyword and formatting requirements are ranked lower and are therefore less likely to reach a human recruiter, regardless of the candidate's actual qualifications [10].

This creates a systemic inefficiency: a learner who has invested months acquiring relevant

1. Software that automates the recruitment workflow, from job posting to candidate selection and hiring decisions `gartner_ats_glossary`.

technical skills may be systematically excluded from consideration simply because their resume does not conform to the structural expectations of a parsing algorithm. The problem is not a lack of competence it is a lack of alignment between how the candidate communicates their profile and how the machine interprets it. Yet no mainstream EdTech platform addresses this issue: they stop at the point of certification, leaving the learner without any guidance on how to translate their acquired skills into a document that can survive automated screening.

2.1.2 The Fragmentation of Job Market Information

The second dimension concerns the discovery of employment opportunities. Job listings are published across a large and heterogeneous ecosystem of platforms: general job boards such as LinkedIn, Indeed, and Glassdoor; specialized aggregators such as Welcome to the Jungle, RemoteOK, and AI Jobs; regional platforms such as Tanitjobs and Eluta; and the career pages of individual companies, powered by applicant management systems such as Lever, Greenhouse, and Workday.

Each of these platforms operates independently, with its own data format, its own search interface, and its own update frequency. A learner seeking employment must manually navigate this fragmented landscape, searching across multiple sources, comparing listings against their own profile, and estimating their compatibility with each position without any objective metric to guide their decision. The data required to connect a learner’s profile to the right opportunities already exists on both sides within candidate CVs and across job market platforms but it remains scattered, unstructured, and never cross-referenced in any systematic way.

2.1.3 The Absence of Semantic Profile-to-Offer Matching

The third dimension concerns the semantic matching between a candidate’s profile and job requirements. Even when a learner identifies a potentially relevant position, accurately assessing their compatibility requires understanding the semantic meaning² of skills, qualifications, and experience rather than relying solely on exact keyword matching. For example, the same competency may appear under different expressions such as “Machine Learning”, “ML”, or “Artificial Intelligence” in English, and “Apprentissage automatique” in French. Traditional keyword-based approaches are often unable to capture these semantic relationships effectively.

Although several recruitment platforms and AI-powered tools provide semantic matching capabilities, these features are generally designed to support recruitment processes rather than learners’ career development. They often provide limited transparency regarding compatibility scores and rarely

2. Semantic similarity refers to the degree to which two pieces of text share the same meaning, independent of their exact wording, typically computed via dense vector representations known as sentence embeddings `reimers_sentencebert_2019`.

integrate resume analysis, job matching, skill gap identification, and personalized learning recommendations within a unified environment.

Consequently, learners still need to interpret matching results, identify missing competencies, and determine which learning activities would improve their employability. This fragmented process remains time-consuming and largely manual. Integrating semantic matching with explainable AI and personalized learning support therefore represents an important opportunity to better connect education with employment.

These three dimensions converge into a single central research question that frames the present project:

“How can artificial intelligence be leveraged within an EdTech platform to automatically bridge the gap between a learner’s acquired competencies and the job market, through intelligent resume analysis, multi-source job aggregation, semantic profile-to-offer matching, and explainable, personalized career recommendations?”

2.2 Study of Existing Solutions

Prior to defining the solution developed in this project, a systematic analysis of existing tools was conducted to identify what is already available on the market, what limitations these tools present, and what gap remains unaddressed. The analysis covers six categories of solutions relevant to the problem space.

2.2.1 Analysis of Existing Tools

Category	Examples	Identified Limitations
• ATS Scoring Tools	Jobscan jobscan_official Resumeworded resumeworded Skillroads skillroads_official	Score a CV against a single job description but do not search for matching positions, do not suggest training to fill skill gaps, and provide limited or no explanation for the scores produced.

Category	Examples	Identified Limitations
• Job Boards	LinkedIn <code>linkedin_official</code> , Indeed <code>indeed_official</code> , Glassdoor <code>glassdoor_official</code> , Welcome to the Jungle <code>wttj_official</code> , RemoteOK <code>remoteok_official</code>	Aggregate job listings but offer no personalized compatibility analysis between the candidate's profile and the listed positions. Recommendations are based on keyword similarity, not semantic understanding.
• Company Career Pages	Lever <code>lever_official</code> , Greenhouse <code>greenhouse_official</code> , Workday <code>workday_official</code>	Each platform uses a distinct data format and interface. No unified access exists for candidates. Application tracking is exclusively employer-side.
• CV Builders	Canva <code>canva_official</code> , Zety <code>zety_official</code> , Kickresume <code>kickresume_official</code> , Novoresume <code>novoresume_official</code>	Focus on the visual design and formatting of resumes but do not verify ATS compatibility, do not assess semantic alignment with target positions, and do not suggest content improvements based on market demand.
• E-Learning Platforms	Coursera <code>coursera_official</code> , Udemy <code>udemy_official</code> , Pluralsight <code>pluralsight_official</code> , edX <code>edx_official</code>	Provide high-quality training content and certification programs but stop at the point of certification. They do not connect learners to the job market, do not analyze the gap between acquired and required skills, and do not recommend specific opportunities based on the learner's profile.
• Career Coaching Tools	Human coaches, LinkedIn Career Explorer <code>linkedin_career_explorer</code>	Offer personalized guidance but are expensive, not scalable, and dependent on individual human expertise. Automated alternatives remain superficial and lack semantic depth.

Tableau 2.1: Analysis of existing solutions in the professional development ecosystem

2.2.2 Comparative Positioning

The following table summarizes the feature coverage of the most representative existing solutions, and positions **the AI Career Assistant** against them across the key dimensions of the problem.

Solution	ATS Score	CV Boost	Job Search	Matching	Gap Analysis	Explain
Jobscan						
Resume-worded						
LinkedIn						
Indeed						
Coursera						
Canva/Zety						
AI Career Assistant						

Figure 2.1: Comparative feature matrix: existing solutions vs. AI Career Assistant (full, partial, or absent)

The comparative analysis reveals a consistent pattern: every existing solution addresses one or two fragments of the professional development pipeline in isolation. No tool currently integrates the complete journey from CV analysis and optimization to multi-source job discovery, semantic matching, gap analysis, and explainable personalized recommendations within a single, unified, AI-powered platform. This observation directly justifies the development of the AI Career Assistant.

2.3 Project Objectives

Based on the problem statement and the gaps identified in the existing solutions, the objectives of the AI Career Assistant project were defined according to the SMART framework (Specific, Measurable, Achievable, Relevant, Time bound). These objectives are organized around the three specialized agents that constitute the system.

The SMART framework:

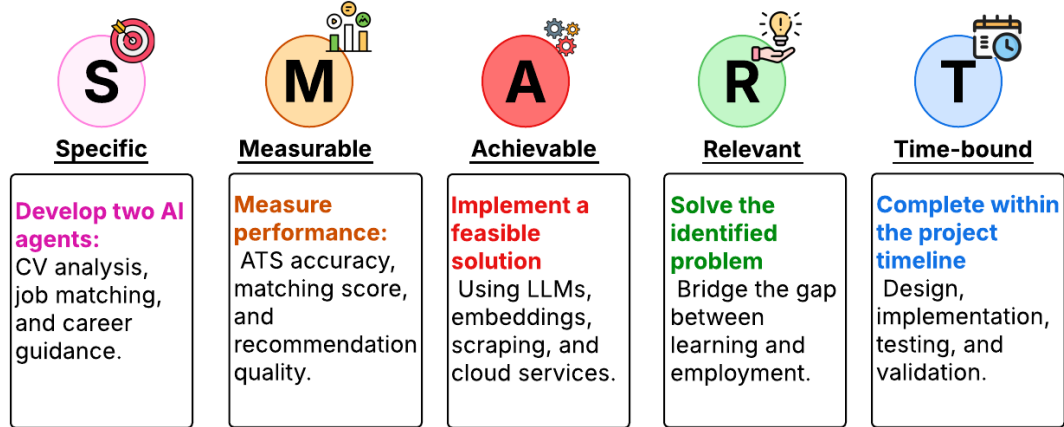


Figure 2.2: SMART-based definition for the AI Career Assistant system.

2.4 Project Scope

Defining a clear and realistic project scope was essential to ensure that the mission remained achievable within the constraints of a final year project while delivering meaningful value to the SUBUL platform. The following table presents the boundaries of the system:

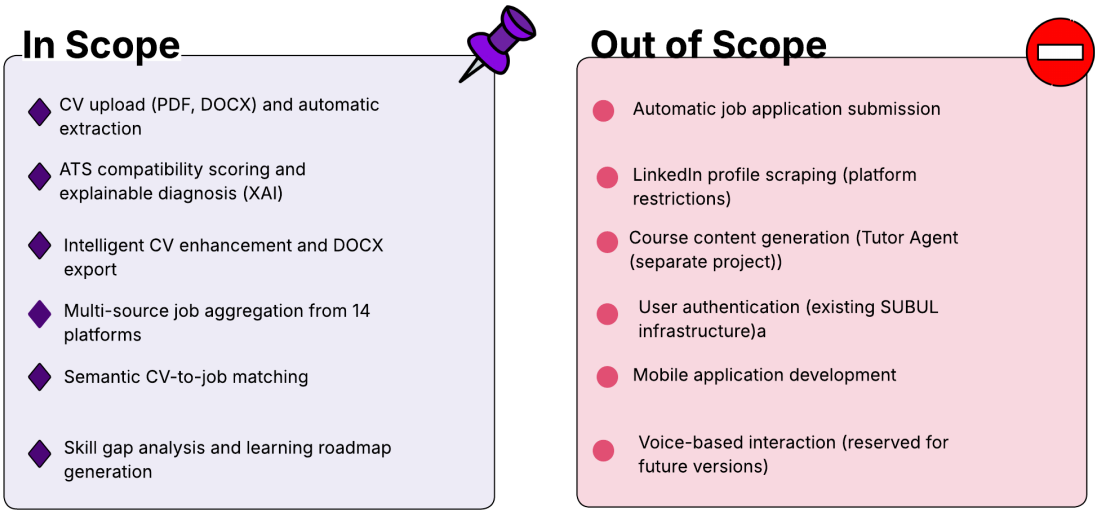


Figure 2.3: Project scope: in scope and out of scope boundaries

2.5 Actors Identification

The AI Career Assistant system involves three distinct actors, each playing a specific role in the system's operation. Their identification is a prerequisite for the use case modeling presented in the following section. The internal orchestration performed by the Supervisor Agent, which routes each

learner request to the appropriate specialized agent (CV Agent, Job Agent, or ATS Agent), is described in the Functional Requirements section rather than modeled as a UML actor, since it operates entirely within the system boundary and does not constitute an external interacting entity.

Actors:

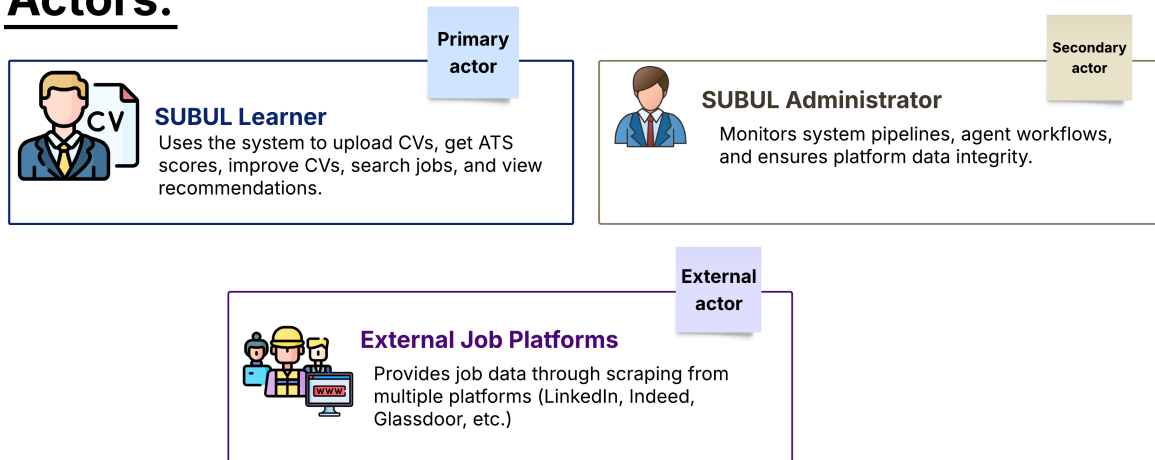


Figure 2.4: Actors of the AI Career Assistant system

2.6 Functional Requirements

The functional requirements define what the AI Career Assistant system must be capable of doing. They are organized around the three specialized pipeline agents coordinated by the Supervisor Agent, plus the conversational assistant available throughout the experience.

2.6.1 CV Agent

ID	Functional Requirement
FR01	Resume upload must be supported in PDF, DOCX, and TXT formats.
FR02	Raw text shall be extracted from the uploaded document and the candidate's professional domain shall be automatically detected.
FR03	Existing CV sections shall be parsed and identified.
FR04	A baseline ATS score (0–100) shall be computed from the CV structure alone key sections, technical skills, contact information, length, and domain keywords independent of any job description.

ID	Functional Requirement
FR05	The baseline score shall be displayed to the learner together with a detailed explanation of each evaluated dimension.
FR06	The learner shall be able to select platform-validated certifications and labs, include or exclude CV sections, and add missing information (languages, experience, education) during the enrichment step.
FR07	Enrichment data shall be merged with the original CV content and submitted to the LLM ³ to generate an improved, ATS-optimized version, strictly reusing information already present in the source document.
FR08	The resulting canonical CV profile shall be persisted in the database and made accessible to all other agents.

Tableau 2.2: Functional requirements CV Agent

2.6.2 Job Agent

ID	Functional Requirement
FR09	Job listings shall be aggregated in real time from 14 heterogeneous sources through a streaming (SSE) pipeline.
FR10	All scraped listings shall be normalized into a unified format and stored with a maximum validity period of 45 days.
FR11	The canonical CV shall be encoded as a semantic vector (MiniLM ⁴) to compute cosine similarity ⁵ against scraped listings.

3. A Large Language Model (LLM) is a deep learning system trained on vast text corpora, built on the Transformer architecture with self-attention mechanisms, capable of understanding and generating human language [aws_llm_definition](#).

4. A compact Transformer-based language model obtained through knowledge distillation [wang_minilm_2020](#).

5. A mathematical measure of the angle between two vectors, used to quantify semantic alignment [steck_cosine_2024](#).

ID	Functional Requirement
FR12	Matched listings shall be enriched via LLM and streamed to the learner in real time as job cards, ranked by compatibility.
FR13	A skill gap analysis tool shall be available, identifying competencies missing relative to current market demand.
FR14	A market analysis view shall be available, summarizing skill demand trends and the distribution of top companies and locations across matched listings.
FR15	A personalized, three-phase learning roadmap shall be generated and enriched by the LLM based on identified gaps.
FR16	A complete PDF report shall be generated, summarizing the candidate's profile, matched positions, skill gaps, and learning roadmap.

Tableau 2.3: Functional requirements Job Agent

2.6.3 ATS Agent

ID	Functional Requirement
FR17	On learner request, a multi-dimensional compatibility score (skills, experience, education, and job designation) shall be computed between the canonical CV and a specific selected job.
FR18	The resulting score shall be displayed to the learner and used as the baseline ("before") score for the subsequent targeted enhancement.
FR19	The learner shall be able to select an output format (ATS Classic, Modern, Canadian, Europass, or Basic) before requesting a targeted CV enhancement for the selected job.

ID	Functional Requirement
FR20	A dedicated enhancement endpoint shall rewrite the CV for the selected job, using the ATS Agent's compatibility score as the consistent baseline, strictly reusing information already present in the canonical CV without inventing degrees, skills, or experience.
FR21	The enhanced CV shall be exported as a downloadable DOCX file, accompanied by the before/after compatibility scores.

Tableau 2.4: Functional requirements ATS Agent

2.6.4 Conversational Assistant

ID	Functional Requirement
FR22	A conversational assistant shall be available throughout the learner's experience, responding in French or English based on automatically detected language.
FR23	The assistant shall have contextual access to the learner's canonical profile and currently matched job listings.
FR24	Conversation history shall be persisted in the database for continuity across sessions.

Tableau 2.5: Functional requirements Conversational Assistant

2.7 Non-Functional Requirements

Non-functional requirements define the quality attributes the system must exhibit, independent of specific functionalities, and constitute the evaluation criteria for the system's overall quality.

Category	ID	Requirement
Performance	NFR01	CV analysis pipeline shall complete within 30 seconds.
Performance	NFR02	Job search and matching pipeline shall return results within 60 seconds.
Scalability	NFR03	A minimum of 1,000 concurrent users shall be supported without performance degradation.
Security	NFR04	All CV data shall be encrypted and compliant with applicable data protection regulations.
Reliability	NFR05	Minimum 99.9% uptime shall be maintained. Scraper failures shall not affect other components.
Explainability	NFR06	Every ATS score and matching result shall include a human-readable structured explanation.
Maintainability	NFR07	Each agent shall be independently deployable and updatable without impacting other components.
Usability	NFR08	All interactions shall be accessible through an intuitive interface requiring no technical knowledge.
Multilingual	NFR09	Inputs and chatbot interactions in French and English shall be correctly processed with consistent results.
Data Freshness	NFR10	A maximum job listing age of 45 days shall be enforced with on-demand re-scraping capability.

Tableau 2.6: Non-functional requirements

2.8 Use Case Modeling

2.8.1 General Use Case Diagram

The following diagram presents the overall use case model of the AI Career Assistant system, illustrating the interactions between the three identified actors and the system's eleven core use cases.

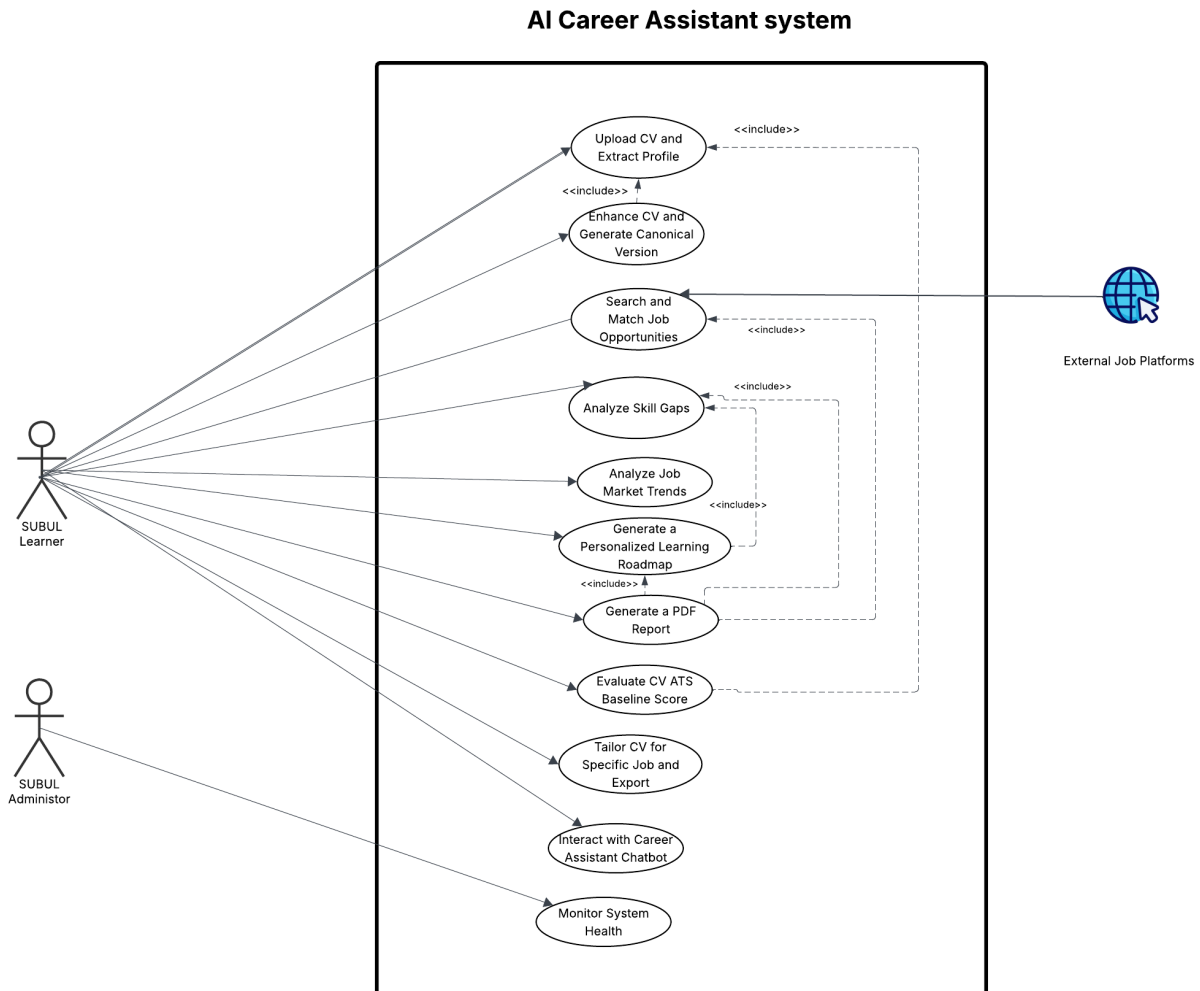


Figure 2.5: General use case diagram of the AI Career Assistant system

2.8.2 Use Case Descriptions

This section details each of the eleven use cases identified in the general diagram, specifying for every case its primary actor, preconditions, trigger, normal and alternative flows, and postconditions. The following table provides a summary before each use case is described individually.

ID	Use Case	Primary Actor	Brief Description
UC01	Upload CV and Extract Profile	SUBUL Learner	Upload a CV and extract its structured content and professional domain.
UC02	Enhance CV and Generate Canonical Version	SUBUL Learner	Enrich the CV with certifications, labs, and missing information to generate a canonical improved CV.
UC03	Search and Match Job Opportunities	SUBUL Learner / External Job Platforms	Scrape 14 job platforms in real time and rank listings by semantic similarity to the canonical CV.
UC04	Analyze Skill Gaps	SUBUL Learner	Identify competencies missing relative to current market demand.
UC05	Analyze Job Market Trends	SUBUL Learner	Consult trends in demanded skills, seniority levels, and salaries.
UC06	Generate a Personalized Learning Roadmap	SUBUL Learner	Generate a three-phase, LLM-enriched learning roadmap based on identified gaps.
UC07	Generate a PDF Report	SUBUL Learner	Generate a complete PDF report summarizing the profile, matches, gaps, and roadmap.
UC08	Evaluate CV ATS Baseline Score	SUBUL Learner	Compute a structural quality score of the CV, independent of any job description.
UC09	Tailor CV for Specific Job and Export	SUBUL Learner	Compute a job-specific compatibility score, then generate and download a tailored, ATS-optimized CV with before/after scores.

ID	Use Case	Primary Actor	Brief Description
UC10	Interact with Career Assistant Chatbot	SUBUL Learner	Interact with a multilingual chatbot aware of the learner's profile and matched jobs.
UC11	Monitor System Health	SUBUL Administrator	Monitor the health of scraping pipelines, agent workflows, and data integrity.

Tableau 2.7: Summary of primary use cases

Each of the eleven use cases summarized above is now detailed individually, from UC01 to UC11.

UC01 Upload CV and Extract Profile

Attribute	Description
Actor	SUBUL Learner
Objective	Upload a CV and extract its structured content and professional domain.
Preconditions	The learner has a valid SUBUL account.
Trigger	The learner uploads a PDF, DOCX, or TXT file.
Normal Flow	1. CV file is uploaded (PDF, DOCX, or TXT). 2. Raw text is extracted from the document. 3. The professional domain is automatically detected. 4. Existing CV sections are parsed and identified. 5. The extracted profile is stored for use by subsequent use cases.

Attribute	Description
Alternative Flow	Unsupported format or unreadable document → error message displayed.
Postconditions	The extracted, structured profile is available for baseline scoring and enrichment.

Tableau 2.8: UC01 Upload CV and Extract Profile

UC02 Enhance CV and Generate Canonical Version

Attribute	Description
Actor	SUBUL Learner
Objective	Enrich the CV with validated certifications, labs, and missing information, and generate a canonical, ATS-optimized CV version.
Preconditions	UC01 completed (<i>«include»</i>).
Trigger	The learner is redirected to the enrichment screen after uploading the CV.

Attribute	Description
Normal Flow	1. Learner selects platform-validated certifications and labs. 2. Learner includes or excludes CV sections. 3. Learner adds missing information (languages, experience, education). 4. Enrichment data is merged with the original CV and submitted to the LLM. 5. The LLM rewrites the CV in a structured, ATS-optimized form, strictly reusing information already present in the source document. 6. The resulting canonical CV is persisted in CosmosDB as the learner's reference profile.
Alternative Flow	Learner skips enrichment → the extracted CV is used as the canonical profile without enhancement.
Postconditions	A canonical, structured CV is stored and available to all subsequent use cases.

Tableau 2.9: UC02 Enhance CV and Generate Canonical Version

UC03 Search and Match Job Opportunities

Attribute	Description
Actor	SUBUL Learner, External Job Platforms
Objective	Aggregate job listings from 14 sources in real time and rank them by semantic compatibility with the canonical CV.
Preconditions	A canonical CV is available (UC02).

Attribute	Description
Trigger	Automatic activation once the canonical CV is available, or explicit learner request to refresh listings.
Normal Flow	1. The canonical CV is encoded as a semantic vector (MiniLM). 2. The streaming pipeline activates 14 scraping sources in parallel (LinkedIn, Indeed, Greenhouse, RemoteOK, Tanitjobs, Welcome to the Jungle, and others). 3. Listings are filtered by cosine similarity against the CV vector, enriched via LLM, and scored. 4. Matched listings are streamed and displayed in real time as job cards, ranked by compatibility.
Alternative Flow	A scraper fails for a specific source → remaining sources continue; the failure is logged and visible to the administrator.
Postconditions	A ranked list of matched jobs is available for skill gap analysis, market analysis, and ATS scoring.

Tableau 2.10: UC03 Search and Match Job Opportunities

UC04 Analyze Skill Gaps

Attribute	Description
Actor	SUBUL Learner
Objective	Identify the competencies missing from the learner's profile relative to current market demand.
Preconditions	UC03 completed (<i>«include»</i>).
Trigger	The learner opens the Skills Gap tool from the job interface.

Attribute	Description
Normal Flow	1. The system retrieves the candidate's skill set and aggregated market requirements from matched listings. 2. Skills are classified as matched, partial, or missing. 3. Missing skills are prioritized by market frequency. 4. The gap report is displayed to the learner.
Alternative Flow	No significant gap identified → system displays a positive compatibility summary.
Postconditions	The gap report is available as input for the learning roadmap (UC06).

Tableau 2.11: UC04 Analyze Skill Gaps

UC05 Analyze Job Market Trends

Attribute	Description
Actor	SUBUL Learner
Objective	Provide the learner with an overview of market trends relevant to their domain.
Preconditions	UC03 completed.
Trigger	The learner opens the Market tool from the job interface.
Normal Flow	1. The system aggregates data from matched and scraped listings. 2. In-demand skills, top companies, and top locations are computed. 3. The market overview is displayed to the learner.

Attribute	Description
Alternative Flow	Insufficient listings for the domain → system indicates limited data availability.
Postconditions	Market insights are available to inform the learner's roadmap and job search strategy.

Tableau 2.12: UC05 Analyze Job Market Trends

UC06 Generate a Personalized Learning Roadmap

Attribute	Description
Actor	SUBUL Learner
Objective	Generate a structured, three-phase learning roadmap based on identified skill gaps.
Preconditions	UC04 completed (<i>«include»</i>).
Trigger	The learner opens the Roadmap tool from the job interface.
Normal Flow	1. The prioritized list of missing skills is retrieved. 2. The LLM organizes the skills into a three-phase learning plan, ordered by priority and prerequisite dependency. 3. The roadmap is displayed with estimated effort per phase.
Alternative Flow	No gaps identified → system indicates the profile is already well aligned with the market.
Postconditions	The learner has a clear, phased plan to improve their employability.

Tableau 2.13: UC06 Generate a Personalized Learning Roadmap

UC07 Generate a PDF Report

Attribute	Description
Actor	SUBUL Learner
Objective	Generate a complete, downloadable PDF report summarizing the learner's profile, matches, gaps, and roadmap.
Preconditions	UC06 completed (<i>«include»</i>).
Trigger	The learner requests a report from the job interface.
Normal Flow	1. The system consolidates the canonical profile, matched jobs, gap analysis, and roadmap. 2. A structured PDF report is generated. 3. The report is made available for download.
Alternative Flow	Report generation fails → the learner is notified and invited to retry.
Postconditions	A downloadable PDF report is available to the learner.

Tableau 2.14: UC07 Generate a PDF Report

UC08 Evaluate CV ATS Baseline Score

Attribute	Description
Actor	SUBUL Learner
Objective	Compute a structural quality score of the CV, independent of any job description, and explain it in detail.
Preconditions	UC01 completed (<i>«include»</i>).

Attribute	Description
Trigger	Automatic evaluation immediately after CV extraction.
Normal Flow	1. The local scoring formula evaluates key sections, technical skills, contact information, length, and domain keywords. 2. A baseline score (0–100) is computed. 3. The score is displayed with a section-by-section explanation of what is missing or weak.
Alternative Flow	Extracted content is too sparse to evaluate reliably → system flags the CV as incomplete and invites enrichment.
Postconditions	A baseline ATS score and explanation are available to the learner before any enrichment or job-specific evaluation.

Tableau 2.15: UC08 Evaluate CV ATS Baseline Score

UC09 Tailor CV for Specific Job and Export

Attribute	Description
Actor	SUBUL Learner
Objective	Compute a job-specific compatibility score, then generate and download a version of the CV tailored to that job, with a measurable score improvement.
Preconditions	UC02 and UC03 completed. A job listing has been selected.
Trigger	The learner clicks “Calculate ATS Score” on a matched job card, then selects an output format and clicks “Enhance”.

Attribute	Description
Normal Flow	1. The ATS Agent computes a multi-dimensional compatibility score between the canonical CV and the selected job (skills, experience, education, designation). 2. The score is displayed in a dedicated modal and stored as the “before” value. 3. The learner selects an output format (ATS Classic, Modern, Canadian, Europass, or Basic) and requests enhancement. 4. The document language is detected and technical keywords are extracted from the job description. 5. A dedicated enhancement endpoint rewrites the CV for the job, reusing the ATS Agent’s score as the consistent “before” baseline and strictly reusing information already present in the canonical CV. 6. The “after” score is computed using the stored “before” score for consistency. 7. The enhanced CV is generated in the selected format and returned as a downloadable file with before/after scores.
Alternative Flow	The score is already optimal → the learner is notified that no further enhancement is needed.
Postconditions	The learner downloads the job-tailored CV and sees the real score progression.

Tableau 2.16: UC09 Tailor CV for Specific Job and Export

UC10 Interact with Career Assistant Chatbot

Attribute	Description
Actor	SUBUL Learner

Attribute	Description
Objective	Provide a multilingual conversational assistant, aware of the learner's profile and matched jobs, available throughout the experience.
Preconditions	The learner is authenticated on the SUBUL platform. A CV profile may or may not exist.
Trigger	The learner opens the chatbot and submits a message.
Normal Flow	1. The learner submits a message in French or English. 2. The language is automatically detected. 3. If available, the canonical profile and matched jobs are included in the response context. 4. A relevant, contextualized response is generated. 5. The conversation history is persisted and remains accessible to the learner.
Alternative Flow	The question falls outside the assistant's scope → the learner is redirected to the relevant platform feature.
Postconditions	The learner's question is answered and the conversation is saved for continuity.

Tableau 2.17: UC10 Interact with Career Assistant Chatbot

UC11 Monitor System Health

Attribute	Description
Actor	SUBUL Administrator
Objective	Monitor the health of the scraping pipelines, agent workflows, and the integrity of the stored data.

Attribute	Description
Preconditions	The administrator has valid platform access.
Trigger	The administrator accesses the monitoring dashboard.
Normal Flow	1. The administrator consults the status of the 14 scraping sources. 2. The administrator reviews agent workflow logs (CV Agent, Job Agent, ATS Agent). 3. Data integrity indicators are checked (job listing freshness, failed scrapes, storage consistency).
Alternative Flow	An anomaly is detected → the administrator is alerted and can trigger corrective actions (e.g., re-scraping).
Postconditions	The administrator has an up-to-date view of system health and data integrity.

Tableau 2.18: UC11 Monitor System Health

2.9 Expected Deliverables

The AI Career Assistant system comprises the following deliverables, organized across five components:

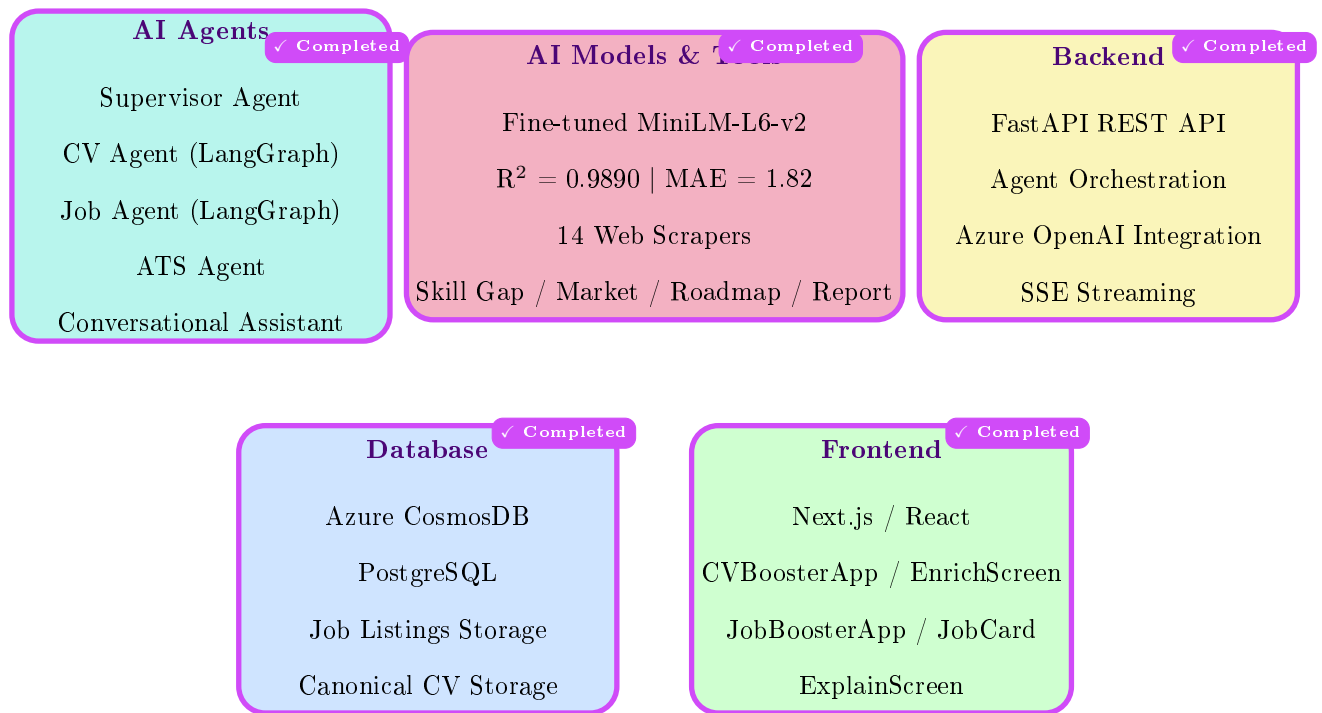


Figure 2.6: AI Career Assistant system deliverables overview

Conclusion

This chapter has formalized the analytical and functional foundation of the AI Career Assistant system. The examination of the problem across its three dimensions automated screening opacity, job market fragmentation, and the absence of semantic matching demonstrated that no existing solution addresses these challenges within a unified framework. The comparative analysis confirmed that available tools operate in isolation, each covering a single fragment of the professional development pipeline.

The resulting specification encompassing objectives, scope, actors, functional and non-functional requirements, and use cases constitutes a complete and traceable reference for the design and implementation phases that follow. The following chapter presents the architectural and methodological choices that translate these requirements into a deployable solution.

METHODOLOGY AND DEVELOPMENT

APPROACH

Plan

1	Presentation of the Host Organization and the SUBUL Platform	4
2	The EdTech and HRTech Landscape: Context and Challenges	5
3	Business Model and Personalized Career Guidance	7
4	Design Constraints and Project Challenges	9

Introduction

This chapter first justifies the foundational architectural decision that shaped the entire project: the adoption of an agentic AI architecture rather than a single generative pipeline or conventional rule-based software. It then presents the project’s development methodology, combining Scrum for the iterative delivery of the system’s functional blocks with CRISP-ML(Q) for the rigorous handling of its data-science and machine-learning components, and explains how the two methodologies were fused into a single, coherent working approach. The chapter concludes by translating these methodological foundations into the system’s global architecture, its multi-agent design, and its technological stack.

3.1 Why an Agentic AI Architecture

Before any methodology could be selected, a foundational architectural question had to be answered: should the system be built as a single large language model pipeline, as conventional rule-based software, or as a structured multi-agent system? The problem at hand is inherently dual. It requires deterministic reliability a compatibility score must be reproducible, and a rewritten resume must never contain a fabricated fact while also requiring linguistic and contextual intelligence to interpret requests and produce fluent, tailored text. No single paradigm satisfies both demands, which motivates the examination below of two opposite extremes before justifying the intermediate position ultimately adopted.

3.1.1 Limitations of a Single LLM Pipeline

Delegating the entire process extraction, scoring, rewriting, and export to a single unconstrained language model call was discarded for three reasons. First, a score meant to represent objective, comparable progress cannot come from a generative process that yields different results across identical executions. Second, a single generative call provides no structural guarantee against fabricating missing degrees, skills, or experience a constraint merely requested in a prompt is not a constraint enforced. Third, generation and computation are different acts: a language model cannot natively evaluate a scoring formula or assemble a formatted document.

3.1.2 Limitations of Conventional Software Development without AI

The opposite extreme a purely rule-based system with fixed conditional branches and no AI reasoning was equally discarded. Interpreting open-ended, naturally phrased learner requests cannot be reduced to an exhaustive set of hand-coded conditions, which remain brittle against the variability natural language inherently produces. Moreover, a monolithic implementation of this kind tends

to entangle logically distinct concerns into a single codebase, undermining the modularity a system expected to evolve over time requires.

3.1.3 The Adopted Position: Agentic AI Orchestrated with LangGraph

The retained architecture reconciles both demands rather than choosing one. Operations requiring reliability and auditability score computation, document parsing, export generation, data persistence remain deterministic code. Operations requiring contextual judgment and fluency resume rewriting, explanation generation, intent interpretation are delegated to a large language model, invoked under conditions enforced before and after each call. This reconciliation is made explicit through an orchestration layer: a directed graph of processing states where each node performs one defined operation and every transition is governed by an inspectable condition. This layer was implemented using LangGraph, chosen for its ability to express such multi-step reasoning as an explicit, auditable graph rather than as opaque control flow.

3.1.4 Comparison of Agent Orchestration Frameworks

Among available multi-agent frameworks, AutoGen [11] is optimized for open-ended conversational collaboration between agents, offering weaker guarantees on deterministic, conditional control flow. CrewAI [12] provides a simpler role-based abstraction but less fine-grained control over conditional transitions and typed shared state. LangGraph [13] was selected for its explicit state-graph model, its native conditional branching, and its typed, inspectable shared state – properties that align directly with the reliability and traceability the system requires.

3.2 Software Development Methodology: Scrum

3.2.1 Comparative Analysis of Development Methodologies

The choice of a development methodology cannot be made in isolation from the specific constraints of this project a limited team size, an evolving scope, and the need to absorb corrections discovered only once the system was actually in use. Table 3.1 compares the candidate development methodologies considered against these constraints.

Criterion	Kanban	Extreme Programming (XP)	Scrum
Core Principle	Visualize and limit work in progress on a continuous task flow, without fixed iteration cycles [14].	Ensure code quality through strict team practices such as pair programming and test-first development [15].	Structure development into short, fixed-length cycles, each delivering a testable increment [16].
Relevance to This Project	Flexible for a variable task flow, but does not structure work around clearly demonstrable functional milestones aligned with the system's four functional pillars.	Its quality-oriented practices are relevant in principle, but their strict application assumes a dedicated team this project did not mobilize.	Its incremental breakdown naturally matches the system's decomposition into independent functional blocks (CV Agent, Job Agent, ATS Agent, Conversational Assistant).
Limitation if Used Alone	Does not demonstrate progress through clearly delimited functional milestones, which is essential to convey the project's advancement.	Imposes team practices (systematic pair programming, strict test-driven development) disproportionate for an individual or small-team academic project.	Requires tracking discipline (velocity, retrospectives) that was not formalized with dedicated tooling in this project.

Criterion	Kanban	Extreme Programming (XP)	Scrum
Decision Taken	Rejected as the primary methodology due to the absence of structured deliverable milestones.	Rejected as the primary methodology due to the disproportionate scale of its team practices.	Retained as the primary development methodology, combined with CRISP-ML(Q) for the data/ML component.

Tableau 3.1: Comparison of development methodologies according to their relevance to this project

3.2.2 Justification of the Scrum Choice

Beyond the comparative analysis, the decisive argument for Scrum lies in the natural correspondence between its incremental delivery model and the way the system’s functional capabilities were structured: resume ingestion and baseline evaluation, opportunity discovery and semantic matching, job-specific compatibility scoring combined with dedicated CV tailoring, and a persistent conversational assistant each constitute a largely independent, deliverable block of functionality. Scrum allowed each of these blocks to be developed, tested, and refined iteratively, which proved essential when corrections were required after a functional block was already delivered.

3.3 Data Science Methodology: CRISP-ML(Q)

3.3.1 Comparative Analysis of Data-Science Methodologies

Selecting a data-science methodology required weighing not only the analytical rigor of established process models, but also their capacity to account for a machine learning artifact that continues to operate once deployed, rather than one whose evaluation ends at the close of a single experiment. Table 3.2 compares the candidate data-science methodologies

Methodology	Limitation in this Project's Context
CRISP-DM [17]	Assumes a static dataset analyzed once, with no formal phase for production monitoring. This does not match the project's reality: the fine-tuned matching model is deployed and queried continuously, and the underlying job-listing corpus is perpetually refreshed by an autonomous multi-source scraping pipeline.
KDD (Knowledge Discovery in Databases) [18]	Oriented toward pattern discovery in historical data for analytical purposes; it does not address model deployment or maintenance in a live system.
TDSP (Team Data Science Process) [19]	Closer to CRISP-ML(Q) in its production orientation, but imposes formalized team roles and tooling disproportionate to the scale of an academic final-year project.
CRISP-ML(Q) [20]	Selected because it retains the analytical rigor of CRISP-DM while explicitly adding Deployment and Monitoring & Maintenance phases, matching the reality of a fine-tuned model actually deployed in production and a data corpus that evolves continuously.

Tableau 3.2: Comparison of data-science methodologies

3.3.2 Justification of the CRISP-ML(Q) Choice

The decisive factor was the presence of a genuinely deployed machine learning artifact a fine-tuned bi-encoder model built on top of MiniLM-L6-v2, achieving an R^2 of 0.9890 and an MAE of 1.82 on the compatibility-scoring task combined with a continuously growing corpus of scraped job listings. These two characteristics introduce concerns, such as model reliability in production and the freshness of the underlying data, that CRISP-DM does not formally address, but that CRISP-ML(Q) explicitly incorporates through its Deployment and Monitoring and Maintenance phases.

3.4 Hybrid Methodology: Combining Scrum and CRISP-ML(Q)

3.4.1 Genesis of the Hybrid Approach

Scrum was initially adopted on its own to structure the pace of software delivery. As the project progressed, however, it became apparent that each sprint was not merely producing a software deliverable, but was also advancing a specific stage of the data-science lifecycle. For instance, the sprint dedicated to semantic matching corresponded precisely to the Modeling phase of CRISP-ML(Q), while the final stabilization sprint during which a scoring consistency issue was diagnosed and corrected corresponded to the Monitoring and Maintenance phase. This correspondence, observed by analyzing the project's actual development history rather than planned in advance, motivated the explicit formalization of a hybrid methodology.

3.4.2 Rationale for Combining Both Methodologies

Scrum and CRISP-ML(Q) answer two different and complementary questions. Scrum organizes *when* and *how* work is planned and delivered iteratively. CRISP-ML(Q) defines *which* technical activities must be carried out, and in what logical sequence, to produce a reliable machine-learning-based system. The project therefore does not follow two competing methodologies, but a single hybrid approach in which every Scrum sprint advances one or more phases of the CRISP-ML(Q) cycle. Figure 3.1 illustrates this articulation for the matching-model sub-project specifically, zooming into the sprints during which concrete, datable milestones were reached for this model.

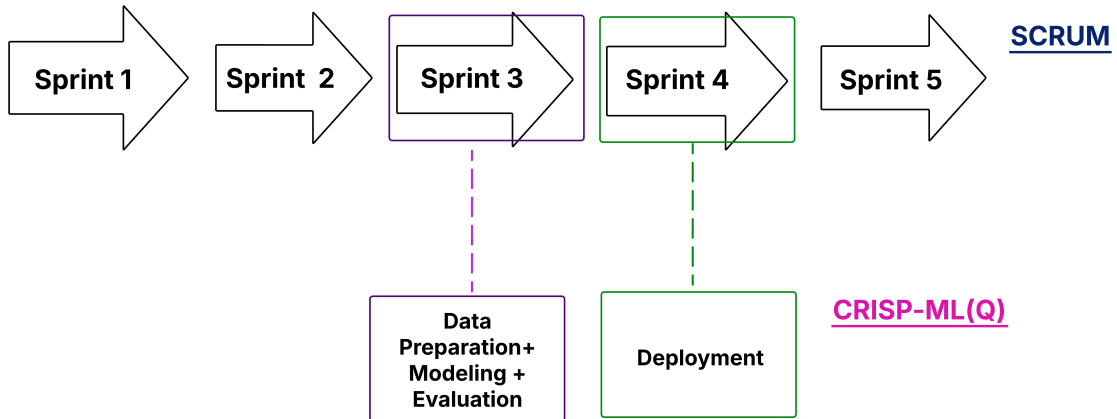


Figure 3.1: Articulation between the macro-level Scrum timeline and the CRISP-ML(Q) phases applied to the matching-model sub-project. The Data Preparation phase shown under Sprint 3 refers specifically to the construction of the model's labeled training dataset (positive/negative CV-job pairs), distinct from the general CV text extraction already covered under Sprint 1 in Table 3.4. Business & Data Understanding and Monitoring & Maintenance are addressed at the project-wide level or discussed as future work (Section 3.5.6), respectively.

3.4.3 Product Backlog Organized by Epic

Rather than listing user stories as an undifferentiated set, the backlog was structured around the four functional pillars already distinguishing the system’s capabilities, so that each epic could be planned, delivered, and evaluated as a coherent, independent unit throughout the iterative development process. The product backlog was organized into epics aligned with the system’s four functional pillars, summarized in Table 3.3.

Epic	Scope
CV Agent	Resume upload and extraction, structural baseline scoring, profile enrichment (certifications, labs, missing information), and generation of the canonical, ATS-optimized CV profile.
Job Agent	Multi-source job scraping, semantic compatibility matching, skill gap analysis, job market analysis, personalized learning roadmap, and PDF report generation.
ATS Agent	Job-specific compatibility scoring for a selected position, feeding a dedicated enhancement endpoint that reuses this score as the consistent “before” baseline and produces a downloadable, job-tailored document.
Conversational Assistant	Multilingual chatbot, contextual awareness of the learner’s profile and matched jobs, and persistence of conversation history.

Tableau 3.3: Product backlog organized by epic

The Supervisor Agent, responsible for the system’s cross-cutting orchestration and intent routing, does not constitute a separate epic: its implementation is intrinsically tied to the integration of the four pillars above, and was therefore delivered as part of the multi-agent assembly iteration (Sprint 4) rather than as an independent, user-facing capability.

3.4.4 Sprint ↔ CRISP-ML(Q) Phase Mapping

Several of the original development iterations advanced the same CRISP-ML(Q) phase and are therefore grouped together in Table 3.4, which presents the correspondence between the project’s development iterations and the CRISP-ML(Q) phases they advanced.

Sprint	Deliverable	CRISP-ML(Q) Phase
1	Foundations and architecture: Cosmos DB / PostgreSQL schema provisioning, FastAPI and Next.js scaffolding, and the CV upload landing screen.	Business and Data Understanding
2	CV Agent: resume extraction, automatic domain detection, baseline ATS scoring, LLM-based CV rewriting with its non-hallucination safeguards, and persistence of the canonical profile.	Data Preparation
3	Job Agent: multi-source scraping pipeline (14 sources) with SSE streaming and cosine-similarity filtering, together with the fine-tuned bi-encoder matching model’s data preparation, training, and evaluation, and the resulting Skill Gap, Market Analysis, Roadmap, and Report modules.	Data Understanding (extended) + Modeling + Evaluation (qualitative and quantitative)
4	ATS Agent, assembly of the Supervisor Agent for cross-agent orchestration, and the Conversational Assistant module.	Deployment

Sprint	Deliverable	CRISP-ML(Q) Phase
5	Stabilization: correction of an ATS scoring inconsistency and robustness fixes.	Monitoring and Maintenance (partial)

Tableau 3.4: Correspondence between development iterations and CRISP-ML(Q) phases

3.4.5 Assessment of the Hybrid Approach

The hybrid approach provided concrete benefits: it allowed a correction discovered late in development the ATS scoring inconsistency addressed in Sprint 5 to be integrated without disrupting the overall project structure, and it ensured that each software deliverable remained anchored to a recognizable data-science activity. However, this assessment must remain honest about its limits: no sprint velocity was measured, no formal retrospectives were documented, and no dedicated Scrum tooling (such as a digital backlog board) was used. The methodology was applied in a lightweight, adapted form suited to an individual/small-team academic project.

3.5 CRISP-ML(Q) Phases Applied to the Project

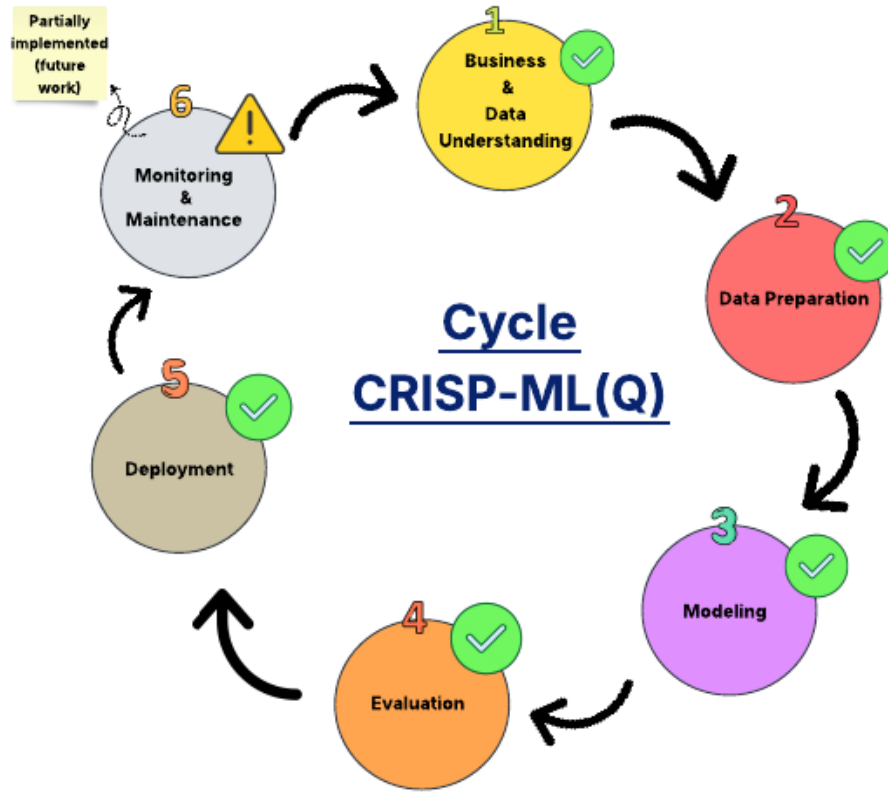


Figure 3.2: CRISP-ML(Q) cycle applied to the AI Career Assistant project

3.5.1 Business and Data Understanding

The objective was to convert SUBUL learning achievements into a measurable employability advantage, drawing on the learner’s CV, the scraped job-listing sources, and the platform’s learning data.

3.5.2 Data Preparation

Raw CV and job-description text was extracted, cleaned, and structured into a canonical, comparable representation.

3.5.3 Modeling

Two complementary machine learning approaches were developed: a lightweight cosine-similarity filter for real-time listing screening, and a fine-tuned bi-encoder regression model for calibrated compatibility scoring. The system’s deterministic ATS scoring formula, by contrast, is a rule-based mechanism rather than a learned model, and is presented separately in Chapter 4.

3.5.4 Evaluation

The fine-tuned matching model was evaluated through standard regression metrics on a held-out test set, while the cosine-similarity filter was validated qualitatively on its ability to correctly pre-select relevant listings ahead of LLM-based enrichment.

3.5.5 Deployment

All trained and pretrained models were loaded once at start-up and served continuously by the backend, alongside the LLM-driven components coordinated by the multi-agent architecture.

3.5.6 Monitoring and Maintenance

This remains the least mature phase: partial robustness mechanisms exist, but no automated drift detection or retraining pipeline was implemented, a limitation discussed further in Chapter 7 and listed as future work.

The full application of these phases to the fine-tuned matching model specifically including its training methodology, architecture, and evaluation benchmark is examined in depth in Chapter 5, which is entirely dedicated to this artifact.

3.6 Global System Architecture

Building on the methodological foundations presented above, this section translates them into the system’s concrete technical architecture. The AI Career Assistant is implemented as a modular monolith rather than a set of independently deployed microservices a deliberate choice given the scale of an academic final-year project, avoiding the operational overhead of network-based inter-service communication while still preserving a clear separation of concerns through four dedicated FastAPI routers.

The React/Next.js frontend communicates with the backend over HTTP for standard request-response interactions and over Server-Sent Events (SSE) for the real-time streaming of scraped job listings. The FastAPI backend organizes four of its functional domains into dedicated routers: **chat_router** for the conversational assistant, **scraping_router** for the multi-source job aggregation pipeline, **jobs_router** for job-related queries, and **enhance_router** for CV enhancement and export. CV intake and persistence (upload, extraction, platform-data retrieval) and the Supervisor Agent’s orchestration endpoints, together with the job-specific ATS scoring endpoints, are exposed directly on the main FastAPI application rather than through a dedicated router, reflecting the modular-monolith design choice discussed above.

Two persistence layers are used according to the nature of the data they store: Azure CosmosDB,

chosen for its flexible NoSQL schema suited to the structurally heterogeneous canonical CV profiles and conversation histories, and Azure PostgreSQL, used for the platform-side data (certifications, labs, positioning quiz) that is inherently relational and schema-stable. The generative components of the system rely on Azure OpenAI’s GPT-4o and GPT-4o-mini models, invoked by the backend for CV rewriting, XAI explanation generation, and intent classification. Finally, the scraping pipeline aggregates job listings in real time from 14 external job board sources.

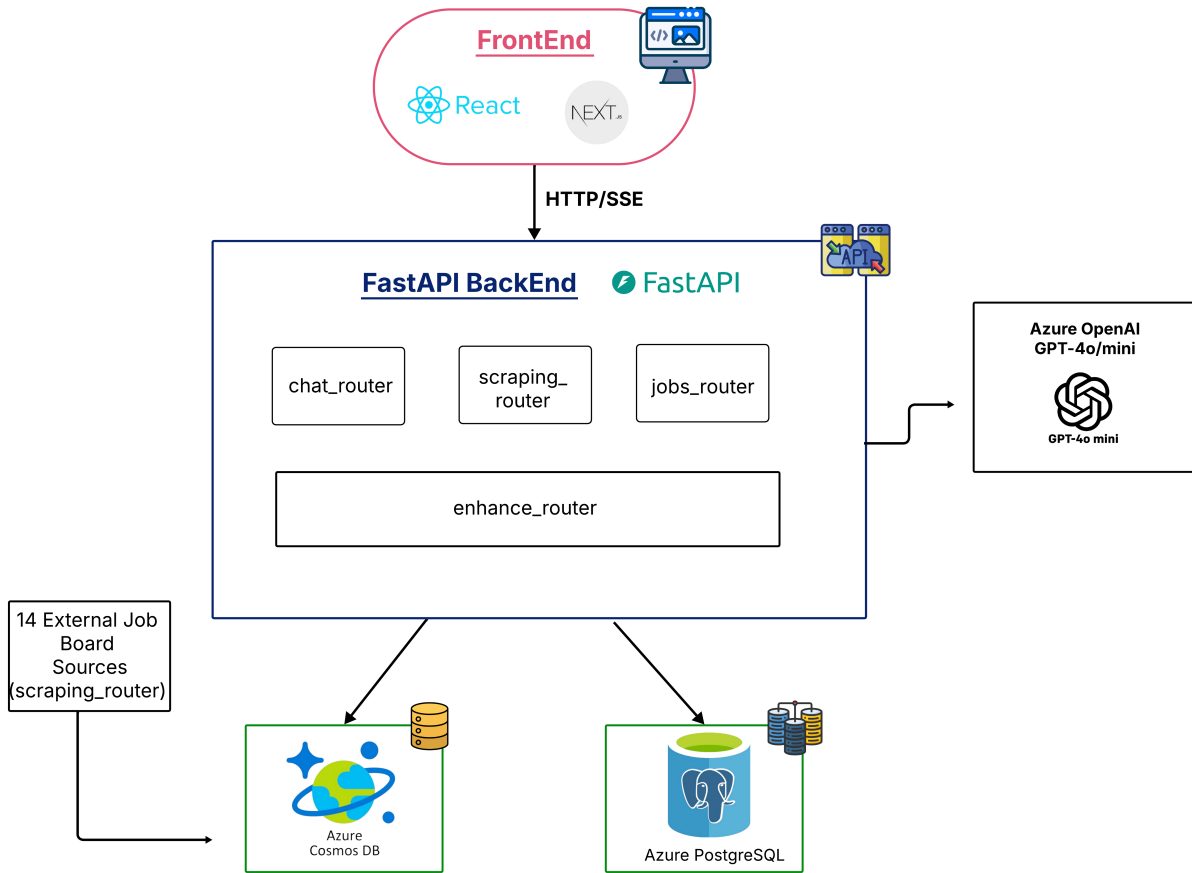


Figure 3.3: Global architecture of the AI Career Assistant system

3.7 Multi-Agent Architecture

Building on the global architecture presented above, this section details the internal orchestration of the four agents coordinated by the Supervisor Agent, each responsible for a distinct functional domain of the AI Career Assistant.

3.7.1 The Supervisor Agent

The Supervisor Agent constitutes the entry point of the agentic pipeline. Upon receiving a learner request whether expressed as free text through the conversational assistant or triggered by a predefined interface action its **analyze_intent** node classifies the underlying intent into one of several

categories (resume enhancement, job search, job-specific compatibility scoring) and routes the request to the corresponding specialized agent through a conditional edge in the LangGraph state graph.

3.7.2 The CV Agent

The CV Agent processes a learner’s resume through a sequence of nine nodes: extraction, domain detection, baseline scoring, missing-section detection, LLM-based improvement, output parsing, post-improvement scoring, DOCX export, and persistence. Its output is the canonical, ATS-optimized CV profile referenced throughout the rest of the system; no document is downloaded at this stage.

3.7.3 The Job Agent

The Job Agent coordinates the discovery and analysis of job opportunities through eight nodes: profile loading, cache verification, multi-source scraping, semantic enrichment and scoring, explanation generation, persistence, skill-gap and roadmap analysis, and report generation.

3.7.4 The ATS Agent

The ATS Agent computes a job-specific compatibility score between the canonical CV and a selected position through a dedicated pipeline: CV preparation, CV structuring, job-description structuring, semantic matching, score computation, and improvement suggestions. This score is subsequently reused as the consistent baseline for the CV tailoring step described in the functional requirements.

3.7.5 Communication Inter-Agents

The interaction between the Supervisor Agent and its three specialized agents follows a strict request-route-aggregate pattern, illustrated in Figure 3.4. When a learner submits a request, the Supervisor Agent’s `analyze_intent` node first classifies the underlying intent, then routes the request to exactly one of the three specialized agents through a conditional edge: the CV Agent if the intent is `boost_cv`, the Job Agent if the intent is `find_jobs`, or the ATS Agent if the intent is `ats_score`. Each specialized agent processes its portion of the workflow independently: the CV Agent returns the canonical CV together with its baseline ATS score, the Job Agent returns the matched job listings along with the identified skill gap and learning roadmap, and the ATS Agent returns the job-specific compatibility score together with its improvement suggestions. These individual results are collected and merged into the final, consolidated response returned to the learner. This explicit, conditional routing logic rather than an implicit or parallel dispatch to all agents is what allows the system to remain auditable: at any point, the exact path a request took through the agent graph can be

reconstructed.

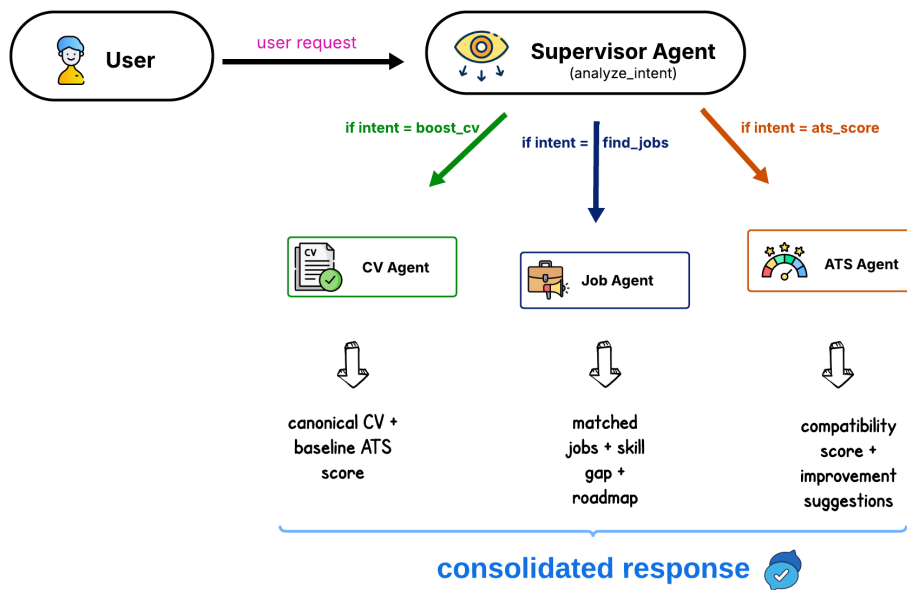


Figure 3.4: Communication flow between the Supervisor Agent and its specialized agents

3.8 Data Flow

Figure 3.5 traces the successive transformations a candidate’s data undergoes across the system, from the initial raw resume upload to the final consolidated career report. Each stage is handled by a distinct module, and the format of the exchanged data evolves accordingly from unstructured raw text, to a structured canonical profile, to numerical compatibility scores, and finally to a structured, human-readable document.



Figure 3.5: Data flow across the AI Career Assistant system, from raw CV upload to the final career report

3.9 Technology Stack

Having presented the system’s architecture, this section details the complete technology stack mobilized to implement it. For each technology, a short definition, its role in this project, and the reason it was preferred over alternative options are given.

3.9.1 Software and Frameworks

Python 3.12 [21] :



Figure 3.6: Python

Python is a general-purpose programming language. It runs the entire backend and hosts the ML pipelines. It was preferred over Node.js or Java for its mature data-science ecosystem (PyTorch, sentence-transformers), unavailable to the same extent in those languages.

FastAPI [22] :



Figure 3.7: FastAPI

FastAPI is a Python web framework for building APIs. It serves all backend routes and coordinates calls to Azure OpenAI, the databases, and the scraping pipeline. It was preferred over Flask or Django for its native asynchronous support, required to avoid blocking on any of these concurrent operations.

Uvicorn [23] :



Figure 3.8: Uvicorn

Uvicorn is an ASGI server. It executes the FastAPI application in production. It was preferred over alternative ASGI servers as the reference implementation recommended for FastAPI, with native SSE support.

React 19 [24] :



Figure 3.9: React

React 19 is a JavaScript library for building user interfaces. It renders the platform's screens (upload, enrichment, job results). It was preferred over Vue or Angular for its declarative state model, well suited to the platform's multi-step user journey.

Next.js [25] :



Figure 3.10: Next.js

Next.js is a React framework. It handles routing and rendering for the frontend. It was preferred over a bare React setup for its built-in routing, reducing configuration overhead.

TypeScript [26] :



Figure 3.11: TypeScript

TypeScript is a typed superset of JavaScript. It types all data exchanged between frontend and backend. It was preferred over plain JavaScript to catch integration errors at compile time rather than at runtime.

GitHub [27] :



Figure 3.12: GitHub

GitHub is a version-control and collaboration platform. It hosted the project's source code and history throughout development. It was preferred over alternatives (GitLab, Bitbucket) for its familiarity and integration with the tools already used by the team.

3.9.2 Large Language Model Provider: OpenAI

OpenAI [28] :



Figure 3.13: OpenAI

OpenAI is the large language model provider used throughout the system. Its models power CV rewriting, XAI explanations, intent classification, and the chatbot. Selecting a provider required weighing several criteria beyond raw generation quality, most notably instruction-following fidelity,

API maturity, and native integration with the chosen cloud infrastructure. Table 3.5 presents this comparison against the other prominent providers considered.

Criterion	OpenAI	Google Gemini	Anthropic Claude	Mistral / Llama / DeepSeek
Instruction-Following Fidelity	Strongest and most consistent adherence to strict, negatively-phrased constraints among providers compared decisive for the non-hallucination requirement of CV rewriting.	Competitive, but not the deciding factor for this project.	Closest competitor on this criterion; strong constraint-respecting behavior.	Competitive reasoning, but less consistent constraint adherence observed at the time of comparison.

Criterion	OpenAI	Google Gemini	Anthropic Claude	Mistral / Llama / DeepSeek
Native Cloud Integration	Only provider offering first-party integration with the project's chosen cloud platform (Azure OpenAI Service), sharing account, identity, and billing with the rest of the infrastructure.	Requires provisioning Google Cloud's Vertex AI alongside Azure.	No equivalent direct Azure integration at the time of this project.	No native Azure integration; would require self-hosting or a third-party gateway.
API Maturity & Structured Output	Most extensively documented API among providers compared, with native structured-output support simplifying response parsing.	Mature API, comparable documentation quality.	Mature API, comparable documentation quality.	Less enterprise-grade API stability at the time of this project's design (DeepSeek); Mistral/Llama APIs less structured-output focused.

Criterion	OpenAI	Google Gemini	Anthropic Claude	Mistral / Llama / DeepSeek
Managed vs. Self-Hosted	Fully managed API, no dedicated inference infrastructure required.	Fully managed API.	Fully managed API.	Open-weight models attractive for self-hosting, but this adds operational burden disproportionate to this project's scale.

Tableau 3.5: Comparison of large language model providers by criterion

Across every criterion compared, OpenAI is the only provider that combines instruction-following fidelity strong enough to sustain a strict non-hallucination guarantee with native, first-party integration into the Azure infrastructure already adopted for the rest of the system the combination that ultimately justified its selection.

3.9.3 Cloud Platform: Microsoft Azure

Microsoft Azure [29] :



Figure 3.14: Microsoft Azure

Microsoft Azure is a cloud computing platform. It hosts the backend, both databases, and the LLM inference through Azure OpenAI Service. Selecting a cloud provider required weighing several criteria beyond a single feature comparison, most notably LLM integration, security, and cost consolidation. Table 3.6 presents this comparison against AWS and Google Cloud, the two other major providers considered.

Criterion	Microsoft Azure	AWS	Google Cloud
LLM Integration	Azure OpenAI Service provides first-party, enterprise-grade access to OpenAI's models under the same subscription as the rest of the infrastructure the decisive criterion for this project, since every generative component depends on GPT-4o and GPT-4o-mini.	OpenAI's models are not natively available. Foundation models are accessed through Amazon Bedrock, which offers other model families, but not OpenAI's.	Vertex AI provides Google's own Gemini family natively, but OpenAI's models are not available on the platform.
Security	Unified identity and access management (Azure Active Directory) spanning compute, storage, and AI services, reducing the number of distinct security boundaries to manage.	AWS IAM is comparably mature, but AI service access (Bedrock) follows a partially separate permission model from core compute and storage.	Google Cloud IAM is comparably mature, with the same observation applying to Vertex AI access relative to core services.

Criterion	Microsoft Azure	AWS	Google Cloud
Cost Consolidation	Consumption-based pricing on Cosmos DB request units and PostgreSQL compute, consolidated with AI usage on a single billing surface.	Comparable consumption-based pricing, but billed across separate services (Bedrock, DynamoDB, RDS) with less consolidated visibility.	Comparable consumption-based pricing, with the same billing consolidation limitation.

Tableau 3.6: Comparison of cloud platforms on the criteria relevant to this project

Across these criteria, the decisive factor remained LLM integration: since every generative component of the system depends on OpenAI’s models, provisioning them under the same account, identity boundary, and billing surface as the rest of the infrastructure an advantage neither AWS nor Google Cloud could offer natively at the time this project was designed outweighed any marginal advantage either competitor might hold on isolated infrastructure criteria.

3.9.4 Machine Learning and AI Orchestration Frameworks

LangGraph [13] :



Figure 3.15: LangGraph

LangGraph is a framework for building stateful, multi-agent LLM applications as explicit graphs. It orchestrates the four agents of the system. It was preferred over AutoGen and CrewAI for its explicit state-graph model and native conditional branching, required for auditable, deterministic routing between agents.

sentence-transformers [30] :



Figure 3.16: sentence-transformers

is a library for generating sentence embeddings from pretrained transformer models. It provides the embedding models used in the matching pipeline. It was preferred over larger embedding APIs for its lightweight, CPU-friendly multilingual models, avoiding dedicated GPU infrastructure.

PyTorch [31] :



Figure 3.17: PyTorch

PyTorch is a deep learning framework. It was used to define and fine-tune the custom regression head of the matching model. It was preferred over scikit-learn, which cannot express custom neural architectures with the same flexibility.

scikit-learn `scikit_learn_docs` :



Figure 3.18: scikit-learn

scikit-learn is a general-purpose machine learning library. It computes the cosine similarity used for real-time listing filtering. It was preferred over a custom implementation for the reliability and simplicity of its vectorized similarity operations.

3.9.5 Specific Machine Learning Models Used

paraphrase-multilingual-MiniLM-L12-v2 :

A pretrained multilingual sentence-embedding model. It encodes CVs and scraped job listings for real-time cosine-similarity filtering. It was preferred over a monolingual model for its French/English coverage, and over a larger cross-lingual model for its inference speed, needed at this early filtering stage.

all-MiniLM-L6-v2 :

A pretrained, lightweight sentence-embedding model. It computes semantic similarity between individual skill terms within the ATS module. It was preferred over a larger encoder for its fast inference, sufficient at the granularity of single skill terms rather than full documents.

BiEncoderRegressorFineTuned (custom fine-tuned model) :

A custom regression model fine-tuned on top of MiniLM-L6-v2. It produces the calibrated CV-job compatibility score used in the matching pipeline. It was preferred over a generic pretrained model because raw cosine similarity cannot capture domain-specific factors such as seniority alignment or industry relevance. A bi-encoder architecture was preferred over a cross-encoder because it allows embeddings to be reused across comparisons, which a cross-encoder's per-pair computation cannot scale to.

3.9.6 Databases**Azure Cosmos DB [32] :****Figure 3.19:** Azure Cosmos DB

Azure Cosmos DB is a managed NoSQL database. It stores canonical CV profiles, scraped job listings, and chat history. It was preferred over a relational database for its flexible schema, suited to CV profiles whose structure varies significantly between candidates.

Azure PostgreSQL [33] :**Figure 3.20:** Azure PostgreSQL

Azure PostgreSQL is a managed relational database. It stores the platform's learning data (labs, certifications, quiz). It was preferred over Cosmos DB for this specific data because it is inherently relational and schema-stable.

3.9.7 Document Extraction and Generation**PyMuPDF (fitz) [34] :****Figure 3.21:** PyMuPDF

PyMuPDF (fitz) is a PDF processing library. It extracts text and embedded photos from PDF resumes. It was preferred over alternative PDF libraries for its extraction speed.

python-docx [35] :

is a library for manipulating Word documents. It extracts and generates DOCX resumes in five visual formats. It was preferred as the reference Python library for this format.

ReportLab [36] :



Figure 3.22: ReportLab

ReportLab is a PDF generation library. It produces the final career report. It was preferred over an HTML-to-PDF conversion approach for its fine-grained, code-driven control over layout.

3.9.8 Web Scraping

aiohttp [37] :



Figure 3.23: aiohttp

aiohttp is an asynchronous HTTP client library. It performs requests to the 14 job-board sources in parallel. It was preferred over a synchronous client for consistency with the system's async-first design.

BeautifulSoup [38] :



Figure 3.24: BeautifulSoup

BeautifulSoup is an HTML parsing library. It parses listings from sources without a structured API. It was preferred for its simple, reliable parsing of arbitrary HTML.

SerpAPI [39] :



Figure 3.25: SerpAPI

SerpAPI is a structured search-results API. It was preferred over direct scraping for the sources that actively block automated access or require an authenticated account to browse listings, most notably LinkedIn and Indeed, as well as AI Jobs, TanitJobs, and WhatJobs. Rather than scraping

these platforms directly, the system queries SerpAPI’s Google Jobs engine with a domain-restricted search (e.g. `site:linkedin.com`) and filters the returned results to the target domain, retrieving publicly indexed listings without triggering the login walls or anti-bot mechanisms these platforms enforce against direct scraping.

Conclusion

This chapter has justified the foundational methodological choices of the project. The adoption of an agentic AI architecture, orchestrated with LangGraph, was shown to be the only approach capable of simultaneously satisfying the reproducibility, explainability, and maintainability the system requires, while avoiding the respective limitations of a single-LLM pipeline and of conventional, AI-free software development. This architectural decision, in turn, motivated a hybrid project methodology combining Scrum for the iterative delivery of the system’s functional blocks with CRISP-ML(Q) for the rigorous handling of the data-science and machine-learning components of the system. The following chapter details the internal design of the system’s core pipelines, its underlying data model, and the prompt-engineering and safeguard mechanisms that ensure the reliability of its generative components.

DETAILED DESIGN

Plan

Introduction	13
1 Problem Statement	13
2 Study of Existing Solutions	15
3 Project Objectives	17
4 Project Scope	18
5 Actors Identification	18
6 Functional Requirements	19
7 Non-Functional Requirements	22
8 Use Case Modeling	24
9 Expected Deliverables	36
Conclusion	37

Introduction

Chapter 3 established the system’s global architecture and introduced the four agents at a functional level. This chapter goes one level deeper: it details the internal state-machine design of the CV Agent and Job Agent pipelines, the complete mathematical formulation of the ATS scoring mechanism, the underlying data model shared across both persistence layers, and the prompt-engineering and safeguard mechanisms that constrain the system’s generative components to remain factually faithful to the source data.

4.1 CV Agent Pipeline Design

The CV Agent processes a candidate’s resume through eight sequential nodes, illustrated in Figure 4.1: extraction, domain detection, baseline scoring, missing-section detection, LLM-based improvement, output parsing, post-improvement scoring, and persistence.

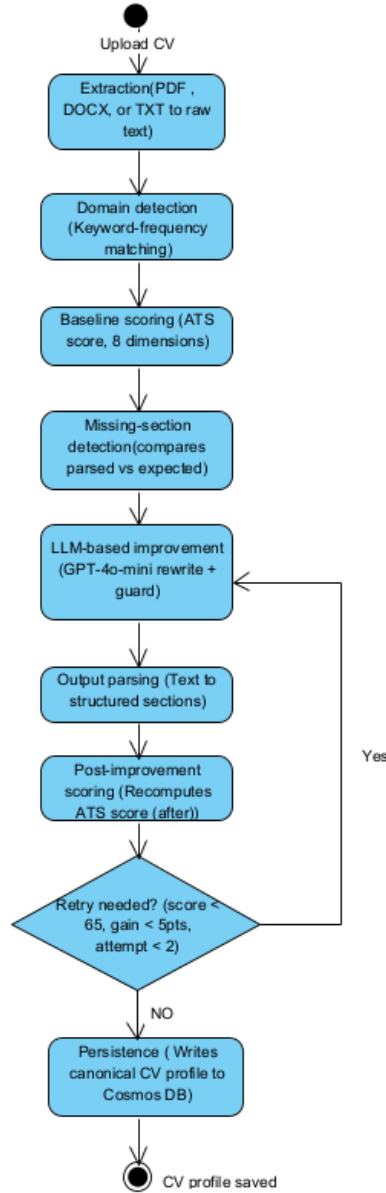


Figure 4.1: State diagram of the CV Agent’s eight-node pipeline

4.1.1 Node-by-Node Description

Extraction : Raw text is extracted from the uploaded document (PDF, DOCX, or TXT), together with any embedded photograph.

Domain detection : The extracted text is classified into one of twelve professional domains through keyword-frequency matching, a deterministic, rule-based classification chosen for its speed and predictability over an LLM-based classification at this early stage.

Baseline scoring : A score out of 100 is computed on the raw extracted text, independently of any target job description, by weighting eight structural and content dimensions of the resume: section presence, skill breadth, experience quality, length, profile summary, contact completeness, keyword density, and language proficiency. This formula is deterministic and rule-based rather than model-driven, a choice made to guarantee that a score presented to the learner as objective progress

remains fully reproducible.

Missing-section detection : The parsed sections are compared against the set of expected resume sections (profile, education, experience, skills, languages) to identify which are absent, informing the enrichment step presented to the learner.

LLM-based improvement : The original resume, together with any enrichment data supplied by the learner in response to the missing sections identified in the previous node additional languages, education, experience entries, or platform certifications is submitted to the language model, which rewrites the entire document into a structured, ATS-optimized version. The model is constrained by a structured prompt built around a non-negotiable anti-hallucination rule and a per-section condition-before-format instruction, and its output is additionally verified by a deterministic guard applied after generation.

Output parsing : The model’s free-text response is parsed into a structured representation (name, contact information, and one ordered list of lines per section), with placeholder lines stripped.

Post-improvement scoring : The same deterministic ATS formula is recomputed on the parsed, improved output, yielding the “after” score compared against the baseline.

Persistence : The resulting canonical profile is written to Azure Cosmos DB as the pipeline’s final step, becoming the reference version consulted by the Job Agent and the ATS Agent. No document is downloaded at this stage: the canonical profile exists purely as a structured reference.

4.1.2 Conditional Retry Logic

Up to two automatic retries are triggered when three conditions hold simultaneously: the post-improvement score remains below 65, the gain achieved over the baseline score is smaller than five points, and the maximum retry count has not yet been reached. In this case, the weakest dimensions of the score breakdown are identified and explicitly named in a second prompt sent to the model, directing it to specifically address those weaknesses using only information already present in the source document. The two resulting rewrites are compared, and the one achieving the higher score is retained.

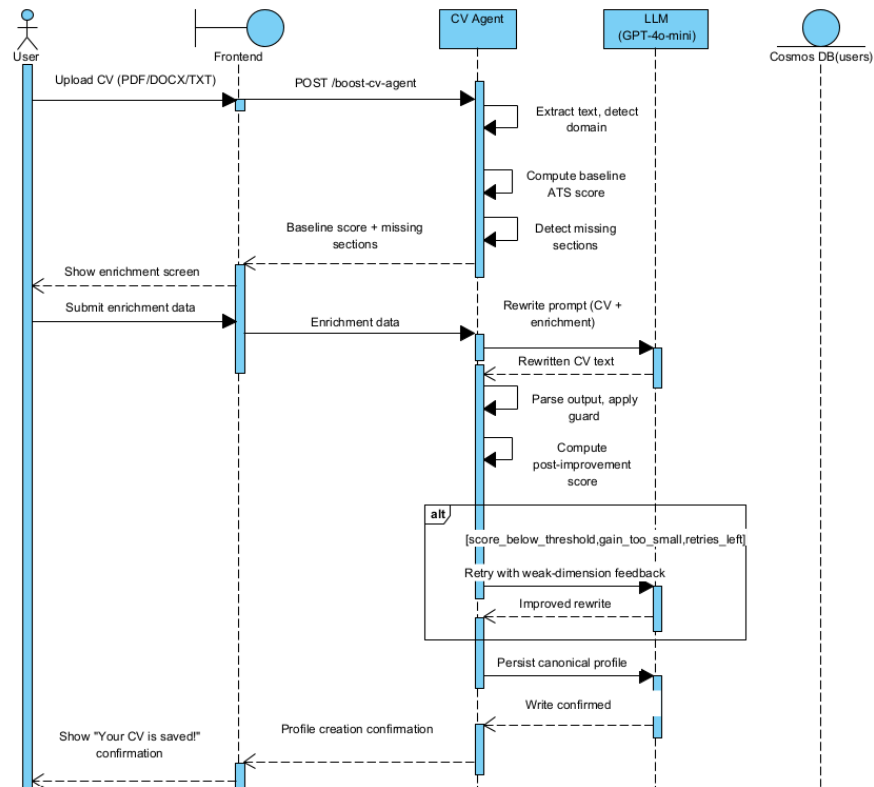


Figure 4.2: Sequence diagram of the resume upload, scoring, enrichment, and persistence flow

4.2 Job Agent Pipeline Design

The Job Agent coordinates opportunity discovery through eight sequential nodes, illustrated in Figure 4.3: profile loading, cache verification, multi-source scraping, semantic enrichment and scoring, explanation generation, persistence, skill-gap and roadmap analysis, and report generation.

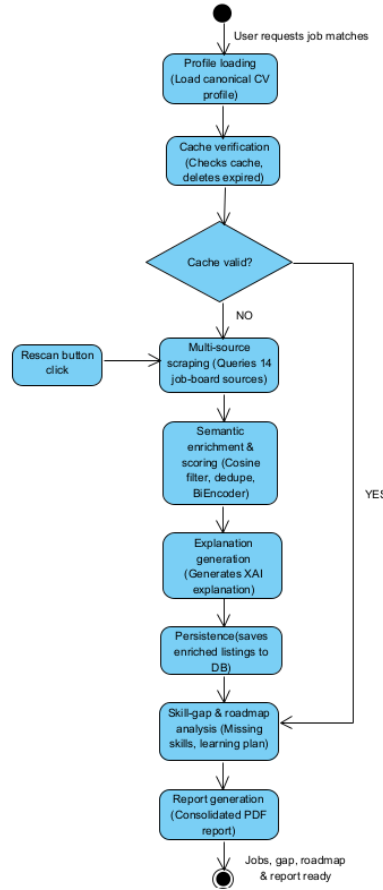


Figure 4.3: State diagram of the Job Agent’s eight-node pipeline

4.2.1 Node-by-Node Description

Profile loading : The canonical CV profile persisted by the CV Agent is retrieved from Cosmos DB, including the inferred job title used to guide the subsequent scraping and matching steps.

Cache verification : The system first checks whether any job listing already exists for the learner in Cosmos DB. Before returning any cached listing, each one is checked against a 45-day expiry window computed from its original publication date on the source job board, rather than from the date it was saved to Cosmos DB, so that a listing scraped late relative to its posting date is not treated as artificially fresh. Listings that have exceeded this window are permanently deleted from Cosmos DB rather than being returned to the learner; listings whose publication date cannot be reliably parsed are conservatively retained. If no valid, non-expired listing remains after this check, scraping is triggered; otherwise, the remaining listings are reused directly, skipping the scraping and enrichment steps entirely. A rescan can also be explicitly requested by the learner through a dedicated action. In this case, the scraping pipeline is simply re-executed rather than clearing existing listings beforehand: each freshly scraped listing that already exists in Cosmos DB, detected through the same duplicate-key mechanism, is skipped without being reprocessed, while only listings not yet present in the learner’s saved data are scored, enriched, and added.

Multi-source scraping : Fourteen job-board sources are queried in parallel either through direct scraping or through SerpAPI-mediated access for platforms that block automated browsing. Each source independently stops collecting listings once it encounters postings older than 45 days, ensuring that only reasonably fresh opportunities enter the pipeline in the first place.

Semantic enrichment and scoring : Each scraped listing is first filtered by cosine similarity between its title and the candidate’s inferred role: listings scoring below 0.60 are discarded before any further, more expensive processing. Before being scored further, a listing is compared against the candidate’s already-saved listings: if a listing with the same identifying key (derived from its URL) or the same title and company already exists, it is skipped entirely and processing moves on to the next listing, preventing duplicate entries. Before any listing is scored, the candidate’s structured CV is prepared once, upfront, before the scraping loop begins: its source language is detected, and it is translated to English only if it is not already in English, so that the fine-tuned bi-encoder matching model (Chapter 5) is always queried in the language it was trained on. For each listing that reaches this stage, its own language is likewise detected; if it is French, its narrative fields (title, description, required education) are translated to English before scoring, while its skill lists are left untouched, since technical skill names are largely language-agnostic and translating them would risk distorting the exact keyword matching used by the skills-gap computation below. Listings that pass both checks are then scored this way, producing an AI-match score; this AI-match score is the criterion ultimately used to rank the listings presented to the learner. If the fine-tuned model is temporarily unavailable, this node flags the affected listings accordingly rather than silently returning a misleading score, a resilience mechanism detailed in Chapter 5. A second, combined score is computed alongside it as $\text{combined} = \text{AI_match} \sqrt{\text{coverage}}$, where *coverage* is the proportion of the listing’s mandatory skills explicitly found in the candidate’s profile (this combined score is calculated because the AI-match score alone can rate a listing as highly compatible on overall semantic and contextual similarity even when the candidate is missing several of the position’s mandatory skills, so the coverage term grounds the combined score in how many of those required skills the candidate concretely possesses). The combined score is not used for ranking, but serves as a fallback scoring criterion when the AI-match score is unavailable, and is passed to the explanation-generation node below.

Explanation generation (XAI) : For each retained listing, a natural-language explanation of its compatibility is generated by the language model, strictly grounded in the four quantities already computed at the previous node – the cosine similarity, the AI-match score, the combined score, and the matched/missing skill breakdown – so that the explanation narrates the result rather than determining it. This step is deliberately designed as an explainable-AI (XAI) [40] layer rather than an LLM-as-judge scheme: the model is never asked to produce or revise the compatibility score itself, only to translate

values already fixed by the deterministic bi-encoder and coverage formulas into a human-readable rationale. This choice keeps the ranking fully reproducible and prevents the language model’s known scoring inconsistencies from propagating into the number the learner ultimately sees and trusts – a risk an LLM-as-judge [41] design would reintroduce by letting the model influence the score it is meant to explain. A purely template-based alternative was also considered and rejected, since fixed sentence templates cannot adapt their wording to the specific combination of matched and missing skills of each listing, whereas the LLM-generated explanation names the exact skills involved in natural, contextual language while remaining strictly grounded in the same computed values. If explanation generation fails or is disabled for a given listing, this node falls back to a deterministic, formula-only explanation object, so a missing narrative never blocks the pipeline.

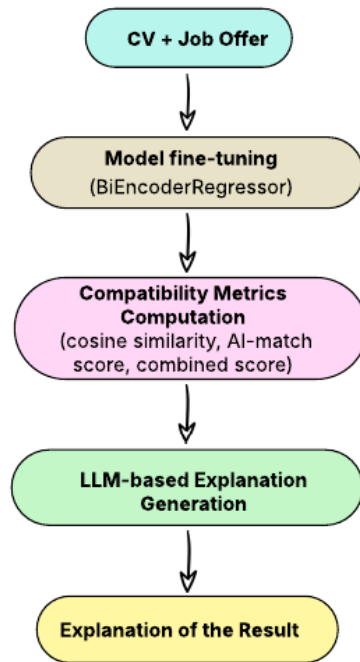


Figure 4.4: Architecture of the compatibility scoring and LLM-based explanation generation pipeline

Persistence : Each enriched, non-duplicate listing is written to Cosmos DB together with its cosine score, AI-match score, combined score, publication date, and explanation.

Report generation : A consolidated report is produced, summarizing the profile, the matched opportunities, the skill gap, and the learning roadmap..

Once this pipeline completes, the learner can consult four complementary views built on its output: a **Matches** view listing every retained job, sorted by AI-match score in descending order; a **Market** view aggregating statistics across the entire corpus of scraped listings (most frequent skills, salary ranges, dominant locations and companies); a **Skill Gap** view detailing the specific

competencies missing relative to the matched market; and a **Roadmap** view presenting the prioritized, prerequisite-ordered learning plan derived from that gap.

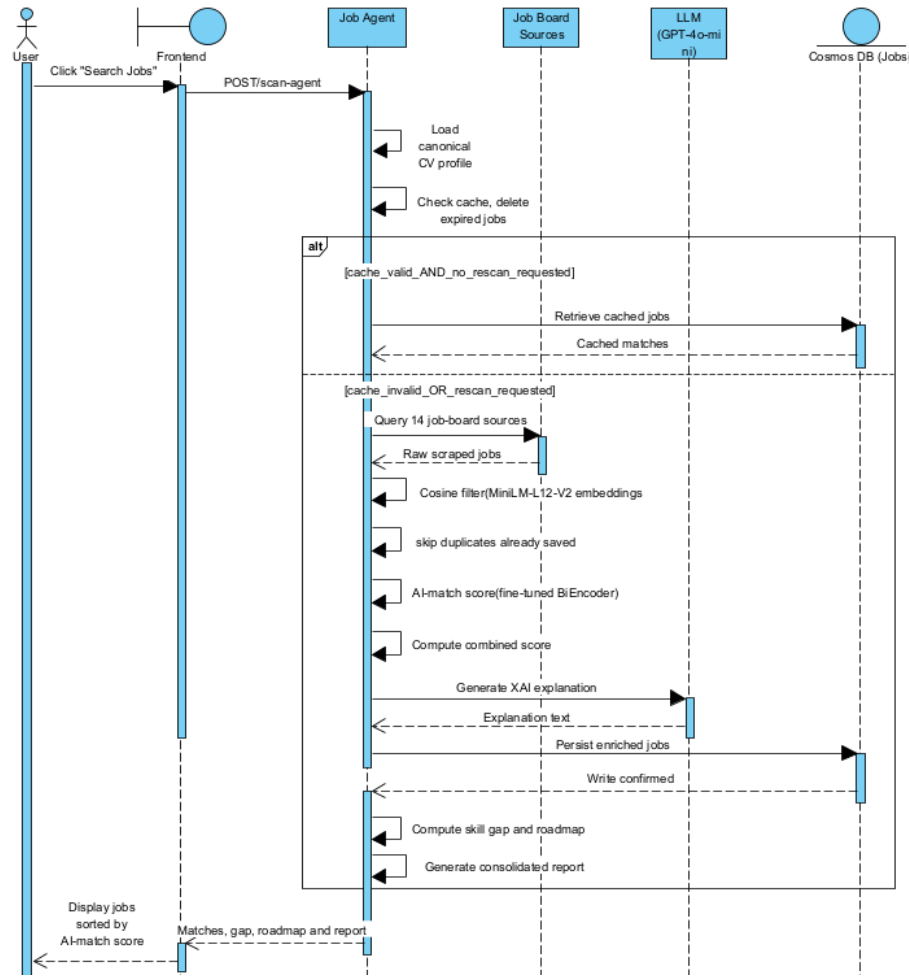


Figure 4.5: Sequence diagram of the multi-source scan pipeline

Note: It should be noted that, while the LangGraph-based Job Agent pipeline described throughout this section is fully implemented and exposed through a dedicated streaming endpoint, the production frontend’s “Search Jobs” action currently defaults to a simpler, non-agentic scraping route for this specific interaction. Wiring this action to the agentic endpoint is identified as a natural next step to fully align the deployed user experience with the architecture presented here.

4.3 ATS Agent Pipeline Design

The ATS Agent computes a job-specific compatibility score between the canonical CV and a selected position through six sequential nodes, illustrated in Figure 4.6: CV preparation, CV structuring, job-description structuring, semantic matching, score computation, and improvement suggestions.

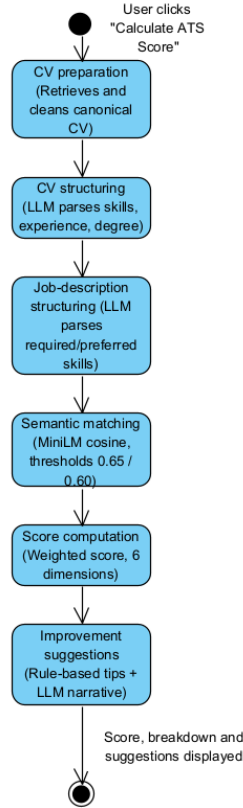


Figure 4.6: State diagram of the ATS Agent’s six-node pipeline

4.3.1 Node-by-Node Description

CV preparation : The canonical CV profile persisted by the CV Agent is retrieved and prepared for comparison against the selected job.

CV structuring : The prepared CV is parsed by the language model into a structured representation isolating its skills, experience, degree, designation, and languages.

Job-description structuring : The selected job is parsed into an equivalent structured representation, isolating its required and preferred skills, required experience, required degree, job title, and required languages.

Semantic matching : The candidate’s skills are compared individually against the job’s required and preferred skills through sentence embeddings, using two distinct similarity thresholds: 0.65 for required skills and 0.60 for preferred skills, to accommodate near-equivalent skill phrasing (for example, “ML” and “Machine Learning”).

Score computation : A weighted compatibility score out of 100 is computed across six independently scored dimensions: required skills (55%), preferred skills (10%), experience (15%), job-title alignment (10%), degree (5%), and languages (5%). This dimension-by-dimension design, distinct from the single AI-match score used by the Job Agent for bulk matching against the scraped job corpus, allows the ATS Agent to produce a fully auditable breakdown for a single, precisely compared

job, together with the specific missing required and preferred skills. The resulting total score though not this detailed breakdown is the value subsequently reused as the consistent baseline for any job-specific enhancement.

Improvement suggestions : Rule-based tips are first generated for each weak dimension of the breakdown (missing required skills, insufficient experience, degree mismatch, missing languages), then enriched with a language-model-generated narrative explanation when available, producing concrete, actionable suggestions displayed to the learner. These suggestions currently remain informational: the subsequent job-specific enhancement step, described later in this chapter, re-extracts relevant keywords independently from the raw job description rather than directly consuming this dimension-by-dimension breakdown an integration gap identified as future work.

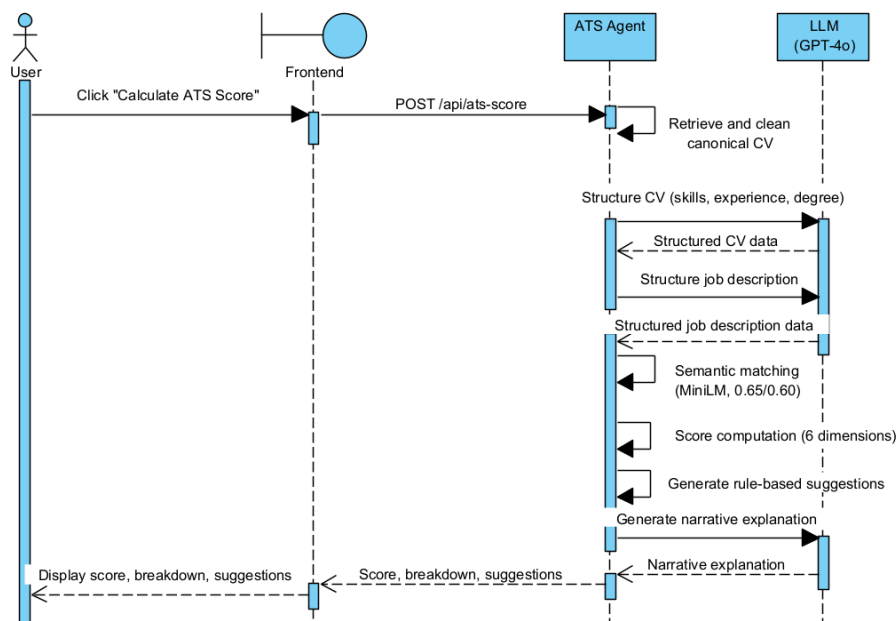


Figure 4.7: Sequence diagram of the ATS Agent’s job-specific scoring pipeline

4.3.2 Job-Specific CV Enhancement Endpoint

Distinct from the ATS Agent’s scoring pipeline described above, a dedicated enhancement endpoint independently re-extracts the target job’s title, description, and requirements from the raw posting text, and submits them to the same LLM-based rewriting mechanism used by the CV Agent (Section ??), tailoring the canonical CV to the selected listing while reusing the ATS Agent’s total score as the consistent “before” baseline.. This endpoint intentionally does not consume the ATS Agent’s dimension-by-dimension breakdown directly, an integration gap discussed further in Chapter 7.

4.4 The ATS Scoring Formula

The ATS scoring mechanism is a deterministic, rule-based formula rather than a learned model, chosen specifically to guarantee the reproducibility a score shown to a learner as objective progress must possess. Table 4.1 details the eight weighted dimensions composing the 100-point score.

Dimension	Weight	Evaluation Criterion
Sections	20	Presence of the key structural sections (profile, education, experience, skills, projects).
Skills	18	Breadth of technical skills explicitly listed, or coverage of job-specific required skills when a target position is known.
Experience	18	Presence, dating, bullet-point structuring, and action-verb usage within the experience section.
Length	12	Total word count, scored against an empirically defined ideal range.
Summary	10	Presence, length, and professional vocabulary of the profile section.
Contact	8	Presence of email, phone number, and LinkedIn profile.
Keywords	8	Density of domain-specific keywords relative to a reference list.
Languages	6	Presence of a languages section with proficiency levels.

Tableau 4.1: Weighted dimensions of the deterministic ATS scoring formula

This formula is computed twice within the CV Agent’s pipeline once on the raw extracted

resume (baseline score) and once on the LLM-rewritten output (post-improvement score) and is entirely independent of the job-specific compatibility formulas used by the Job Agent and the ATS Agent, detailed respectively in the corresponding sections of this chapter.

4.5 Data Model

The system persists data across two databases, each chosen for the structural nature of the information it stores: Azure Cosmos DB, a flexible NoSQL store, for the structurally heterogeneous CV profiles, job listings, and chat history; and Azure PostgreSQL, a relational store, for the inherently stable, tabular learning data of the SUBUL platform. Figure 4.8 presents the class diagram of the entities persisted across both databases.

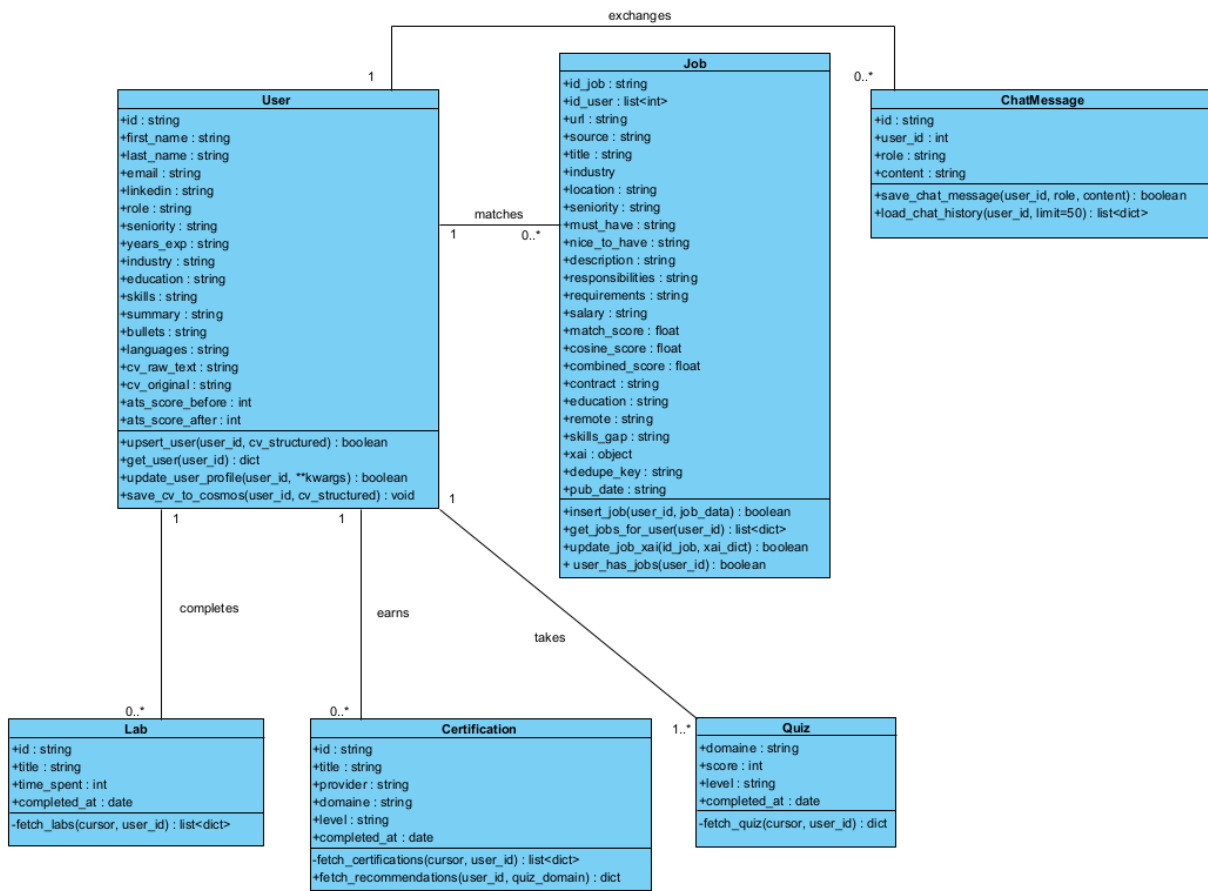


Figure 4.8: Class diagram of the entities persisted in Azure Cosmos DB and Azure PostgreSQL

4.5.1 Azure Cosmos DB Entities

User : Stores the canonical CV profile: identity fields (name, email, LinkedIn), professional fields (role, seniority, years of experience, industry, education, skills, summary, achievement bullets, languages), the raw and original CV text, and the baseline and post-improvement ATS scores.

Job : Stores each matched listing: title, description, required skills, location, seniority, source, URL, publication date, cosine-similarity score, fine-tuned model score, combined compatibility score,

and its associated explanation.

ChatMessage : Stores each turn of the conversational assistant: a unique identifier, the user identifier, the role (user or assistant), and the message content. Messages are retrieved in insertion order for a given user, which is sufficient to reconstruct the conversation as experienced within a session.

4.5.2 Azure PostgreSQL Entities

Lab : Stores a completed hands-on exercise on the SUBUL platform: identifier, title, completion date, and score.

Certification : Stores a validated certification: identifier, title, issuing organization, and date.

Quiz : Stores the learner’s positioning-quiz result: detected domain, score, and proficiency level.

These platform-side entities are inherently relational and schema-stable, in contrast with the structurally heterogeneous CV profiles and job listings stored in Cosmos DB a distinction that motivated the choice of two separate persistence technologies rather than a single database for the entire system.

4.6 Prompt Engineering and Anti-Hallucination Safeguards

The reliability of the CV Agent’s generative step rests on two complementary mechanisms: a structured prompt that constrains the model’s behavior at generation time, and a deterministic guard that verifies its output afterward.

4.6.1 Structured Prompt Design

The system prompt submitted to the language model is organized around an explicit priority hierarchy rather than a flat list of instructions. It opens with a non-negotiable rule, stated before any formatting guidance, prohibiting the invention of any degree, employer, skill, or achievement not explicitly present in the source document. Each resume section is then governed by a two-step instruction: first, a condition verifying whether genuine data exists in the source CV for that section; second, and only if that condition is met, a formatting instruction for how to render it. This condition-before-format structure is what prevents the model from filling an empty section with plausible-sounding but fabricated content, a common failure mode of less constrained prompts. The prompt further specifies the target language (detected from the source CV), injects any learner-provided enrichment data (missing languages, education, or experience entries), and explicitly lists any section the learner chose to omit.

4.6.2 The Anti-Hallucination Guard

Even with this prompt structure, a language model can still generate a section unsupported by the source document, particularly when the source CV is sparse. To defend against this, the system applies a deterministic post-generation check: the raw source text is scanned for lexical signals associated with education (degree names, institution keywords) and with professional experience (employment and internship keywords). If the model's output contains an EDUCATION or PROFESSIONAL EXPERIENCE section while the corresponding signal is entirely absent from the source text, that section is programmatically removed from the output before it is parsed, exported, or persisted. This mechanism is a direct application of a principle followed throughout the system's design: a generative step is never trusted on its own to enforce a hard constraint, since a constraint merely requested in a prompt is not a constraint enforced. The overall interaction between the LLM call, this guard, and the CV Agent's conditional retry logic was already illustrated in Figure 4.2; Figure 4.9 instead details the internal decision logic of the guard itself.

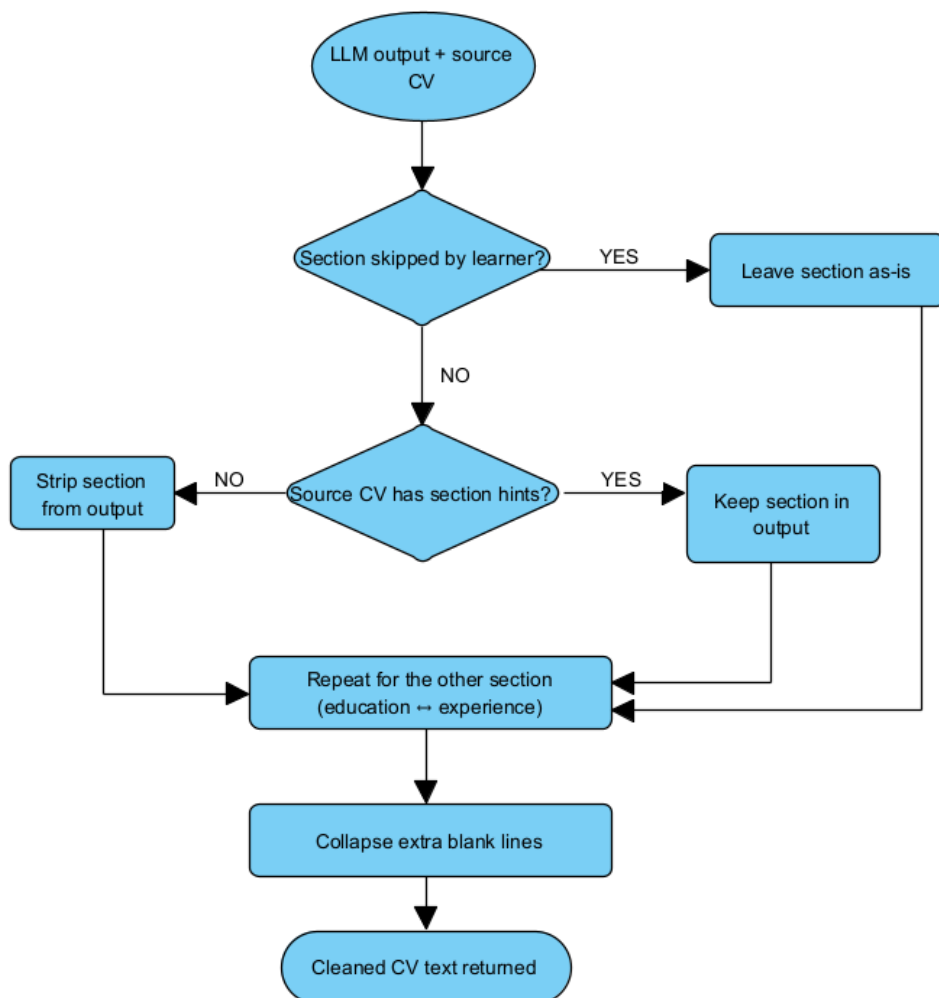


Figure 4.9: Internal decision logic of the anti-hallucination guard

Conclusion

This chapter has detailed the internal design underlying the system’s architecture: the node-by-node state machines of the CV Agent, the Job Agent, and the ATS Agent, the weighted formula guaranteeing the reproducibility of the ATS score, the data model shared across the system’s two databases, and the prompt-engineering and guardrail mechanisms that constrain the reliability of the system’s generative components. The following chapter is entirely dedicated to the fine-tuned matching model referenced throughout this chapter, examined according to the CRISP-ML(Q) phases specifically applied to this artifact.

FINE-TUNED CV-TO-JOB MATCHING MODEL

Plan

Introduction	39
1 Why an Agentic AI Architecture	39
2 Software Development Methodology: Scrum	40
3 Data Science Methodology: CRISP-ML(Q)	42
4 Hybrid Methodology: Combining Scrum and CRISP-ML(Q)	44
5 CRISP-ML(Q) Phases Applied to the Project	48
6 Global System Architecture	49
7 Multi-Agent Architecture	50
8 Data Flow	52
9 Technology Stack	52
Conclusion	63

Introduction

This chapter is entirely dedicated to a single artifact: the fine-tuned bi-encoder regression model that produces the AI-match score used by the Job Agent, as established in Chapter 4. While Chapter 3 justified the adoption of CRISP-ML(Q) at the level of the entire project, and Chapter 4 briefly situated this model within the Job Agent’s pipeline, this chapter examines it in full depth by applying the six CRISP-ML(Q) phases specifically to it: business and data understanding, data preparation, modeling, evaluation, deployment, and monitoring. It closes with an honest account of a methodological limitation affecting one part of the evaluation, disclosed rather than concealed.

5.1 Business and Data Understanding

The business objective of this artifact is narrow and precise: given a learner’s CV and a scraped job posting, produce a single, calibrated compatibility score between 0 and 100 that reflects not only surface keyword overlap, but seniority alignment, industry relevance, and experience adequacy. A generic pretrained sentence-embedding model, compared through raw cosine similarity, has no notion of these domain-specific factors; fine-tuning was therefore required to align the model’s output with what recruitment relevance actually means in this context, rather than with generic semantic similarity alone.

No proprietary CV or job data was available in sufficient volume for this purpose at the time the model was built. The publicly available HuggingFace dataset `michaelozon/candidate-matching-synthetic` [42] was used instead, providing 10,000 synthetic resumes, 2,500 synthetic job postings, and a match table linking each job to its relevant resumes. Although synthetic, this dataset exposes the same structural fields (role, seniority, industry, years of experience, skills) that the system’s own canonical CV profiles and scraped job listings expose, making it a reasonable proxy for the target domain.

5.2 Data Preparation

5.2.1 Ground-Truth Scoring Function

Since the dataset provides only a binary relevance signal (a resume is either listed as relevant to a job or not), a deterministic scoring function was designed to convert this binary signal into a continuous 0–100 training target, combining five weighted factors:

Factor	Weight	Rule
Required skills overlap	40	Proportion of the job's mandatory skills present in the resume, scaled to 40 points.
Preferred skills overlap	15	Same principle applied to the job's nice-to-have skills.
Seniority alignment	± 20	+20 if exact match, +10 if one level apart, -10 if two levels apart (over- or under-qualified alike).
Industry match	20	Flat bonus if the resume's industry equals the job's industry.
Experience penalty	-30	Penalizes a shortfall between the job's minimum expected experience and the candidate's actual years of experience.

Tableau 5.1: Weighted factors of the deterministic ground-truth scoring function

The resulting score, clipped to $[0, 100]$, serves as the regression target the fine-tuned model is trained to reproduce from raw text alone, without direct access to these structured fields at inference time. This same function is reused, unmodified, later in this chapter to construct the ranking evaluation's ground truth (Section 5.4.3), ensuring both the training signal and the evaluation signal are grounded in one single, consistent definition of relevance.

5.2.2 Pair Construction and Split

Positive pairs were built directly from the dataset's match table (each job paired with its listed relevant resumes); negative pairs were built by sampling, for each job, 30 resumes not listed as relevant. Both groups were scored with the function above, concatenated, shuffled, and reduced to a fixed working set of 150,000 pairs. This set was split, stratified by label, into: Since these sampled negatives are only *assumed* non-relevant (a resume simply absent from the dataset's capped list of listed matches, rather than confirmed irrelevant), a small proportion may in fact be reasonable, unlisted matches a common

and generally tolerated approximation in this kind of implicit negative sampling, whose impact is diluted by the large pool of true negatives available per job.

- **112,500 pairs (75%)** for training,
- **15,000 pairs (10%)** for validation,
- **22,500 pairs (15%)** held out for final testing, never used during training or hyperparameter selection.

Each pair’s resume and job posting were serialized into a single structured text string (“**Role:** ... | **Seniority:** ... | **Skills:** ...”), matching the format used throughout the rest of the system when constructing model inputs.

5.3 Modeling

5.3.1 Architecture

The model, `BiEncoderRegressorFineTuned`, is built on top of `sentence-transformers/all-MiniLM-L6-v2` a distinct, English-only variant from the multilingual `paraphrase-multilingual-MiniLM-L12-v2` model used elsewhere in the system for the scraping pipeline’s cosine filter (Chapter 3). Notably, this is the same base checkpoint used by the ATS Agent’s own skill-level semantic matching (Chapter 4, `ats_matcher.py`), which relies on the same `all-MiniLM-L6-v2` model prior to any task-specific fine-tuning — the two components share a backbone, but the fine-tuned regression model examined in this chapter is a separate, specialized artifact built on top of it.

The resume and job texts are each encoded independently through the same shared encoder (a true bi-encoder, not a cross-encoder, for the scalability reasons discussed in Chapter 3), then mean-pooled over non-padding tokens into a single 384-dimensional vector:

$$\mathbf{e} = \frac{\sum_{t=1}^T m_t \cdot \mathbf{h}_t}{\sum_{t=1}^T m_t} \quad (5.1)$$

where $\mathbf{h}_t$ is the encoder’s hidden-state output for token t , $m_t \in \{0, 1\}$ is the attention mask (1 for real tokens, 0 for padding), and T is the sequence length. This averages only the genuine tokens of the input, ignoring padding, producing a single fixed-size vector regardless of the original text’s length.

Four interaction features are then concatenated: the two embeddings themselves, their absolute difference, and their element-wise product, yielding a 1536-dimensional feature vector:

$$\mathbf{f} = [\mathbf{e}_{cv}, \mathbf{e}_{job}, |\mathbf{e}_{cv} - \mathbf{e}_{job}|, \mathbf{e}_{cv} \odot \mathbf{e}_{job}] \quad (5.2)$$

This feature vector is passed through a regression head (1536→768→256→64→1), with batch

normalization and dropout (0.2) after each of the first two hidden layers, and a final sigmoid activation constraining the output to $[0, 1]$, rescaled to a 0–100 score at inference time. The model totals 24.1M parameters, of which 5.1M are trainable.

5.3.2 Training Configuration

The model was trained for 10 epochs with a batch size of 32, using **AdamW** [43] with a differential learning rate (the encoder at 110^{-5} , the regression head at ten times that rate) and a one-cycle learning-rate schedule [44]. Gradient norms were clipped to 1.0. Training ran entirely on CPU, on Azure Machine Learning, no GPU being available in the allocated compute environment; this explains the comparatively long training time for a model of this size.

5.4 Evaluation

5.4.1 Regression Performance on the Held-Out Test Set

Table 5.2 reports the model’s performance on the 22,500-pair test set introduced in Section 5.2.2, which was never used during training or hyperparameter tuning. The three regression metrics reported below are defined as:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \quad (5.3)$$

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad \text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (5.4)$$

where y_i is the true ground-truth score, $\hat{y}_i$ the model’s predicted score, $\bar{y}$ the mean of all true scores, and n the number of test pairs (22,500). R^2 measures the proportion of variance in the true scores explained by the model (1.0 is perfect); MAE gives the average error in points; RMSE gives a similar error measure that penalizes large individual errors more heavily than MAE.

Metric	Value	Interpretation
R^2	0.9890	The model explains 98.9% of the variance in the ground-truth score.

Metric	Value	Interpretation
MAE	1.82 pts	Average absolute error, on a 0–100 scale.
RMSE	2.51 pts	Root-mean-square error, penalizing larger errors more heavily.
Mean bias	−0.10	Near-zero, indicating no systematic over- or under-scoring.

Tableau 5.2: Regression performance on the held-out test set

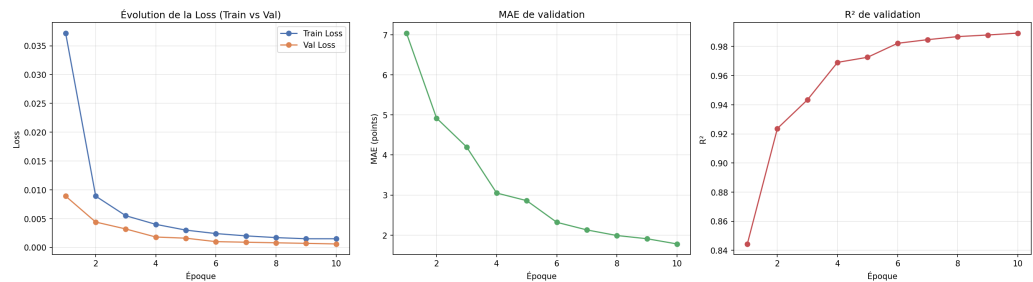


Figure 5.1: Training curves: loss, validation MAE, and validation R^2 over 10 epochs

Comparing validation and test performance confirms the model generalizes rather than overfits: validation R^2 reached 0.9892, against a test R^2 of 0.9890 — a gap of only 0.0002, well under the 0.02 threshold conventionally used to flag overfitting. The training loss decreased monotonically across all 10 epochs without the validation loss ever diverging upward, further supporting this conclusion.

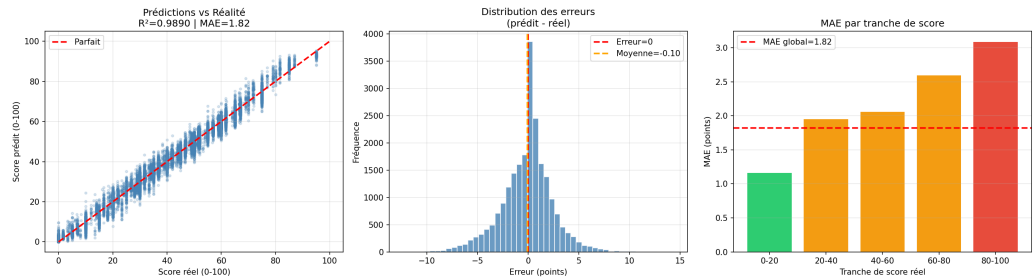


Figure 5.2: Regression performance on the held-out test set

The error distribution is tightly centered around zero, with a standard deviation small enough that over 90% of predictions fall within 10 points of the true score. Error increases mildly for the

highest score band (80–100), a common pattern reflecting the greater difficulty of finely discriminating among already-strong matches.

5.4.2 Qualitative Benchmark Suite

Beyond aggregate regression metrics, a suite of five targeted qualitative tests was run to probe specific failure modes a single R^2 value cannot reveal. Each test is described below with what it does and why it is needed, since a regression metric alone cannot reveal these specific failure modes.

Discrimination test. *What it does:* scores six hand-crafted CV–job pairs spanning the full difficulty spectrum, from a perfect match to a complete mismatch. *Why:* verifies that the model’s output is not just numerically accurate on average, but correctly orders qualitatively different levels of fit.

```
=====
👤 VERDICT BENCHMARK
=====
⚠️ CERTAINS CAS HORS RANGE – vérifier
R² test : 0.9249
MAE test : 5.91 pts
Vitesse : 26.5 ms/inférence
=====
```

Cas	Attendu	Obtenu	Statut
✅ Match Parfait	~75-100	97.3	✅
👉 Bon Match (industrie diff.)	~50-75	57.8	✅
👉 Match Partiel (junior →senior)	~30-55	25.8	⚠️ Hors range
👉 Match Partiel (skills manquants)	~10-50	28.1	✅
❌ Mauvais Match (rôle diff.)	~0-20	1.1	✅
❌ Aucun rapport	~0-10	0.0	✅

Figure 5.3: Discrimination benchmark console output

Result: five of the six cases fell within their expected range (97.3 perfect match, 57.8 good match, 28.1 missing skills, 1.1 wrong role, 0.0 no relation); the junior-to-senior case scored 25.8 against an expected 30–55, falling just below range.

Adversarial resistance test. *What it does:* scores three deliberately deceptive resumes designed to look artificially strong: one keyword-stuffed with zero real experience, one with a matching job title but entirely unrelated skills, and one with correct seniority and skills but a completely unrelated industry. *Why:* a model relying too heavily on superficial keyword overlap could be fooled into over-scoring these cases; this test checks whether the model instead penalizes them appropriately, as a real recruiter would.

```

=====
🔗 TESTS ADVERSARIAUX – CVs trompeurs
=====
Objectif : le modèle doit pénaliser ces CVs malgré les bons mots-clés

```

Cas	Max attendu	Obtenu	Statut
🔥 CV keyword-stuffé (0 exp) └ Plein de bons mots-clés mais débutant absolu	< 40	31.0	✅ Résiste
🔥 Bon titre, mauvaises skills └ Titre correct mais compétences hors sujet	< 30	18.7	✅ Résiste
🔥 Senior DS mais industrie Restaurant └ Seniority et skills ok mais industrie incompatible	< 50	53.8	❌ TROMPÉ

```

=====
⚠️ 1/3 cas : modèle trompé par des CVs artificiels
→ Le modèle accorde trop de poids aux mots-clés vs expérience réelle
=====

```

Figure 5.4: Adversarial resistance benchmark console output

Result: two of the three cases were correctly resisted – the keyword-stuffed, zero-experience resume scored 31.0/100 against a <40 threshold, and the mismatched-skills-with-matching-title case scored 18.7/100 against a <30 threshold. The industry-mismatch case was not resisted: a resume with correct seniority and skills but an unrelated industry (Restaurant, against a FinTech-context posting) scored 53.8/100 against an expected <50, an honest limitation carried forward to the Monitoring and Maintenance section of this chapter.

Coherence test. *What it does:* scores two groups of resumes against a single fixed job posting, varying only one factor at a time (first industry, then seniority) while holding all other fields constant. *Why:* isolates whether the model’s score changes follow a logically expected direction when a single dimension is varied, rather than reacting erratically.


```

TEST COHÉRENCE – Industrie & Séniorité inversée
=====

✦ Groupe 1 – Impact industrie (même rôle DS)
Règle attendue : CV_FinTech > CV_Healthcare > CV_Restaurant

DS Senior – FinTech (industrie identique)      → 91.0/100
└─ Attendu : le plus haut
DS Senior – Healthcare (industrie proche)      → 56.4/100
└─ Attendu : score intermédiaire ~60
DS Senior – Restaurant (industrie lointaine)   → 56.9/100
└─ Attendu : le plus bas du groupe

✗ Ordre NON respecté : 91.0 | 56.4 | 56.9
→ Le modèle ne distingue pas bien la séniorité/industrie
-----

✦ Groupe 2 – Inversion séniorité RH
Règle attendue : CV_Junior > CV_Mid > CV_Senior (surqualifié pénalisé)

RH Junior – Retail (match parfait séniorité)   → 96.4/100
└─ Attendu : le plus haut
RH Mid – Retail (séniorité légèrement supérieure) → 90.4/100
└─ Attendu : score intermédiaire ~50
RH Senior – Retail (surqualifié)               → 66.2/100
└─ Attendu : score plus bas (surqualifié)

✅ Ordre respecté : 96.4 > 90.4 > 66.2
-----

⚠ INCOHÉRENCE DÉTECTÉE – Le modèle ne trie pas toujours bien
=====

```

Figure 5.5: Coherence test console output

Result: the seniority-inversion group preserved the expected ordering ($96.4 > 90.4 > 66.2$, correctly penalizing overqualification for an entry-level posting); the industry-proximity group did not — FinTech scored highest as expected (91.0), but Healthcare (56.4) and Restaurant (56.9) were reversed and nearly indistinguishable despite representing markedly different industry proximity to the posting.

Linguistic robustness test. *What it does:* compares scores for semantically equivalent CV/job pairs expressed differently in wording, first via abbreviations (“ML” vs “Machine Learning”), then via full English-to-French translation. *Why:* a model that scores semantically identical content very differently based on surface wording would be unreliable across the varied phrasing real CVs and job postings actually use.

```

=====
🌐 TEST ROBUSTESSE LINGUISTIQUE
=====
Objectif : scores similaires pour des CV identiques en
anglais vs français, et abréviations vs mots complets

★ Abréviations vs Mots complets
  CV avec abréviations (ML, NLP, DL)          → 94.1/100
  CV avec mots complets (Machine Learning, NLP...) → 97.7/100
  Différence : 3.6 pts (tolérance : ≤ 10 pts)
  → ✅ Robuste

★ CV Anglais vs CV Français
  CV en anglais          → 92.6/100
  CV en français (même contenu) → 87.3/100
  Différence : 5.3 pts (tolérance : ≤ 15 pts)
  → ✅ Robuste

=====
✅ ROBUSTESSE LINGUISTIQUE OK – Scores stables
=====

```

Figure 5.6: Linguistic robustness test console output

Result: abbreviation pairs stayed within the 10-point tolerance; English-French pairs showed a larger, but still bounded, gap — expected and disclosed, since the MiniLM backbone is not a multilingual model.

Calibration test. *What it does:* scores two deliberately ambiguous, “borderline” CV–job pairs, neither clearly strong nor clearly weak matches. *Why:* checks that the model’s score scale is meaningful in practice — that a genuinely intermediate case actually lands in an intermediate score range (30–65) rather than being pushed toward an artificially extreme value, which would distort the ATS-style verdicts (Strong/Good/Partial/Weak) built on top of this score.

```

=====
🚧 TEST CALIBRATION – Est-ce que 50/100 = vraiment "moyen" ?
=====
Méthode : on crée des cas "frontière" et vérifie qu'ils
tombent autour de 50. Un bon modèle calibré le fait.

★ DS Mid – industrie diff + skills partiels
  Note   : Même rôle mais industrie diff + Junior vs Senior + skills partiels
  Score  : 19.0/100 (zone neutre attendue : 35-65)
  → 🚩 Hors zone neutre [35-65]

★ RH Mid – même industrie, rôle proche
  Note   : Même industrie, rôle proche mais légèrement surqualifié
  Score  : 92.7/100 (zone neutre attendue : 30-60)
  → 🚩 Hors zone neutre [30-60]

=====
🚩 CALIBRATION À REVOIR – Les cas moyens ne tombent pas autour de 50
→ Le modèle est probablement trop optimiste ou trop pessimiste
=====

```

Figure 5.7: Calibration test console output

Result: neither borderline case landed in its expected neutral range: the industry-mismatch-with-partial-skill case scored 19.0/100 against an expected 35–65, and the same-industry, slightly-overqualified case

scored 92.7/100 against an expected 30–60. Rather than settling into an intermediate band, the model pushed both ambiguous cases toward an extreme. **Interpretation.** The remaining adversarial failure, the industry-proximity inconsistency observed in the coherence test, and one of the two miscalibrated borderline cases all point to the same root cause, traceable to the ground-truth scoring function itself (Section 5.2.1): the industry factor is a flat binary bonus (+20 for an exact match, +0 otherwise), with no notion of relative industry proximity. The model appears to have faithfully learned this structural property of its training signal rather than exhibiting an independent learning defect: any two non-matching industries are treated as equivalently distant regardless of their actual domain proximity, which explains why Healthcare (56.4) and Restaurant (56.9) received near-identical scores against the same FinTech-seniority-matched posting in the coherence test, and why a Restaurant-industry candidate with otherwise correct seniority and skills was still scored above the adversarial threshold here.

A graded industry factor is identified as a concrete direction for future work, replacing the current binary match with a continuous similarity measure between industry names – for instance, cosine similarity computed with the multilingual sentence encoder already used elsewhere in the system (Chapter 3), rather than a hand-picked per-pair value, to keep the factor’s scale objectively derived rather than manually chosen. Since this factor feeds directly into the ground-truth scores used to train the model (Section 5.2.1), adopting it would require regenerating all 150,000 training pairs and retraining the model from scratch, rather than a lightweight adjustment to the existing checkpoint. Given the substantial time this full retraining cycle would demand, this improvement is deliberately left as a perspective for future work rather than undertaken within the scope of this project.

5.4.3 Baseline Comparison and Ranking Evaluation

To situate the fine-tuned model’s contribution relative to a non-learned baseline, and to evaluate it on the ranking task the deployed system actually performs (given one learner’s CV, rank candidate job postings), a dedicated post-hoc evaluation script was run without any retraining of the saved model. This evaluation reuses the same ground-truth scoring function defined in Section 5.2.1, so that the notion of relevance used here is identical to the one the model was trained against, rather than a separately defined criterion.



Figure 5.8: TF-IDF vs fine-tuned MiniLM: scores, ranking, cross-validation, and pipeline demonstration

Section A (Baseline separation) : Compared against a TF-IDF [45] and cosine-similarity baseline, the fine-tuned model achieves a markedly larger separation between relevant and non-relevant scores (47.6 points versus 14.3 points), and a near-perfect pair-level accuracy (99.9% versus 90.1%) — confirming that the fine-tuning effort yields tangible value over a purely lexical method.

```

=====
SECTION A — BASELINE COMPARISON: TF-IDF vs Fine-Tuned MiniLM
=====

Average score by ground-truth relevance:
Method          Relevant Jobs   Non-Relevant   Separation
-----
TF-IDF + Cosine      21.7         7.4          +14.3
Fine-Tuned MiniLM    65.0        17.4          +47.6

Pair-level accuracy (relevant job ranked above random non-relevant):
Method          Accuracy
-----
TF-IDF + Cosine      90.1%
Fine-Tuned MiniLM    99.9%

INTERPRETATION:
- A perfect model always scores relevant jobs higher than non-relevant ones
- Separation = avg(relevant) - avg(non-relevant): higher is better
- MiniLM advantage: understands semantics (ML = Machine Learning, etc.)

```

Figure 5.9: Console output: baseline separation, TF-IDF vs MiniLM

Section B (Ranking metrics) : Three complementary ranking metrics were computed at

cutoffs $K \in \{1, 3, 5, 10\}$:

$$\text{Precision@}K = \frac{|\{\text{relevant jobs in top-}K\}|}{K} \quad (5.5)$$

$$\text{Recall@}K = \frac{|\{\text{relevant jobs in top-}K\}|}{|\{\text{all relevant jobs}\}|} \quad (5.6)$$

$$\text{NDCG@}K = \frac{\text{DCG@}K}{\text{IDCG@}K}, \quad \text{DCG@}K = \sum_{i=1}^K \frac{\text{rel}_i}{\log_2(i+1)} \quad (5.7)$$

where NDCG follows the standard formulation introduced by Järvelin and Kekäläinen [46].

where rel_i is the graded relevance (the ground-truth score/100) of the item ranked at position i , and $\text{IDCG@}K$ is the DCG of the ideal, perfectly sorted ranking. $\text{Precision@}K$ measures how many of the top- K jobs shown to the learner are genuinely relevant; $\text{Recall@}K$ measures how many of all relevant jobs are captured within the top- K ; $\text{NDCG@}K$ additionally penalizes relevant jobs that are present but ranked too low, rewarding the best matches being placed at the very top rather than merely being included somewhere in the list.

Table 5.3 reports these metrics, computed over 44 evaluated learner profiles, evaluated from the learner’s perspective: ranking candidate job postings for a fixed CV, mirroring the Job Agent’s actual use case (Chapter 4) rather than the inverse, recruiter-style direction of ranking candidates for a fixed job.

```

=====
SECTION B – RANKING METRICS (Precision@K, Recall@K, NDCG@K)
Perspective: for a given learner's CV, rank candidate job postings
=====

(Metrics computed over 44 resumes with at least one relevant job and enough candidate
s per K)

Metric          TF-IDF      MiniLM      Gain (MiniLM - TF-IDF)
-----
Precision@1      0.5455      1.0000      +0.4545  ↑  (n=44)
Precision@3      0.5000      0.7652      +0.2652  ↑  (n=44)
Precision@5      0.4909      0.5591      +0.0682  ↑  (n=44)
Precision@10     0.2886      0.2886      +0.0000  ≈  (n=44)
Recall@1         0.2114      0.4924      +0.2811  ↑  (n=44)
Recall@3         0.5197      0.8788      +0.3591  ↑  (n=44)
Recall@5         0.8705      0.9841      +0.1136  ↑  (n=44)
Recall@10        1.0000      1.0000      +0.0000  ≈  (n=44)
NDCG@1          0.7399      0.9966      +0.2567  ↑  (n=44)
NDCG@3          0.7858      0.9967      +0.2109  ↑  (n=44)
NDCG@5          0.8498      0.9981      +0.1483  ↑  (n=44)
NDCG@10         0.8964      0.9988      +0.1024  ↑  (n=44)

KEY RESULT – NDCG@5:
  TF-IDF  NDCG@5 = 0.8498  (n=44)
  MiniLM  NDCG@5 = 0.9981  (n=44)  (gain=+0.1483)

INTERPRETATION:
- Precision@K: of the top-K jobs shown to the learner, how many are truly relevant?
- Recall@K: of all relevant jobs, how many appear in the top-K shown?
- NDCG@K: are the best jobs ranked at the very top, not just present in top-K?
- A gain > 0 means MiniLM outperforms TF-IDF on that metric
=====

```

Figure 5.10: Console output: Precision@K, Recall@K, and NDCG@K

Metric	TF-IDF	MiniLM	Gain
Precision@1	0.5455	1.0000	+0.4545
Precision@3	0.5000	0.7652	+0.2652
Precision@5	0.4909	0.5591	+0.0682
Precision@10	0.2886	0.2886	+0.0000
Recall@1	0.2114	0.4924	+0.2811
Recall@3	0.5197	0.8788	+0.3591
Recall@5	0.8705	0.9841	+0.1136
Recall@10	1.0000	1.0000	+0.0000
NDCG@1	0.7399	0.9966	+0.2567

Metric	TF-IDF	MiniLM	Gain
NDCG@3	0.7858	0.9967	+0.2109
NDCG@5	0.8498	0.9981	+0.1483
NDCG@10	0.8964	0.9988	+0.1024

Tableau 5.3: Ranking metrics: TF-IDF baseline vs fine-tuned MiniLM, learner’s perspective (n=44)

The fine-tuned model outperforms the TF-IDF baseline across nearly every cutoff reported in Table 5.3: NDCG@5 of 0.9981 versus 0.8498, Precision@3 of 0.7652 versus 0.5000. The two methods converge only at $K = 10$, a mathematical consequence of K approaching the total number of candidate jobs evaluated per learner (at most 15) rather than a weakness of either method.

Section C (Cross-validation stability) : A 5-fold cross-validation confirms this advantage is stable rather than a favorable single split: MiniLM achieves $\text{NDCG@5} = 0.9999 \pm 0.0002$ across folds, against TF-IDF’s 0.9344 ± 0.0353 .

```

=====
SECTION C – 5-FOLD CROSS-VALIDATION (Stability Check)
=====
NOTE: Model is NOT retrained. Folds verify score stability.

Fold      TF-IDF NDCG@5  MiniLM NDCG@5  TF-IDF P@1  MiniLM P@1
-----
Fold 1      0.9553          1.0000        0.9048        1.0000
Fold 2      0.9610          1.0000        0.8333        1.0000
Fold 3      0.9429          0.9994        0.8500        1.0000
Fold 4      0.9480          1.0000        0.8462        1.0000
Fold 5      0.8649          1.0000        0.7000        1.0000
-----
TF-IDF      NDCG@5: 0.9344 ± 0.0353 | P@1: 0.8268 ± 0.0680 | MAE: 23.19 ± 0.90
MiniLM      NDCG@5: 0.9999 ± 0.0002 | P@1: 1.0000 ± 0.0000 | MAE: 1.72 ± 0.26

✅ STABLE – MiniLM NDCG@5 std=0.0002 < 0.05 → no lucky split

FOR THE REPORT:
'A 5-fold cross-validation on the evaluation set confirms model stability.
MiniLM achieves NDCG@5 = 0.9999 ± 0.0002,
vs TF-IDF baseline 0.9344 ± 0.0353.'
```

Figure 5.11: Console output: 5-fold cross-validation, TF-IDF vs MiniLM

Section D (End-to-end demonstration) : For a fixed “Senior Data Scientist, 6 years, FinTech” profile evaluated against eight candidate postings, the model correctly ranked genuinely aligned opportunities above a deliberately constructed distractor posting listing every required keyword but explicitly targeting entry-level candidates — confirming the model weighs seniority alignment

alongside keyword overlap rather than being misled by surface term matching alone.

```

=====
Rank #1 - Senior Data Scientist @ FinCorp (FinTech)
Score   : 96.9/100
Verdict : ● Strong match → Recommend applying
Skills  : 3/3 required skills matched
         Matched : sql, python, machine learning
         Missing : none
Seniority: ✔ CV-Senior | Job requires=Senior
Job title: Senior Data Scientist

Rank #2 - Lead Data Scientist @ BankSecure (Banking)
Score   : 87.1/100
Verdict : ● Strong match → Recommend applying
Skills  : 4/4 required skills matched
         Matched : deep learning, sql, python, machine learning
         Missing : none
Seniority: ✔ CV-Senior | Job requires=Senior
Job title: Lead Data Scientist

Rank #3 - Data Scientist @ MedAI (Healthcare)
Score   : 81.2/100
Verdict : ● Strong match → Recommend applying
Skills  : 3/3 required skills matched
         Matched : sql, python, machine learning
         Missing : none
Seniority: ✔ CV-Senior | Job requires=Senior
Job title: Data Scientist

Rank #4 - Data Scientist @ QuickStart (FinTech, entry-level trap)
Score   : 72.4/100
Verdict : ● Good match → Worth considering
Skills  : 4/4 required skills matched
         Matched : deep learning, sql, python, machine learning
         Missing : none
Seniority: ⚠ CV-Senior | Job requires=Junior
Job title: Data Scientist

Rank #5 - ML Engineer @ TechStack (Tech)
Score   : 68.3/100
Verdict : ● Good match → Worth considering
Skills  : 3/3 required skills matched
         Matched : deep learning, python, machine learning
         Missing : none
Seniority: ✔ CV-Senior | Job requires=Senior
Job title: ML Engineer

Rank #6 - Chef @ TasteHouse (Restaurant)
Score   : 19.7/100
Verdict : ● Weak match → Not recommended
Skills  : 0/2 required skills matched
         Matched : none
         Missing : menu design, cooking
Seniority: ✔ CV-Senior | Job requires=Senior
Job title: Chef

Rank #7 - Junior Data Analyst @ RetailPlus (Retail)
Score   : 9.7/100
Verdict : ● Weak match → Not recommended
Skills  : 1/2 required skills matched
         Matched : sql
         Missing : excel
Seniority: ⚠ CV-Senior | Job requires=Junior
Job title: Data Analyst

Rank #8 - HR Coordinator @ PeopleFirst (Retail)
Score   : 0.1/100
Verdict : ● Weak match → Not recommended
Skills  : 0/2 required skills matched
         Matched : none

```

Figure 5.12: Console output: ranked job postings for a fixed CV

Honest methodological caveat. The evaluation sample used in this subsection was drawn directly from the full dataset rather than from the 22,500-pair split held out during training (described

in Section 5.2.2 above). Some evaluated pairs may therefore have been encountered during the model's 10 training epochs. This does not affect the regression metrics reported earlier in this section, which were computed on a genuinely unseen test set; it does mean the near-perfect ranking scores reported here should be interpreted as reflecting a combination of genuine generalization and possible partial memorization, rather than generalization alone. Reconstructing the exact original held-out split, or evaluating on an entirely hand-authored set of CVs and job postings absent from the source dataset, are identified as directions to further strengthen this evaluation.

5.5 Deployment

The fine-tuned model, together with its tokenizer, is loaded once at backend start-up and served continuously as a singleton in memory, avoiding the latency cost of reloading weights on every request. At an average inference time of approximately 26–260 ms per pair depending on the host machine's CPU, it is invoked by the job-matching pipeline (Chapter 4) each time a learner's job search runs, where it produces the AI-match component of the combined compatibility score for each scraped job listing. It is not invoked by the ATS Agent, which relies instead on its own dimension-by-dimension weighted scorer, as established in Chapter 4. Because the fine-tuned encoder is English-only (see Architecture above), scoring a French CV against an English-language job posting, or the reverse, is the scenario most exposed to the linguistic gap disclosed in the Linguistic robustness test earlier in this chapter. To reduce this exposure at the pipeline level rather than through retraining, the candidate's structured CV is translated to English once, upfront, before a scan's scraping loop begins — a single language detection call, followed by translation only if the CV is not already in English — so that the model consistently receives input in the language it was trained on. Symmetrically, the job-matching pipeline (Chapter 4) translates a listing's narrative fields to English at scoring time, but only for listings that are both already past the cosine-similarity filter and detected as French, keeping the added translation cost proportional to the relevant subset of listings rather than the full scraped volume. Should the initial loading attempt fail (for example, a transient file-read error), the singleton is no longer left permanently unavailable. At the start of each scan, the pipeline actively retries loading the model up to three times, thirty seconds apart, streaming a concise **"Please wait..."** event to the learner's interface for each retry in progress, and a final **"Please try again in a few minutes."** event if all three attempts fail. If every attempt fails, scoring proceeds in a degraded mode already built into the existing pipeline: each affected listing's AI-match score is left unset (rendered as "--" rather than a percentage in the learner-facing view), while the cosine-similarity filter, the skills-gap analysis, and the resulting combined score — none of which depend on the fine-tuned model — continue to function

normally, so the learner still receives a usable, if less precise, set of ranked results instead of no results at all.

5.6 Monitoring and Maintenance

While the model’s loading is now resilient to transient in-process failures, as described in the Deployment section above, this resilience remains confined to a single backend instance and to the retry attempts made at the start of a single scan: the model still shares its process with every other backend feature, so a non-recoverable crash of that process would interrupt the entire application, not just job matching, and a health status specific to the matching model, queryable independently of the general server health check, is not yet exposed. Isolating the model into a dedicated scoring microservice, exposing its health independently, and eventually distributing it across multiple instances behind a load balancer, remain unaddressed and are carried forward as future work below, rather than already resolved by the retry mechanism described above.

Consistent with the project-wide assessment in Chapter 3, this phase remains the least mature for this artifact specifically. No mechanism currently tracks whether the distribution of live scraped job postings or learner CVs drifts away from the characteristics of the synthetic training data over time, and no scheduled retraining pipeline exists to refresh the model as the job market or the platform’s user base evolves.

Conclusion

This chapter has examined the fine-tuned CV-to-job matching model through the full CRISP-ML(Q) cycle applied specifically to this artifact: the business rationale for fine-tuning over a generic embedding model, the construction of a graded training signal from a synthetic dataset, the bi-encoder architecture and its training configuration, a multi-faceted evaluation combining regression metrics, a targeted qualitative benchmark, and a ranking comparison against a non-learned baseline, its integration into the Job Agent’s production pipeline, and an honest account of the monitoring this artifact still lacks. The following chapter shifts from the system’s design and its central model to their concrete implementation across the project’s five development sprints.

IMPLEMENTATION

Plan

Introduction	65
1 CV Agent Pipeline Design	65
2 Job Agent Pipeline Design	68
3 ATS Agent Pipeline Design	72
4 The ATS Scoring Formula	75
5 Data Model	76
6 Prompt Engineering and Anti-Hallucination Safeguards	77
Conclusion	79

Introduction

While Chapters 4 and 5 detailed the internal design of the system’s components, this chapter turns to how that design was concretely built: the technical environment the project was developed in, the chronological progression of its development iterations, the reliability practices adopted to make the resulting system trustworthy, and a walkthrough of the learner-facing journey the implemented system now supports.

6.1 Technical Environment

6.1.1 Development Tools

The backend was developed in Python within Visual Studio Code, using FastAPI’s auto-reload server for local iteration and Git for version control, hosted on GitHub. The frontend was developed in TypeScript with Next.js and React, sharing the same repository as the backend to keep API contract changes and their corresponding frontend adjustments in a single commit history.

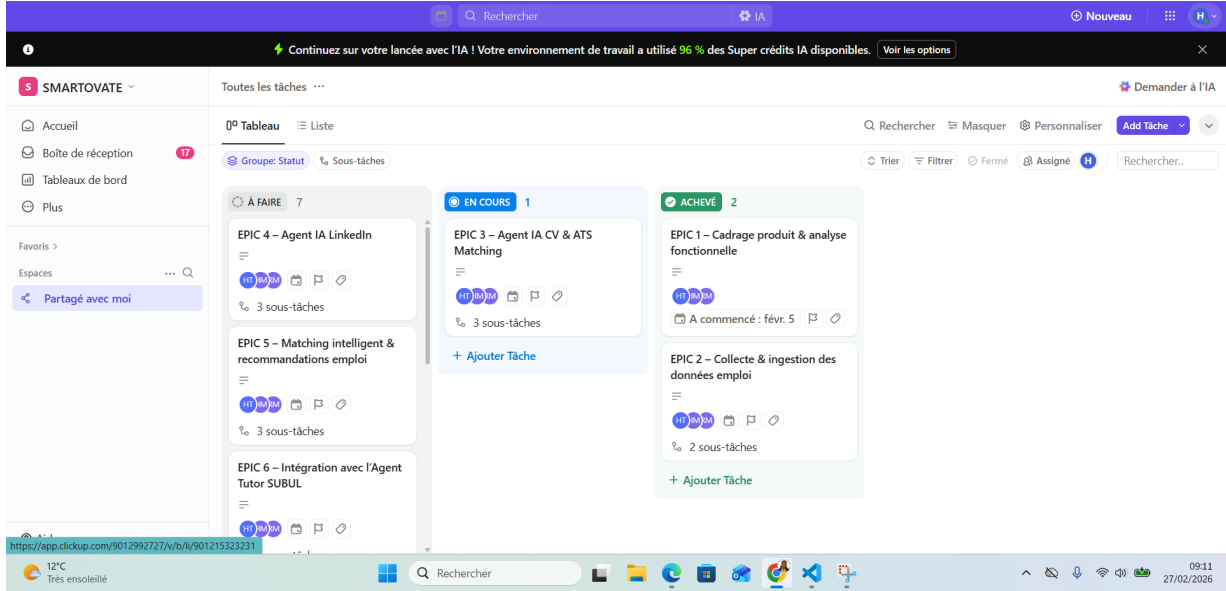
6.1.2 Execution Environment and Cloud AI/ML Services

The backend was developed and executed locally within Visual Studio Code, using FastAPI’s development server for testing and iteration. Azure Machine Learning notebooks were used separately for the training and evaluation of the fine-tuned matching model, while Azure OpenAI and cloud-based services supported the AI components of the platform.¹

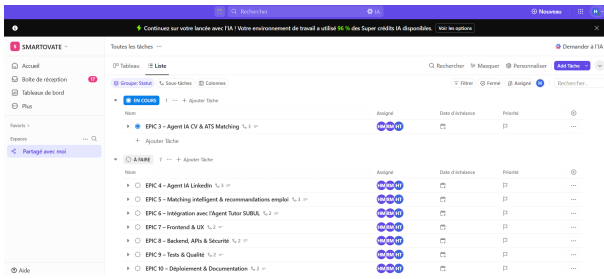
6.1.3 Project Management Tooling

ClickUp [47] was used throughout the internship as the single source of truth for the Scrum backlog: the product backlog was organized into Epics, each broken down into sub-tasks, and tracked through three statuses *À faire* (To Do), *En cours* (In Progress), and *Achevé* (Done). Each Epic was tagged with the specific chapter and component of the architecture it implemented, keeping the backlog directly traceable to the design decisions documented in Chapters 3 through 5.

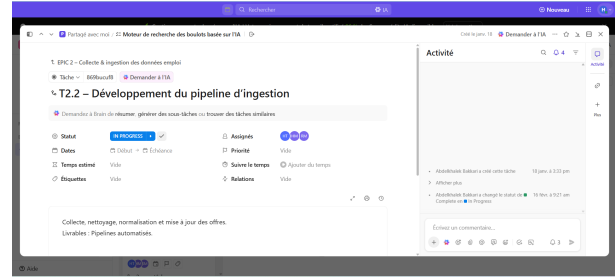
1. Azure was selected primarily because Smartovate already held an active Azure subscription, avoiding additional procurement overhead during the internship.



(a) Epics board view



(b) Epics list view



(c) Sub-task detail view

Figure 6.1: Scrum backlog organization in ClickUp: (a) Epics board view, (b) Epics list view, and (c) sub-task detail view.

6.2 Product Backlog and User Stories

Chapter 3 introduced the product backlog at the level of its four functional epics (Table 3.3), organized in ClickUp as shown in Figure 6.1. This section refines that epic-level view in two stages, kept deliberately separate so that each can be read on its own. Table 6.1 first decomposes each of the four epics into the concrete features planned for it the product backlog proper. Table 6.2 then expresses every one of those features as one Connextra-style user story [48], so that each backlog feature is traceable to exactly the story that implements it.

6.2.1 Product Backlog by Feature

Every feature in Table 6.1 was assigned a MoSCoW priority (**M**ust have, **S**hould have, **C**ould have) during Sprint Planning, jointly by the Product Owner and the developer. Seventeen of the twenty-two features fall directly under one of the four functional epics defined in Chapter 3. The remaining five features database schema provisioning, backend/frontend scaffolding, job-specific CV

enhancement, degraded-mode resilience, and end-to-end testing are technical or cross-cutting capabilities that support multiple parts of the platform² rather than belonging to a single functional agent. They are therefore grouped under *Foundations (cross-cutting)* rather than forced into one of the four functional epics, following the same principle used in Chapter 3 for treating the Supervisor Agent as a cross-cutting concern rather than as a fifth functional epic.

ID	Epic / Fonctionnalité	Description	Pr.
PB-01	CV Agent	CV upload landing screen entry point of the onboarding journey.	M
PB-02	CV Agent	Resume upload and extraction from PDF or DOCX files.	M
PB-03	CV Agent	Automatic professional-domain detection.	M
PB-04	CV Agent	Structural baseline ATS score, computed internally before any AI rewriting and persisted to Cosmos DB for traceability not surfaced on screen to the learner.	M
PB-05	CV Agent	Enrichment prompts for missing sections (languages, education, professional experience).	S
PB-06	CV Agent	AI-based CV rewriting and ATS optimization, guarded against invented degrees, jobs, or skills.	M
PB-07	CV Agent	Automatic persistence of the enriched, canonical CV profile.	M
PB-08	Job Agent	Multi-source job scraping across job boards matching the learner's profile.	M
PB-09	Job Agent	Cosine title-similarity pre-filtering of scraped listings.	S

2. This grouping follows common Scrum practice of using a technical "enabler" epic for capabilities that do not map to a single user-facing feature [16].

ID	Epic / Fonctionnalité	Description	Pr.
PB-10	Job Agent	AI-computed compatibility score and explainable match analysis for every matched job, including score interpretation, matched/missing skills, and skill-coverage percentage.	M
PB-11	Job Agent	Skill gap analysis against matched listings.	M
PB-12	Job Agent	Job market overview (top skills, companies, and locations).	S
PB-13	Job Agent	Personalized, phased learning roadmap built from the skill gap.	S
PB-14	Job Agent	Consolidated PDF career report generation.	C
PB-15	ATS Agent	Job-specific, dimension-by-dimension ATS compatibility scoring.	M
PB-16	Foundations (cross-cutting)	LLM-based job-specific CV enhancement endpoint that adapts the learner's resume to a selected job listing by extracting relevant keywords and requirements from the target description.	S
PB-17	Conversational Assistant	Contextual conversational assistant over the learner's matches, skill gap, and general IT topics.	S
PB-18	Conversational Assistant	Bilingual (French/English) assistant replies.	C
PB-19	Foundations (cross-cutting)	Cosmos DB and PostgreSQL schema provisioning.	M
PB-20	Foundations (cross-cutting)	FastAPI backend and Next.js frontend scaffolding in a shared repository.	M

ID	Epic / Fonctionnalité	Description	Pr.
PB-21	Foundations (cross-cutting)	Degraded, cosine-only scoring mode when the fine-tuned matching model fails to load.	M
PB-22	Foundations (cross-cutting)	Structured manual end-to-end test passes exercising the four-agent pipeline (Supervisor, CV, Job, and ATS Agents) together, with full test automation identified as future work.	M

Tableau 6.1: Product backlog: features by epic, with MoSCoW priority (M = Must Have, S = Should Have, C = Could Have)

6.2.2 User Stories

Table 6.2 restates each feature above as a single user story, following the standard Connextra template “As a ⟨role⟩, I want ⟨capability⟩, so that ⟨benefit⟩” and grouped by the sprint in which it was delivered. Each story’s MoSCoW priority matches that of the feature it implements in Table 6.1; note, however, that the two tables are ordered differently by epic in Table 6.1 and by delivery sprint here so a story’s numeric ID does not coincide with that of the feature it implements. Each “Epic associé” repeats the epic the corresponding feature was decomposed from. The four Foundations stories under Sprint 1 and Sprint 5 are, accordingly, phrased from the development team’s perspective rather than the learner’s, since they capture technical enablers and the end-to-end testing effort itself rather than a learner-facing capability.

ID	Epic associé	User Story	Pr.
Sprint 1			
US-01	Foundations (cross-cutting)	As the development team, I want the Cosmos DB and PostgreSQL schemas provisioned before any feature work begins, so that user profiles and job records can be persisted reliably from the first sprint onward.	M

ID	Epic associé	User Story	Pr.
US-02	Foundations (cross-cutting)	As the development team, I want the FastAPI backend and Next.js frontend scaffolded within a single shared repository, so that API contract changes and their corresponding frontend adjustments stay in one commit history.	M
US-03	CV Agent	As a learner, I want to land on a CV upload screen as soon as I open the platform, so that I can begin my onboarding journey without any additional setup step.	M
Sprint 2			
US-04	CV Agent	As a learner, I want to upload my CV as a PDF or DOCX file, so that the platform can read and analyze my profile without me re-entering my information manually.	M
US-05	CV Agent	As a learner, I want the system to automatically detect my professional domain, so that the scoring and recommendations I receive are relevant to my actual field.	M
US-06	CV Agent	As the development team, I want a baseline ATS score computed for each CV before any AI rewriting is applied, so that the quality gain produced by the improvement step can be measured and traced, even though this score is not surfaced to the learner in the confirmation screen.	M
US-07	CV Agent	As a learner, I want to be prompted for any missing section (languages, education, professional experience), so that my profile is complete before the AI improvement step runs.	S

ID	Epic associé	User Story	Pr.
US-08	CV Agent	As a learner, I want my CV rewritten and ATS-optimized by AI without any invented information, so that I can trust the improved version still reflects my real background.	M
US-09	CV Agent	As a learner, I want my enriched profile saved automatically to my account, so that I do not have to re-upload my CV for every subsequent action on the platform.	M
Sprint 3			
US-10	Job Agent	As a learner, I want the platform to scrape job offers from multiple job boards matching my profile, so that I discover more relevant opportunities than I could find by searching manually.	M
US-11	Job Agent	As a learner, I want each scraped job pre-filtered by title similarity to my profile, so that irrelevant postings do not waste processing time or clutter my results.	S
US-12	Job Agent	As a learner, I want an AI-computed compatibility score with an explainable breakdown for every matched job, including score interpretation, matched/missing skills, and skill-coverage percentage, so that I can prioritize which offers to apply to first and understand why each job matches my profile.	M
US-13	Job Agent	As a learner, I want to see exactly which skills I am missing for a given job, so that I know precisely what to learn to become a stronger candidate for it.	M

ID	Epic associé	User Story	Pr.
US-14	Job Agent	As a learner, I want a market overview of all the listings scraped for me (top skills, companies, and locations), so that I understand broader industry trends beyond my own individual matches.	S
US-15	Job Agent	As a learner, I want a personalized, phased learning roadmap built from my skill gap, so that I know what to learn first and in what logical order.	S
US-16	Job Agent	As a learner, I want to download a single consolidated PDF report of my profile, matches, skill gap, and roadmap, so that I can keep and share my career analysis outside the platform.	C
Sprint 4			
US-17	ATS Agent	As a learner, I want a detailed, dimension-by-dimension ATS score against one specific job description, so that I understand precisely why I do or do not match that particular role.	M
US-18	Foundations (cross-cutting)	As a learner, I want to generate a version of my CV tailored specifically to a selected job listing using AI-based enhancement, so that I can apply with a resume adapted to the requirements of that specific opportunity.	S
US-19	Conversational Assistant	As a learner, I want to ask a conversational assistant open-ended questions about my matches, skill gap, or general IT topics, so that I get guidance without navigating between multiple screens.	S

ID	Epic associé	User Story	Pr.
US-20	Conversational Assistant	As a learner, I want the conversational assistant to reply in whichever language I write to it, so that I can interact with it naturally.	C
Sprint 5			
US-21	Foundations (cross-cutting)	As a learner, I want the platform to keep working in a degraded, cosine-only scoring mode if the fine-tuned matching model temporarily fails to load, so that a transient technical issue never blocks my job search entirely.	M
US-22	Foundations (cross-cutting)	As the development team, I want to manually exercise the four-agent pipeline (Supervisor, CV, Job, and ATS Agents) together through structured end-to-end walkthroughs, so that integration issues are caught before an increment is demonstrated to the Product Owner, with full test automation left as a future improvement.	M

Tableau 6.2: Product backlog: user stories by sprint and associated epic, with MoSCoW priority (M = Must Have, S = Should Have, C = Could Have)

Across the five sprints, the product backlog contains twenty-two user stories, fourteen Must, six Should, and two Could a distribution that is deliberately front-loaded towards Must-priority stories in the earlier sprints, so that the core, non-negotiable pipeline (upload, scoring, matching) was always demonstrable at each sprint review even if a Should- or Could-priority refinement had to slip to a later sprint. Each story was additionally estimated in Fibonacci story points during Sprint Planning, jointly by the Product Owner and the developer, for sprint-capacity planning purposes.

6.3 Chronological Development Iterations

Development was carried out under Scrum[16], across five sprints, each closing with a working, demonstrable increment reviewed with the project’s supervisor at Smartovate.

Sprint	Focus	Main Deliverables
1	Foundations & Architecture	Requirements consolidation (Chapter 2), architecture and stack validation (Chapter 3), Cosmos DB / PostgreSQL schema setup, FastAPI and Next.js scaffolding.
2	CV Agent	The eight-node CV Agent pipeline (Chapter 4): extraction, domain detection, deterministic ATS scoring, LLM-based improvement with its anti-hallucination guard, and persistence.
3	Job Agent & Fine-Tuned Matching Model	Multi-source scraping across fourteen job boards, cosine pre-filtering, and the fine-tuned bi-encoder model’s data preparation, training, and evaluation on Azure Machine Learning (Chapter 5), wired into the Job Agent’s scoring node; the skill-gap analysis, roadmap generation, and report generation nodes completing the Job Agent’s pipeline (Chapter 4); and the resulting Matches, Market, Skill Gap, Roadmap, and Career Report views of the learner-facing dashboard.

Sprint	Focus	Main Deliverables
4	ATS Agent & Conversational Assistant	The six-node, job-specific ATS Agent (Chapter 4); the separate job-specific CV enhancement endpoint (Chapter 4, Section 4.3.2), which re-extracts keywords from the target job description independently of the ATS Agent's breakdown; and the conversational chat assistant, capable of answering open-ended questions about the learner's matched job listings, skill gap, market analysis, learning roadmap, own profile, and general IT-related topics.
5	Integration, Resilience & Refinement	Model-loading retry and graceful degradation (Chapter 5), bilingual French/English handling, comprehensive model evaluation, frontend refinement, and structured manual end-to-end testing.

Tableau 6.3: The five development sprints and their main deliverables

Each sprint closed with a sprint review demonstrating the working increment to the Product Owner, followed by a short retrospective whose feedback was incorporated directly into the planning of the next sprint, in accordance with the Scrum ceremonies summarized in Table 6.4.

Role / Ceremony	Applied on this project
Product Owner	M. Abdelkhalek is the CEO of Smartovate, responsible for validating the increment delivered at the end of each sprint against the platform's evolving business requirements and product vision.
Technical Advisor	Ms. Insaf, who provided technical guidance throughout the project, advised AI-related design decisions, reviewed experimental results, and helped identify and correct issues whenever model behaviour or evaluation results required refinement.
Development Team	A single developer, me, responsible for the system architecture, implementation, model development, testing, and integration across all five sprints.
Sprint Planning / Review / Retrospective	Sprint Planning was conducted at the beginning of each sprint, Sprint Reviews were held at the end of each sprint to demonstrate the working increment to the Product Owner, and Sprint Retrospectives were carried out immediately afterwards to identify improvements that were applied to the following sprint.

Tableau 6.4: Scrum roles and ceremonies as applied on this project

Implementation Roadmap and Gantt Chart

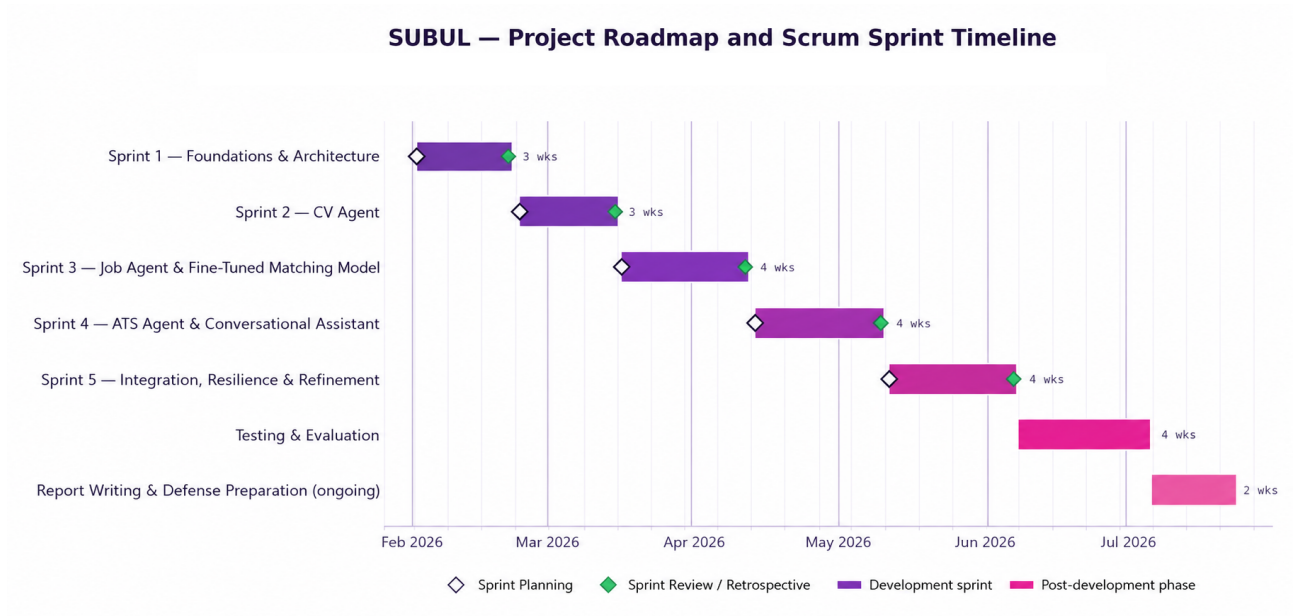


Figure 6.2: Project roadmap and sprint-aligned Gantt chart (February July 2026)

Figure 6.2 presents the chronological implementation roadmap of the SUBUL platform.

The project was organized into five Scrum sprints, each ending with a functional increment demonstrated and reviewed with the project stakeholders. Sprint 1 focused on establishing the technical foundations and validating the system architecture, including the backend/frontend scaffolding and database setup. Sprint 2 implemented the CV Agent pipeline, covering resume extraction, analysis, improvement, and profile persistence. Sprint 3 focused on the development of the Job Agent and the integration of the fine-tuned matching model, including job retrieval, semantic matching, and career-oriented analysis. Sprint 4 introduced the ATS Agent and the conversational assistant capabilities. Sprint 5 concentrated on system integration, resilience improvements, refinement, and final validation activities. Sprint Planning, Sprint Review, and Retrospective milestones are represented to highlight the iterative Scrum execution cycle throughout the development process. After the development sprints, dedicated phases were allocated to system testing and evaluation, followed by ongoing report writing and defense preparation until the end of the project.

6.4 Reliability Best Practices

Beyond feature delivery, three cross-cutting reliability practices were deliberately built into the system rather than left to chance, each already detailed in its originating chapter and summarized here as a coherent set of engineering choices.

6.4.1 Traceability

Every backlog item in ClickUp was tagged with the specific chapter and section of this report it implements, so that any design decision documented in Chapters 3 through 5 can be traced back to the sprint and task that implemented it. At the system level, both the CV Agent’s and the Job Agent’s LangGraph-based pipelines log a structured message at the completion of each node (Chapter 4), while only the CV Agent’s final baseline and post-improvement ATS scores not the intermediate breakdown or retry decisions are persisted to Cosmos DB alongside the canonical profile. Together, these mechanisms give both the developer and, in principle, a future monitoring process a step-by-step account of how a given result was produced.

6.4.2 Retry Mechanisms

The fine-tuned matching model’s loading is not left to fail permanently on a transient error: the pipeline actively retries loading it up to three times, thirty seconds apart, before falling back to a degraded scoring mode rather than blocking the learner’s search entirely, as detailed in Chapter 5’s Deployment section.

6.4.3 Guardrails

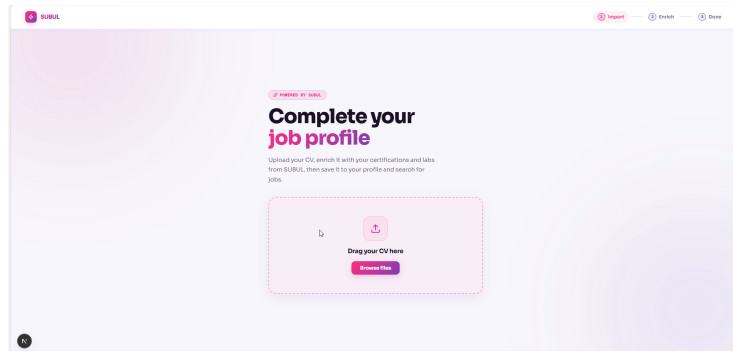
Two deterministic guardrails constrain the system’s generative components, preventing a language model’s non-determinism from compromising a hard requirement. First, the CV Agent’s anti-hallucination guard programmatically strips any generated EDUCATION or PROFESSIONAL EXPERIENCE section unsupported by lexical signals in the source resume (Chapter 4, Section 4.6). Second, the ATS Agent’s and CV Agent’s scoring formulas are entirely rule-based rather than model-generated, guaranteeing that a score presented to the learner as objective progress remains fully reproducible (Chapter 4, Section 4.4).

6.5 Learner Journey Walkthrough

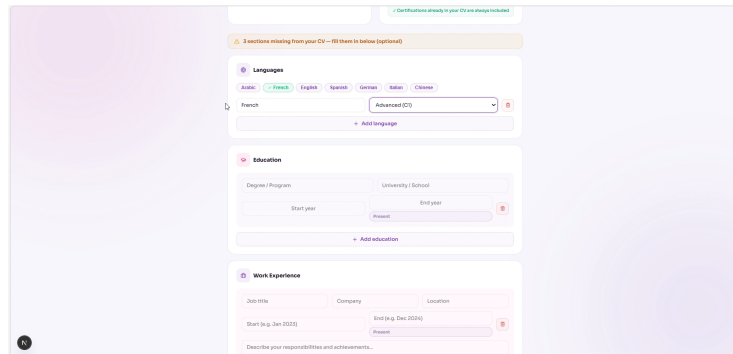
This section illustrates the system built across the five sprints above through the concrete sequence of screens a learner interacts with, from onboarding to receiving actionable career guidance.

CV upload and enrichment: The learner uploads a resume; the CV Agent processes it and prompts for any missing sections it identifies (languages, education, or professional experience). Any information the learner provides at this stage is merged into the resume text before the LLM-based improvement step (Chapter 4), rather than being merely collected and left unused. The interface then confirms the resulting canonical profile has been saved, together with actions to search for jobs

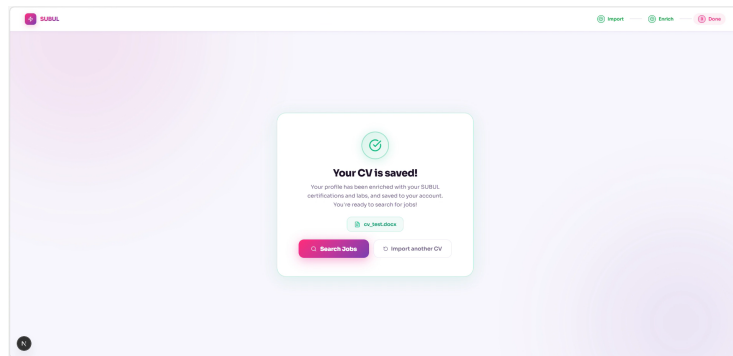
or upload another resume. The resulting baseline and post-improvement ATS scores (Chapter 4) are persisted to Cosmos DB alongside this profile but are not surfaced in this confirmation screen.



(a)



(b)



(c)

Figure 6.3: CV upload and enrichment journey: (a) upload screen, (b) enrichment screen for missing sections, and (c) confirmation screen once the profile is saved.

Job matches: The learner consults the ranked list of opportunities produced by the Job Agent, each listing offering a direct link to apply and an expandable explanation panel detailing the cosine title-similarity score, the AI-match score, and the specific matched and missing skills behind the reported skill coverage (Chapter 4).

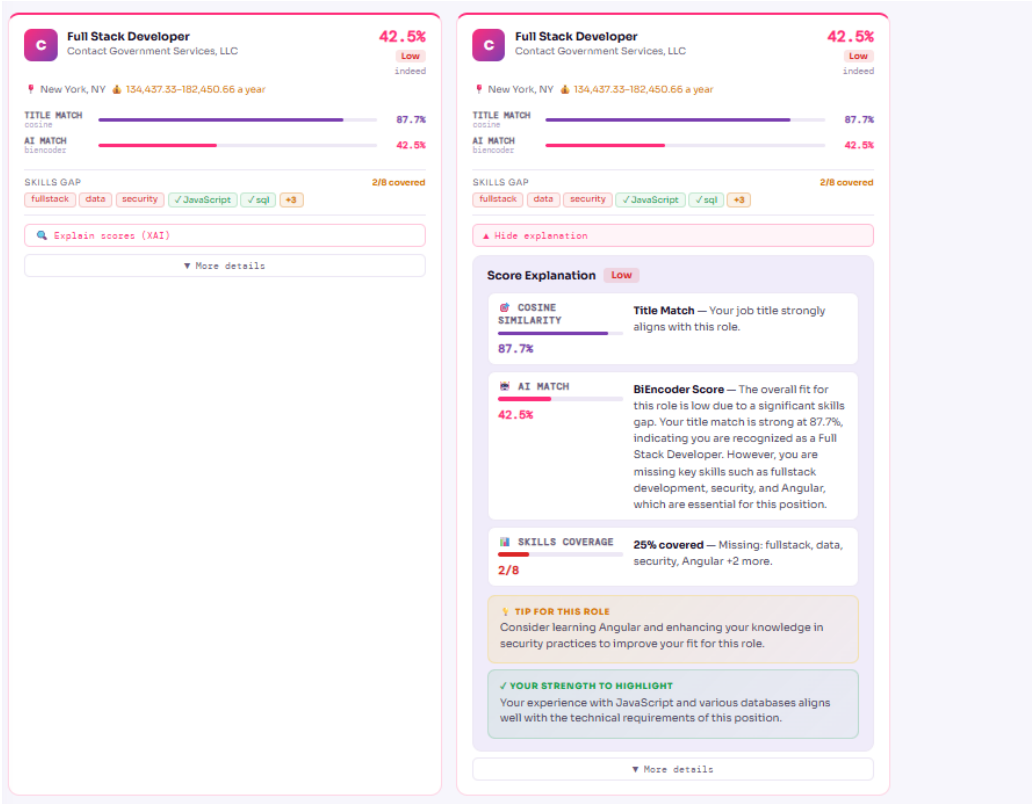


Figure 6.4: The Matches view, listing job postings ranked by score, with an expanded explanation panel showing the score breakdown and skill coverage for one listing

Skill gap analysis: The learner reviews the missing competencies identified across matched listings.

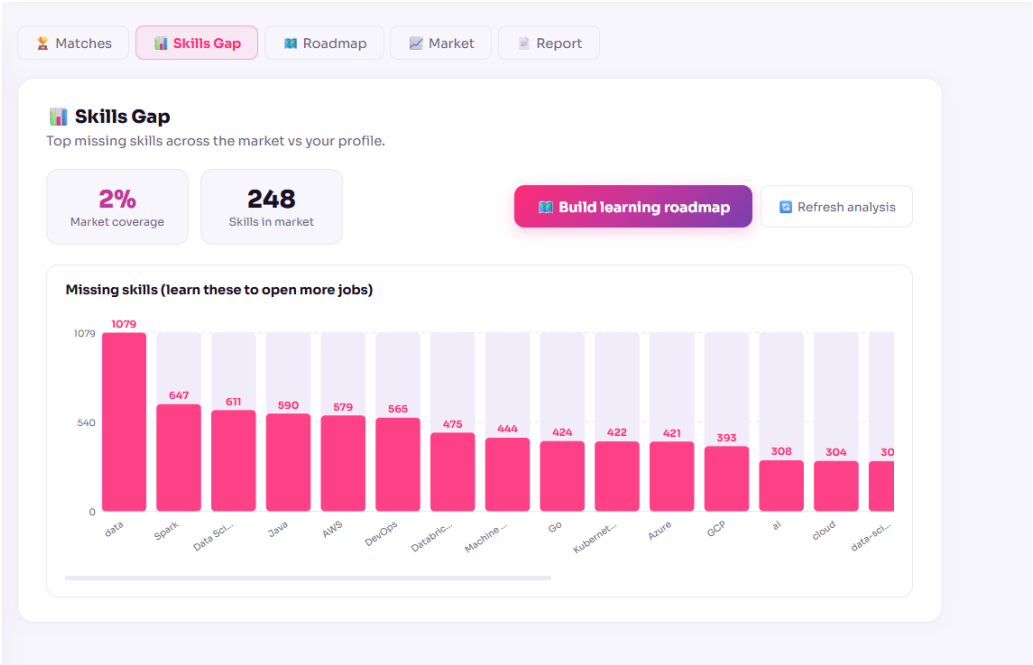


Figure 6.5: Skill gap analysis

Learning roadmap: Building on the skill gap identified above, the Roadmap view organizes the missing competencies into a phased, prerequisite-ordered learning plan , opening with a personalized

message contextualizing the recommended order for the learner’s specific profile. Each phase groups one or more skills under a proficiency level (Beginner, Intermediate, or Advanced) and an estimated duration, together with a short pedagogical description, concrete small-project ideas to practice the skill, and an expandable justification for why it was placed at that point in the sequence. The learner can also request the roadmap to be regenerated at any time.

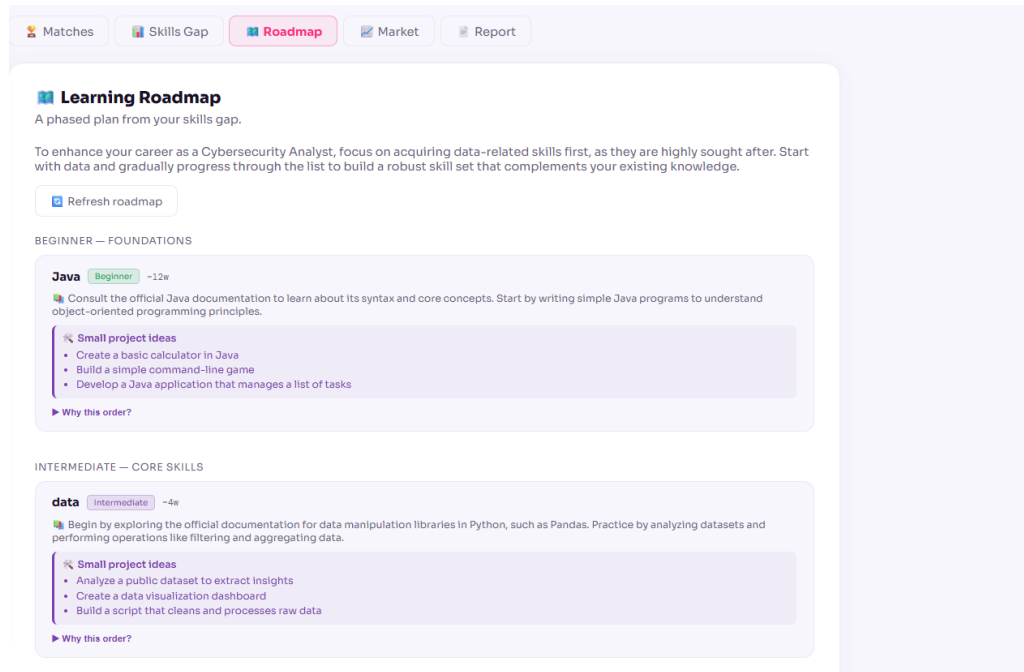


Figure 6.6: Learning roadmap: phased plan with per-skill duration, project ideas, and sequencing rationale

Market overview: The Market view aggregates statistics across the entire corpus of listings scraped for the learner. The first screen reports headline indicators total jobs analyzed, average AI-match score, and a breakdown by match quality alongside the most frequently demanded skills across all matched listings.

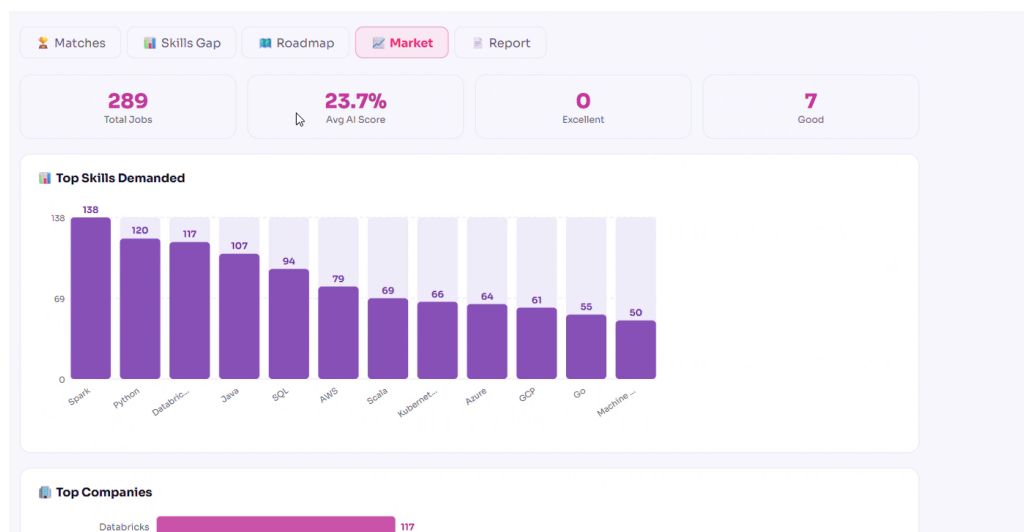


Figure 6.7: Market overview: key indicators and top skills demanded

A second screen reports the companies posting the most listings and the geographic locations where they concentrate, giving the learner a sense of where relevant opportunities are coming from.

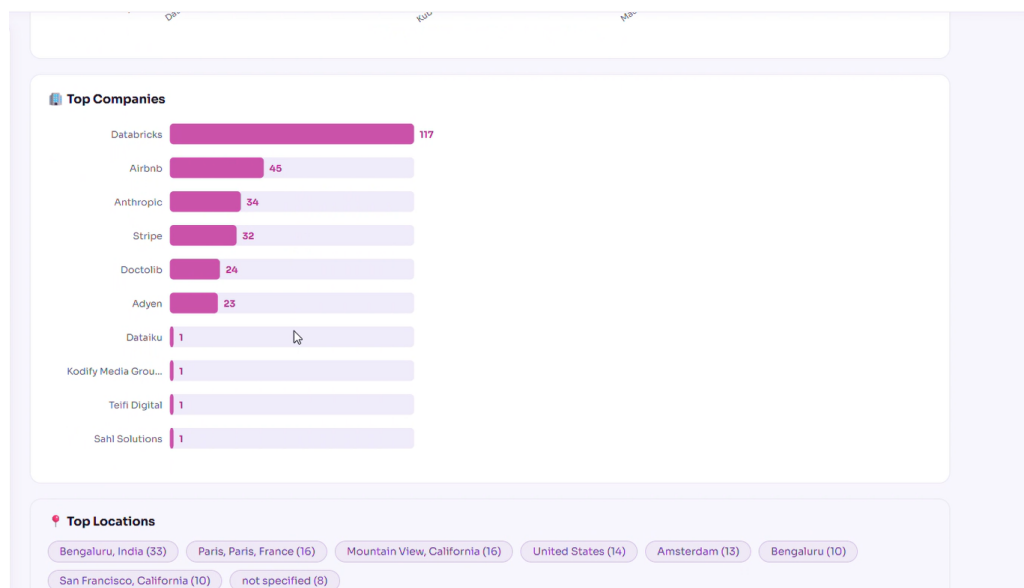


Figure 6.8: Market overview: top companies and top locations

Consolidated career report: At any point, the learner can download a single, consolidated PDF report generated by the Job Agent’s report generation node (Chapter 4), bringing together every view described above into one document: the learner’s profile, a market overview (total jobs analyzed, sources, remote-friendly share, top skills, top companies, and top locations), the skill gap with its market coverage percentage and per-skill job impact, the phased learning roadmap with per-skill duration and a concrete learning tip, and a ranked list of the best job matches with their compatibility score and the specific skills still to learn for each.

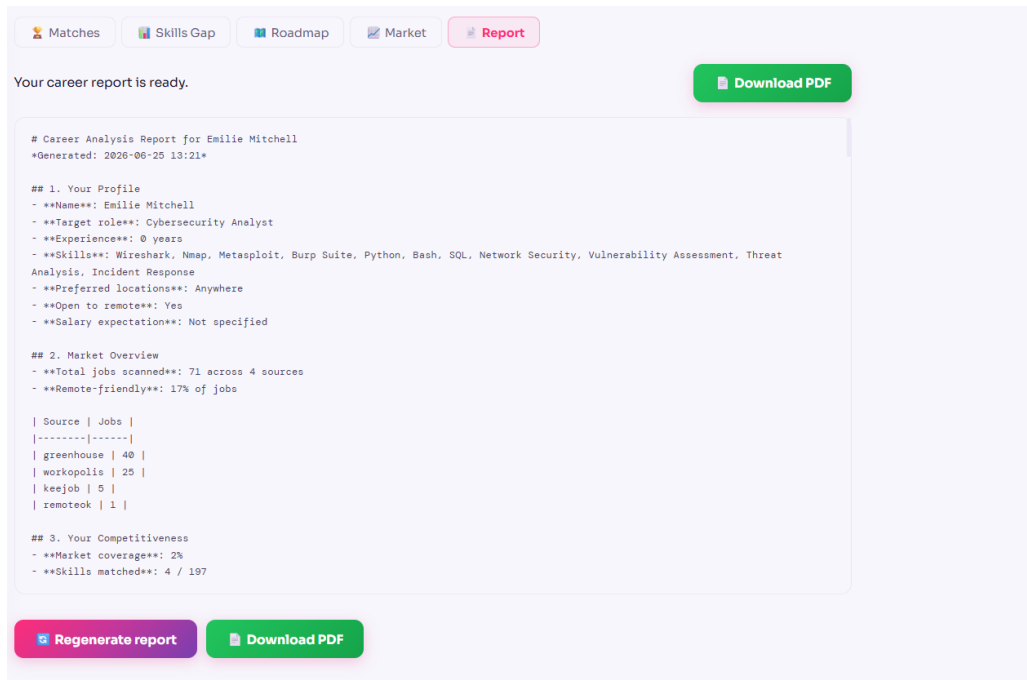


Figure 6.9: Consolidated career report (PDF)

Job-specific ATS scoring: The learner selects a specific job listing and receives the ATS Agent’s detailed, dimension-by-dimension compatibility breakdown.

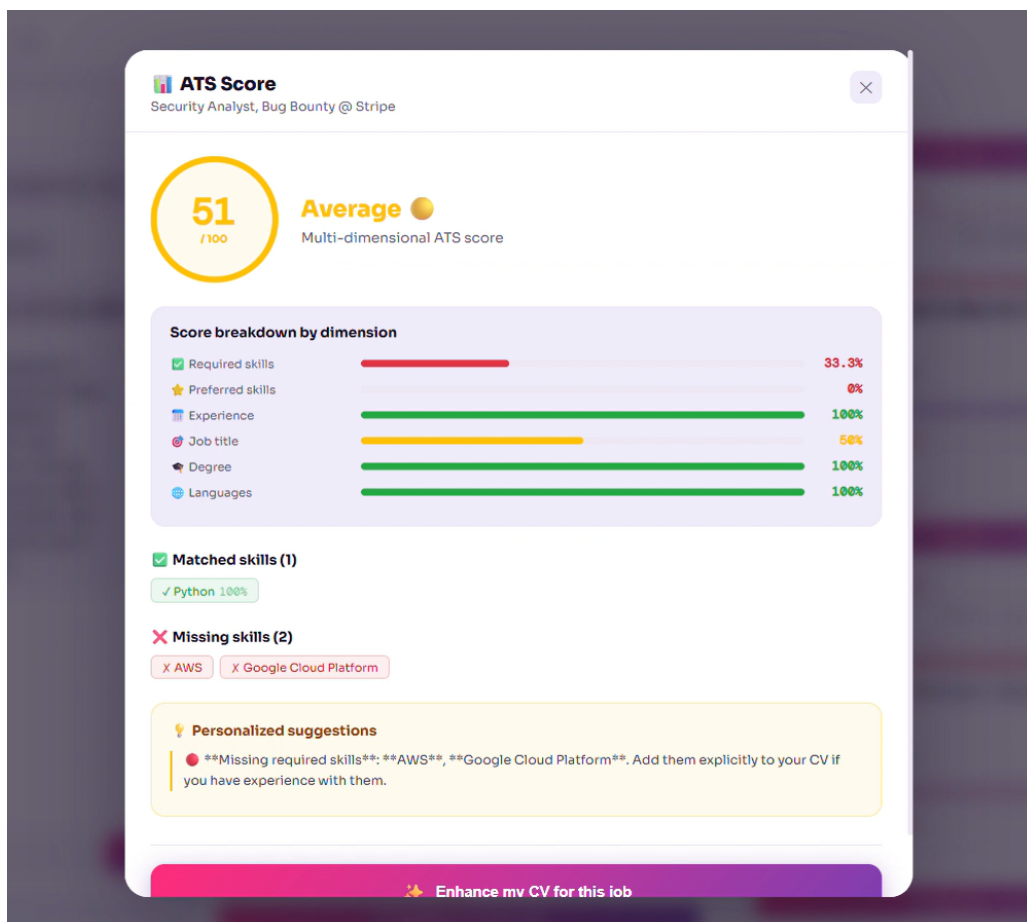
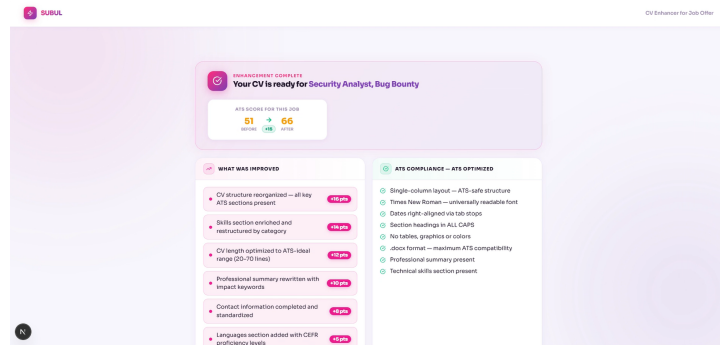
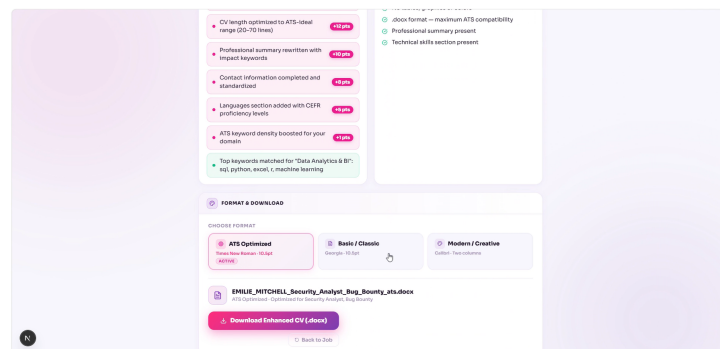


Figure 6.10: Job-specific ATS scoring screen, showing the six-dimension compatibility breakdown

Job-specific CV enhancement: From the same job listing, the learner may separately trigger the job-specific CV enhancement endpoint (Chapter 4, Section 4.3) to tailor the resume specifically to that listing.



(a) ATS score improvement and enhancement summary



(b) Format selection and CV download

Figure 6.11: Job-specific CV enhancement screen

Conversational assistant: The learner interacts with the chatbot for open-ended career guidance questions, spanning their matched job listings, the skill gap and market analyses, the learning roadmap, their own profile, and general IT-related questions.

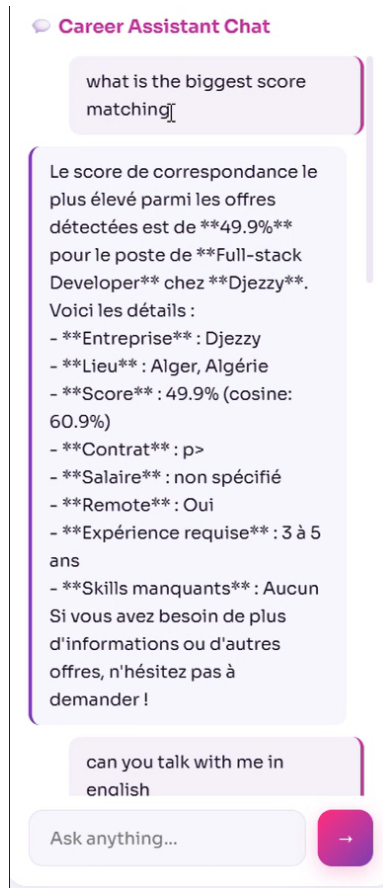


Figure 6.12: Conversational assistant interface

Conclusion

This chapter presented the transition from the system design to its concrete implementation. It described the technical environment used, as well as the project organization following Scrum through a product backlog composed of twenty-two user stories, seventeen derived from the four functional epics previously defined and five grouped under a cross-cutting Foundations category.

The planning and execution of the five Scrum sprints were then presented, with each sprint resulting in a validated functional increment. The chapter also highlighted the reliability mechanisms integrated into the system, including traceability, retry management, and guardrail mechanisms.

Finally, the complete learner journey, from CV upload to the generation of a personalized career report, was presented to illustrate the final development outcome. The following chapter is dedicated to the testing process performed on the system and the analysis of the obtained results.

TESTING, EVALUATION AND RESULTS

Plan

Introduction	81
1 Business and Data Understanding	81
2 Data Preparation	81
3 Modeling	83
4 Evaluation	84
5 Deployment	96
6 Monitoring and Maintenance	97
Conclusion	97

Introduction

The preceding chapters each reported testing and evaluation specific to the component they described: the fine-tuned matching model’s own CRISP-ML(Q) evaluation (Chapter 5), and the learner journey walked through as part of the implementation narrative (Chapter 6). This chapter steps back to present the system’s overall testing strategy, consolidates the model and ATS scoring results into a single coherent account, and closes with the limitations identified across the project, gathered here into one place rather than left scattered across the chapters that first disclosed them.

7.1 Testing Strategy

7.1.1 Approach

Given the single-developer, Scrum-based process described in Chapter 6, testing was carried out primarily through structured manual functional testing at two levels, complemented by the fine-tuned model’s own dedicated, code-driven evaluation.

Per-agent functional testing. At the close of each sprint, the increment delivered was exercised manually against realistic inputs (varied resume formats and content, a range of job titles and seniorities) to confirm it behaved as designed before being marked *Done* in ClickUp and demonstrated at the sprint review (Chapter 6).

End-to-end integration testing. Once all four agents were functionally complete (end of Sprint 4), the platform was tested as a learner would use it, starting from the entry point of the application. This structured, repeated manual walkthrough is the concrete form the end-to-end testing effort scheduled for Sprint 5 (PB-22/US-22, Chapter 6) took; automating this walkthrough into a continuously re-runnable test suite is identified as future work rather than something already in place, a distinction revisited in this chapter’s Scope and Boundaries subsection below.

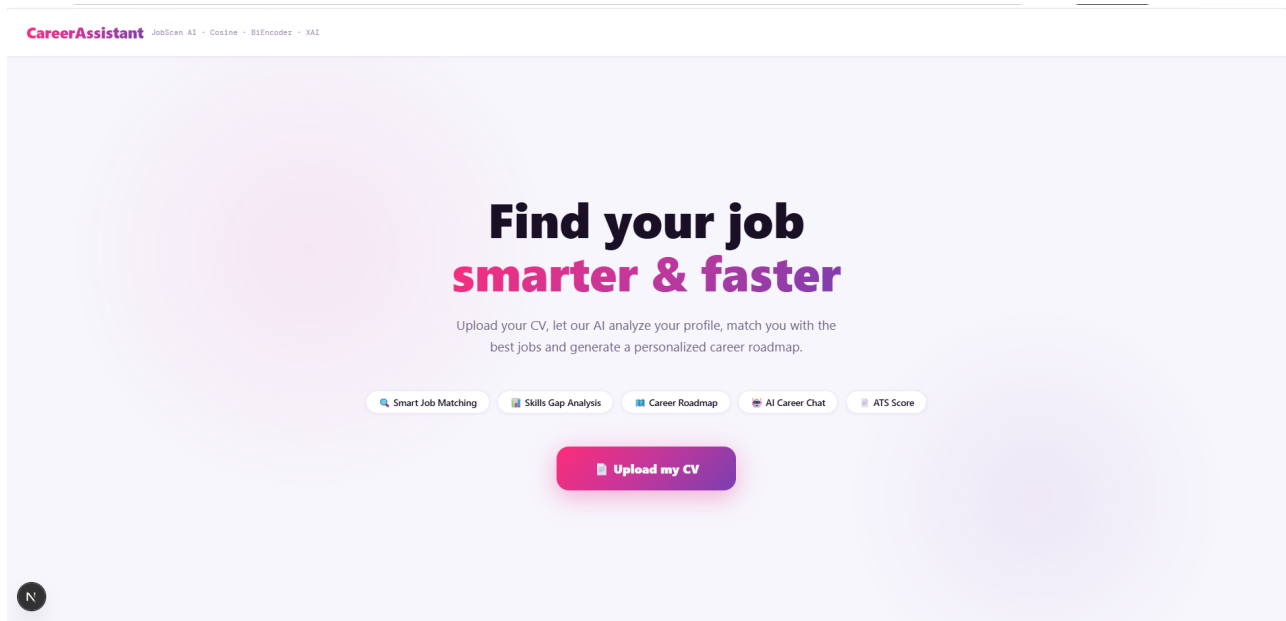


Figure 7.1: Home page of the SUBUL AI Career Assistant

From this entry point, a learner’s full journey was exercised end-to-end: uploading a resume and completing its enrichment, reviewing its baseline ATS score and LLM-based improvement, triggering a job scan, reviewing the ranked job matches, the skill gap analysis, and the phased learning roadmap, consulting the market overview, downloading the consolidated career report, requesting a job-specific ATS comparison, triggering the job-specific CV enhancement for that same listing, and interacting with the conversational assistant - the same journey illustrated in Chapter 6, here re-run specifically as a validation pass rather than a design walkthrough.

Fine-tuned model evaluation. Separately from this functional testing, the fine-tuned matching model at the core of the Job Agent was subjected to its own dedicated, quantitative evaluation regression metrics on a genuinely held-out test set, a five-test qualitative benchmark suite, and a ranking comparison against a non-learned baseline following the Evaluation phase of the CRISP-ML(Q) cycle applied to it in Chapter 5.

7.1.2 Scope and Boundaries

This testing effort did not include automated unit or integration test suites, load testing under concurrent multi-user traffic, or adversarial security testing of the API surface; these remain consistent with the monitoring immaturity already acknowledged in Chapter 5 and are revisited as limitations later in this chapter.

7.2 Consolidated Evaluation Results

7.2.1 Fine-Tuned Matching Model — Synthesis

Table 7.1 consolidates the model’s evaluation results, detailed in full in Chapter 5, into a single summary view.

Evaluation Axis	Consolidated Result
Regression accuracy	$R^2 = 0.9890$, MAE = 1.82 pts on the 22,500-pair held-out test set; validation/test gap of 0.0002, confirming no overfitting.
Qualitative benchmark	4 of 5 targeted tests passed as expected (discrimination, most of adversarial resistance, seniority coherence, linguistic robustness); an industry-related bias was identified and traced to the ground-truth formula’s binary industry factor (Section 5.4.2, Chapter 5).
Baseline comparison	47.6-point score separation and 99.9% pair-level accuracy versus a TF-IDF baseline’s 14.3 points and 90.1%.
Ranking performance	NDCG@5 = 0.9981 versus TF-IDF’s 0.8498; confirmed stable across a 5-fold cross-validation (0.9999 ± 0.0002).

Tableau 7.1: Consolidated fine-tuned matching model evaluation results (full detail in Chapter 5)

7.2.2 ATS Scoring — Before and After Improvement

Beyond the fine-tuned matching model, the CV Agent’s deterministic, eight-dimension ATS formula (Chapter 4, Section 4.4) is itself exercised twice per uploaded resume: once on the raw extracted text (baseline score), and once on the LLM-rewritten, ATS-optimized output (post-improvement score). Because this formula is entirely rule-based rather than model-driven, the same resume re-scored twice

always yields the identical baseline value, making the reported gain a reproducible measure of the enrichment step’s contribution rather than an artifact of a non-deterministic scoring process.

Across the resumes exercised during functional testing, the largest gains were consistently observed on the sections targeted by the missing-section detection node profile summary, contact completeness, and language proficiency consistent with the enrichment step’s design intent of specifically addressing the resume’s weakest dimensions (Chapter 4, Section 4.1) rather than uniformly rewriting sections already close to complete.

7.3 Identified Limitations

The limitations disclosed individually across Chapters 4 through 6 are consolidated here into a single account.

Area	Limitation
Production wiring gap	The frontend’s “Search Jobs” action currently defaults to the non-agentic scraping route rather than the fully implemented LangGraph Job Agent pipeline (Chapter 4, Section 4.2).
Model industry bias	The ground-truth formula’s binary industry factor causes the model to treat any two non-matching industries as equally distant, regardless of real domain proximity (Chapter 5).
Evaluation sample caveat	The ranking evaluation’s sample was not drawn exclusively from the training pipeline’s held-out split, so its near-perfect scores may partly reflect memorization rather than generalization alone (Chapter 5).

Area	Limitation
Residual cross-lingual gap	Even within the platform’s two supported languages, French content translated to English retains a bounded semantic discrepancy relative to genuinely English-authored text (Chapter 5).
Infrastructure resilience	The model shares its process with the rest of the backend; a non-recoverable crash interrupts the entire application, and no health status specific to the matching model is independently queryable (Chapter 5).
Monitoring and retraining	No mechanism tracks data drift between live scraped listings or CVs and the synthetic training data, and no scheduled retraining pipeline exists (Chapter 5).
ATS suggestion integration gap	The ATS Agent’s dimension-by-dimension breakdown remains informational: the job-specific enhancement step re-extracts keywords independently rather than directly consuming it (Chapter 4, Section 4.3).
Testing automation	Validation relied on structured manual testing rather than automated unit, integration, or load-testing suites, as noted in this chapter’s Testing Strategy section; introducing an automated, continuously re-runnable test suite fulfilling the intent originally behind PB-22/US-22 (Chapter 6) is accordingly carried forward as a concrete direction for future work.

Tableau 7.2: Consolidated limitations identified across the system

Conclusion

This chapter has presented the system’s overall testing strategy, combining structured manual functional testing across all four agents with the fine-tuned matching model’s dedicated quantitative evaluation; consolidated the model’s regression, qualitative, and ranking results alongside a concrete before/after account of the CV Agent’s deterministic ATS scoring; and gathered, in a single place, the limitations identified throughout the project. The general conclusion that follows reflects on the project as a whole and outlines the perspectives these limitations open onto.

General Conclusion and Perspectives

Summary of the Work Accomplished

This report has documented the design, development, and evaluation of SUBUL, an AI-powered career assistant conceived to address the gap identified in Chapter 2 between the career guidance learners need and what they typically have access to: a system capable of turning a raw resume into an ATS-optimized profile, automatically discovering and ranking relevant job opportunities, quantifying job-specific compatibility, and guiding the learner toward the skills still missing for their target roles.

Chapter 3 established the system’s global, multi-agent architecture and justified the technology choices underlying it. Chapter 4 detailed the internal design of its three core agents – the CV Agent, the Job Agent, and the ATS Agent – together with the shared data model persisting their outputs (alongside the conversational assistant’s message history and the platform’s own learning data) and the prompt-engineering and anti-hallucination safeguards specifically constraining the CV Agent’s generative rewriting step. Chapter 5 examined, in isolation and in full depth, the fine-tuned bi-encoder model at the heart of the Job Agent’s matching capability, following the CRISP-ML(Q) cycle from its business rationale through to an honestly disclosed account of its remaining limitations. Chapter 6 described how this architecture was concretely built across five Scrum sprints, the reliability practices adopted along the way, and the resulting learner journey. Chapter 7 closed the technical narrative with the system’s overall testing strategy and a consolidated account of its evaluation results and identified limitations.

Collectively, these chapters demonstrate the design, implementation, and evaluation of a system that moves beyond a single generic scoring formula: a CV Agent that improves a resume against a reproducible, rule-based ATS benchmark; a Job Agent that combines a fine-tuned semantic model, achieving a 47.6-point score separation and a 0.9981 NDCG@5 against a non-learned baseline, with a deterministic skills-gap analysis; and an ATS Agent providing a fully auditable, dimension-by-dimension compatibility breakdown for any single job of interest.

Critical Reflection

This work was carried out under real constraints a single-developer project, a fixed internship duration, and limited computational resources for model training that shaped several of the choices documented throughout this report: the use of a synthetic, publicly available dataset in the absence of proprietary matching data (Chapter 5); the reliance on structured manual testing rather than

automated test suites (Chapter 7); and a Monitoring and Maintenance phase left, by the project's own CRISP-ML(Q) assessment, as the least mature stage of its lifecycle (Chapter 5). Rather than treating these constraints as failures to conceal, this report has consistently chosen to disclose them alongside the results they qualify, on the view that an honestly bounded result is more valuable, and more defensible, than an unqualified one.

Perspectives

The limitations identified throughout this project open several directions for future improvements, mainly related to model enhancement, system scalability, and operational maturity.

Enhancement of the Matching Model

- Extending the evaluation of the fine-tuned matching model on larger and more diverse datasets, including multilingual CVs and job descriptions, in order to further validate its generalization capabilities.
- Improving the semantic understanding of multilingual content by exploring more advanced multilingual embedding models and additional language adaptation strategies.
- Replacing the current binary industry matching factor with a continuous semantic similarity measure between industries, followed by regenerating the training dataset and retraining the fine-tuned model to improve its ability to distinguish closely related domains.

Beyond improving the matching model itself, future work should also strengthen the operational robustness and maintainability of the platform.

Infrastructure and MLOps Improvements

- Deploying the fine-tuned matching model as an independent scalable service with dedicated monitoring mechanisms, improving system availability and facilitating future maintenance.
- Introducing automated monitoring of model performance and data distribution changes, together with a continuous retraining pipeline to maintain the model's relevance over time.
- Automating the current manual end-to-end validation by introducing continuous unit, integration, end-to-end, and load-testing pipelines to improve software reliability and deployment readiness.

Platform Evolution

- Extending the conversational assistant capabilities by integrating voice and image-based interactions, enabling learners to communicate with the system through multiple modalities and receive richer

career assistance.

- Expanding the platform with additional career services, such as advanced interview preparation, continuous skill tracking, and long-term career progression analysis.

Final Remarks

The AI Career Assistant developed as part of the SUBUL platform demonstrates that combining reproducible evaluation, explainable scoring, and fine-tuned semantic matching can produce a practical and reliable career guidance system. Beyond its current implementation, the identified perspectives provide a clear roadmap for strengthening the platform’s scalability, multilingual capabilities, and operational maturity. The perspectives outlined above are not presented as afterthoughts, but as the natural continuation of the same standard of rigor applied throughout this project, carried forward into its next iteration.

Bibliography & Webography

- [1] Smartovate. « Official website and contact information ». Contact: contact@smartovate.com. [Accessed: date to be filled by author]. [Online]. Available: <https://www.smartovate.com/>
- [2] Microsoft Corporation. « Microsoft – official website ». [Online]. Available: <https://www.microsoft.com/>
- [3] Amazon Web Services, Inc. « Amazon web services – official website ». [Online]. Available: <https://aws.amazon.com/>
- [4] Google LLC. « Google – official website ». [Online]. Available: <https://www.google.com/>
- [5] NVIDIA Corporation. « Nvidia – official website ». [Online]. Available: <https://www.nvidia.com/>
- [6] IBM. « Ibm – official website », Accessed: Jul. 25, 2026. [Online]. Available: <https://www.ibm.com>
- [7] SUBUL. « Subul – plateforme d'apprentissage intelligente », Accessed: Jul. 25, 2026. [Online]. Available: <https://subul.uk/fr>
- [8] HolonIQ. « Global education technology market to reach \$404b by 2025 ». Accessed: July 25, 2026, Accessed: Jul. 25, 2026. [Online]. Available: <https://www.holoniq.com/notes/global-education-technology-market-to-reach-404b-by-2025>
- [9] Jobscan. « Ats statistics 2026: adoption and market data », Accessed: Jul. 25, 2026. [Online]. Available: <https://cvcraft.roynex.com/blog/ats-statistics-2026>
- [10] Enhancv. « Study reveals ats rarely auto-rejects cvs, debunks 75% myth », Accessed: Jul. 25, 2026. [Online]. Available: <https://itbrief.co.uk/story/study-reveals-ats-rarely-auto-rejects-cvs-debunks-75-myth>
- [11] Microsoft. « Autogen documentation », Accessed: Jul. 25, 2026. [Online]. Available: <https://microsoft.github.io/autogen/>
- [12] CrewAI. « Crewai documentation », Accessed: Jul. 25, 2026. [Online]. Available: <https://docs.crewai.com>
- [13] LangChain AI. « Langgraph documentation », Accessed: Jul. 25, 2026. [Online]. Available: <https://langchain-ai.github.io/langgraph/>
- [14] D. J. Anderson, *Kanban: Successful Evolutionary Change for Your Technology Business*. Blue Hole Press, 2010.

- [15] K. Beck, *Extreme Programming Explained: Embrace Change*. Addison-Wesley, 1999.
- [16] K. Schwaber and J. Sutherland, *The scrum guide*, 2020. [Online]. Available: <https://scrumguides.org>
- [17] P. Chapman et al., « CRISP-DM 1.0: step-by-step data mining guide », SPSS Inc., Tech. Rep., 2000.
- [18] U. Fayyad, G. Piatetsky-Shapiro, and P. Smyth, « From data mining to knowledge discovery in databases », *AI Magazine*, vol. 17, no. 3, pp. 37–54, 1996. [Online]. Available: <https://doi.org/10.1609/aimag.v17i3.1230>
- [19] Microsoft. « What is the team data science process? », Accessed: Jul. 25, 2026. [Online]. Available: <https://learn.microsoft.com/en-us/azure/architecture/data-science-process/overview>
- [20] S. Studer et al., « Towards CRISP-ML(Q): a machine learning process model with quality assurance methodology », *Machine Learning and Knowledge Extraction*, vol. 3, no. 2, pp. 392–413, 2021. DOI: 10.3390/make3020020
- [21] Python Software Foundation. « Python 3.12 documentation », Accessed: Jul. 25, 2026. [Online]. Available: <https://docs.python.org/3.12/>
- [22] FastAPI. « Fastapi documentation », Accessed: Jul. 25, 2026. [Online]. Available: <https://fastapi.tiangolo.com>
- [23] Uvicorn. « Uvicorn documentation », Accessed: Jul. 25, 2026. [Online]. Available: <https://www.uvicorn.org>
- [24] Meta. « React documentation », Accessed: Jul. 25, 2026. [Online]. Available: <https://react.dev>
- [25] Vercel. « Next.js documentation », Accessed: Jul. 25, 2026. [Online]. Available: <https://nextjs.org/docs>
- [26] Microsoft. « Typescript documentation », Accessed: Jul. 25, 2026. [Online]. Available: <https://www.typescriptlang.org/docs/>
- [27] GitHub. « Github », Accessed: Jul. 25, 2026. [Online]. Available: <https://github.com>
- [28] OpenAI. « Openai platform documentation », Accessed: Jul. 25, 2026. [Online]. Available: <https://platform.openai.com/docs>
- [29] Microsoft. « Microsoft azure », Accessed: Jul. 25, 2026. [Online]. Available: <https://azure.microsoft.com>

- [30] Reimers, Nils and Gurevych, Iryna. « Sentence-transformers documentation », Accessed: Jul. 25, 2026. [Online]. Available: <https://www.sbert.net>
- [31] PyTorch. « Pytorch documentation », Accessed: Jul. 25, 2026. [Online]. Available: <https://pytorch.org/docs/>
- [32] Microsoft. « Azure cosmos db documentation », Accessed: Jul. 25, 2026. [Online]. Available: <https://learn.microsoft.com/en-us/azure/cosmos-db/>
- [33] Microsoft. « Azure database for postgresql documentation », Accessed: Jul. 25, 2026. [Online]. Available: <https://learn.microsoft.com/en-us/azure/postgresql/>
- [34] Artifex. « Pymupdf documentation », Accessed: Jul. 25, 2026. [Online]. Available: <https://pymupdf.readthedocs.io>
- [35] python-docx. « Python-docx documentation », Accessed: Jul. 25, 2026. [Online]. Available: <https://python-docx.readthedocs.io>
- [36] ReportLab. « Reportlab – pdf generation library », Accessed: Jul. 25, 2026. [Online]. Available: <https://www.reportlab.com>
- [37] aiohttp. « Aiohttp documentation », Accessed: Jul. 25, 2026. [Online]. Available: <https://docs.aiohttp.org>
- [38] Crummy.com. « Beautiful soup documentation », Accessed: Jul. 25, 2026. [Online]. Available: <https://www.crummy.com/software/BeautifulSoup/>
- [39] SerpAPI. « Serpapi – google jobs api », Accessed: Jul. 25, 2026. [Online]. Available: <https://serpapi.com>
- [40] A. Barredo Arrieta et al., « Explainable Artificial Intelligence (XAI): concepts, taxonomies, opportunities and challenges toward responsible AI », *Information Fusion*, vol. 58, pp. 82–115, 2020. DOI: 10.1016/j.inffus.2019.12.012
- [41] L. Zheng et al., « Judging LLM-as-a-judge with MT-bench and Chatbot Arena », in *Advances in Neural Information Processing Systems*, vol. 36, 2023. [Online]. Available: <https://arxiv.org/abs/2306.05685>
- [42] M. Ozon. « Candidate-matching-synthetic dataset », Accessed: Jul. 25, 2026. [Online]. Available: <https://huggingface.co/datasets/michaelozon/candidate-matching-synthetic>
- [43] I. Loshchilov and F. Hutter, « Decoupled weight decay regularization », in *International Conference on Learning Representations (ICLR)*, 2019. [Online]. Available: <https://arxiv.org/abs/1711.05101>

- [44] L. N. Smith and N. Topin, « Super-convergence: very fast training of neural networks using large learning rates », in *Artificial Intelligence and Machine Learning for Multi-Domain Operations Applications*, 2019. [Online]. Available: <https://arxiv.org/abs/1708.07120>
- [45] K. Spärck Jones, « A statistical interpretation of term specificity and its application in retrieval », *Journal of Documentation*, vol. 28, no. 1, pp. 11–21, 1972.
- [46] K. Järvelin and J. Kekäläinen, « Cumulated gain-based evaluation of IR techniques », *ACM Transactions on Information Systems*, vol. 20, no. 4, pp. 422–446, 2002. DOI: 10.1145/582415.582418
- [47] ClickUp. « Clickup – project management platform », Accessed: Jul. 25, 2026. [Online]. Available: <https://clickup.com/>
- [48] M. Cohn. « User stories applied: for agile software development ». Origin of the Connextra user story template.
- [49] P. Lewis et al., « Retrieval-augmented generation for knowledge-intensive NLP tasks », in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474. [Online]. Available: <https://arxiv.org/abs/2005.11401>
- [50] scikit-learn. « Scikit-learn documentation », Accessed: Jul. 25, 2026. [Online]. Available: <https://scikit-learn.org>
- [51] « Xtendplex ». [Accès le 01/04/2020]. [Online]. Available: <https://www.xtendplex.com/>
- [52] « Image pour comprendre la méthode agile scrum ». [Accès le 02/04/2020]. [Online]. Available: <https://www.tuleap.org/fr/agile/comprendre-methode-agile-scrum-10-minutes/>
- [53] « Méthodologie scrum ». [Accès le 02/04/2020]. [Online]. Available: <https://bit.ly/3gGSEQQ>
- [54] « Bitbucket ». [Accès le 25/04/2020]. [Online]. Available: <https://fr.wikipedia.org/wiki/Bitbucket>
- [55] « Bref aperçu de jira ». [Accès le 25/04/2020]. [Online]. Available: <https://www.atlassian.com/fr/software/jira>
- [56] « Quest-ce que discord ». [Accès le 25/04/2020]. [Online]. Available: <https://www.pocket-lint.com/fr-fr/applications/actualites/151534-quest-ce-que-la-discorde-et-comment-utiliser-lapplication-gratuite-de-chat-textuel-voip>
- [57] « C'est quoi angular ? » [Accès le 25/04/2020]. [Online]. Available: <https://monpetitdev.fr/cest-quoi-angular-definition>

- [58] « Pring framework ». [Accès le 25/04/2020]. [Online]. Available: <https://spring.io/projects/spring-framework>
- [59] « Postman ». [Accès le 26/04/2020]. [Online]. Available: <https://blog.webnet.fr/presentation-de-postman-outil-multifonction-pour-api-web>
- [60] « Mysql database service ». [Accès le 26/04/2020]. [Online]. Available: <https://www.mysql.com/fr>
- [61] « Adobe xd ». [Accès le 26/04/2020]. [Online]. Available: <https://www.adobe.com/fr/products/xd.html>
- [62] « Présentation jwt ». [Accès le 26/04/2020]. [Online]. Available: <https://www.ionos.fr/digitalguide/sites-internet/developpement-web/json-web-token-jwt/>
- [63] « Présentation spring security ». [Accès le 26/04/2020]. [Online]. Available: <https://www.invivoo.com/securiser-application-spring-boot-spring-security/>
- [64] « Les modules angular ». [Accès le 26/04/2020]. [Online]. Available: <https://www.learn-angular.fr/les-modules-angular/>
- [65] « Architecture angular ». [Accès le 14/06/2020]. [Online]. Available: <https://www.apcpedagogie.com/structure-et-architecture-dun-projet-angular/>
- [66] « Patron singleton ». [Accès le 14/06/2020]. [Online]. Available: <https://www.refactoring.guru/fr/design-patterns/singleton>
- [67] « Patron etat ». [Accès le 14/06/2020]. [Online]. Available: <https://www.refactoring.guru/fr/design-patterns/state>
- [68] « Patron proxy ». [Accès le 15/06/2020]. [Online]. Available: <https://www.refactoring.guru/fr/design-patterns/proxy>
- [69] « Présentation jwt ». [Accès le 26/04/2020]. [Online]. Available: <https://www.ionos.fr/digitalguide/sites-internet/developpement-web/json-web-token-jwt/>
- [70] « Présentation spring security ». [Accès le 26/04/2020]. [Online]. Available: <https://www.invivoo.com/securiser-application-spring-boot-spring-security/>
- [71] .
- [72] M. S. Espinoza, J. Goncalves, P. Leita, J. L. G. Sanchez, and A. Herreros, « Inverse kinematics of a 10 dof modular hyper-redundant robot resorting to exhaustive and error-optimization methods: a comparative study », in *Robotics Symposium and Latin American Robotics Symposium (SBR-LARS), 2012 Brazilian*, IEEE, 2012, pp. 125–130.

- [73] M. S. Espinoza, A. I. Pereira, and J. Gonçalves, « Optimization methods for hyper-redundant robots' inverse kinematics in biomedical applications », in *AIP Conference Proceedings*, vol. 1479, 2012, p. 818.
- [74] M. Tham, *Writing research theses or dissertations*, University of Newcastle Upon Tyne, 2001. [Online]. Available: <http://lorien.ncl.ac.uk/ming/dept/Tips/writing/thesis/thesis-intro.htm>

Abstract

This report presents *AI Career Assistant*, an AI-powered career assistant developed as part of Smartovate's SUBUL platform to support learners throughout their job search journey. The solution relies on a multi-agent architecture orchestrated with LangGraph and powered by Large Language Models through Azure OpenAI. It integrates a CV Agent responsible for extracting, restructuring, and optimizing resumes according to Applicant Tracking System (ATS) best practices, a Job Agent that automatically collects, analyzes, and ranks job opportunities using a fine-tuned semantic matching model, and an ATS Agent providing explainable job-specific compatibility scoring. These services are complemented by a bilingual conversational assistant. The backend is implemented with FastAPI, Cosmos DB, and PostgreSQL, while the frontend is developed with Next.js. The project follows a hybrid Scrum/CRISP-ML(Q) methodology, combining software engineering practices with a structured machine learning lifecycle. The obtained results demonstrate that combining Large Language Models, semantic retrieval, and fine-tuned matching models enables the delivery of an explainable, reproducible, and personalized AI-based career assistance platform.

Keywords: Artificial Intelligence, Career Assistant, Multi-agent Architecture, LangGraph, Large Language Models, ATS, Semantic Matching, Fine-tuning, CRISP-ML(Q), Scrum

Résumé

Ce mémoire présente *AI Career Assistant*, un assistant de carrière basé sur l'intelligence artificielle développé dans le cadre de la plateforme SUBUL de Smartovate afin d'accompagner les apprenants dans leur recherche d'emploi. La solution repose sur une architecture multi-agents orchestrée par LangGraph et exploitant les grands modèles de langage via Azure OpenAI. Elle intègre un agent CV chargé d'extraire, restructurer et optimiser les curriculum vitae selon les bonnes pratiques des systèmes ATS, un agent Job responsable de la collecte, de l'analyse et du classement des offres d'emploi grâce à un modèle de matching sémantique fine-tuné, ainsi qu'un agent ATS fournissant une évaluation explicable de la compatibilité entre un CV et une offre d'emploi. L'ensemble est complété par un assistant conversationnel bilingue. Le backend est développé avec FastAPI et s'appuie sur Cosmos DB et PostgreSQL, tandis que le frontend est réalisé avec Next.js. Le développement a suivi une méthodologie hybride Scrum/CRISP-ML(Q), permettant de combiner les pratiques du génie logiciel avec un cycle de vie structuré pour les modèles d'intelligence artificielle. Les résultats obtenus démontrent qu'une approche combinant grands modèles de langage, recherche sémantique et modèles fine-tunés permet de fournir une assistance de carrière explicable, reproductible et personnalisée.

